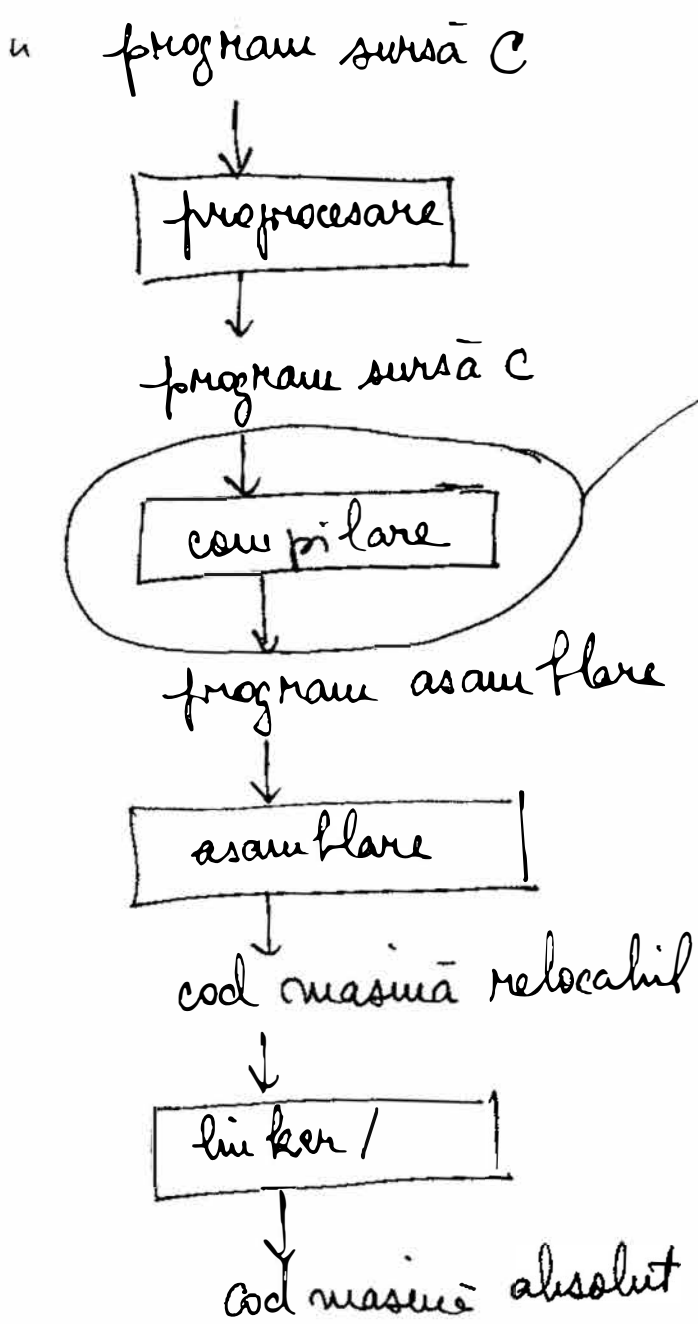
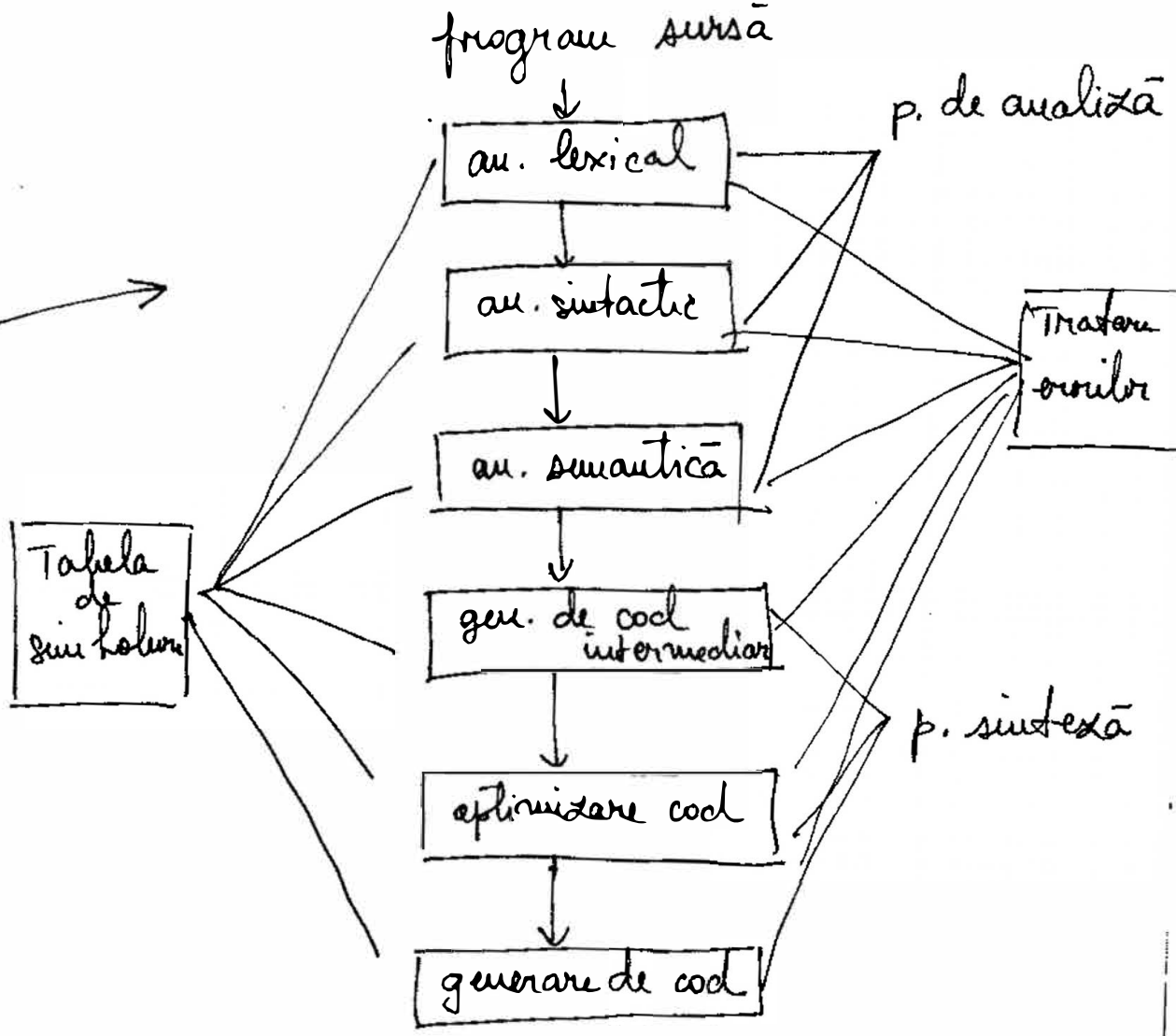
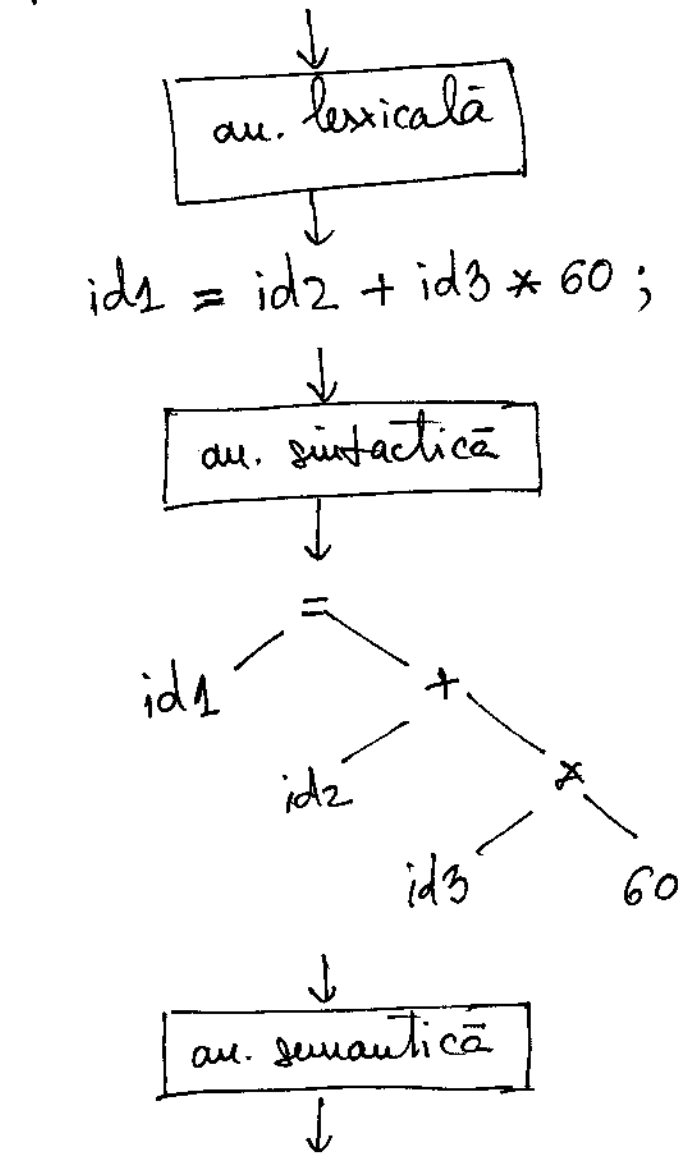
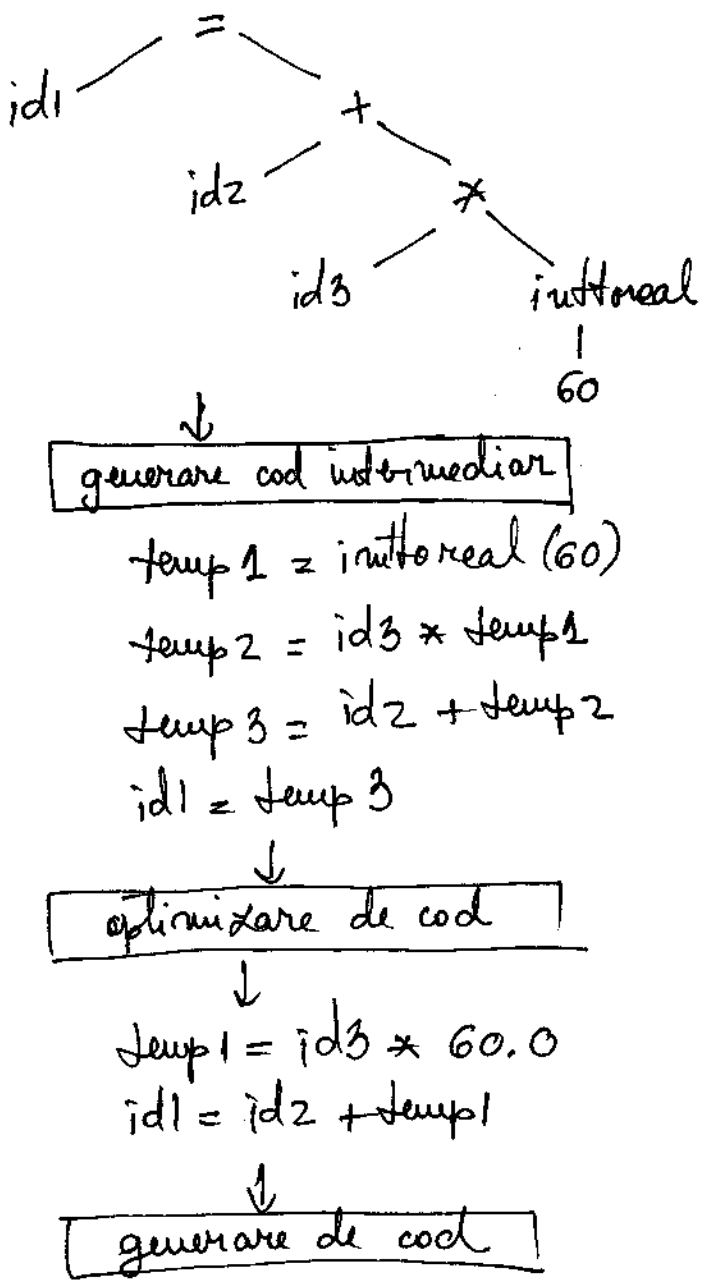

program sursă C
preprocesare
program sursă C
compilare
program asamblare
asamblare
cod mașină relocabil
linker /
cod mașină absolut


program sursă
an. lexical
an. sintactic
an. semantică
gen. de cod intermediar
optimizare cod
generare de cod
Tabela de simboluri
p. de analiză
Tratarea erorilor
p. sinteză

poziție = inițial + rata * 60;



poziție, inițial, rata → real



```
MOVF  id3, R2
MULF  #60.0, R2
MOVF  id2, R1
ADDF  R2, R1
MOVF  R1, id1
```

T. calculabilității – studiul matematic al Mașinilor de Calcul (M.C.) și al capab. lr

- Model < M.C.
  date le — șiruri de simboluri

Def.
Un alfabet este o mulțime finită de simboluri. (Notat Σ).

ex: alfabet romanesc $\{a, .., z\}$
alfabet binar $\{0, 1\}$

Def.
Un șir peste un alfabet este o secvență finită de simboluri din alfabet.

Notație

- ε → șirul vid
- Σ^* → mulțimea tuturor șirurilor peste Σ.

Def
Orice mulțime de șiruri peste un alfabet Σ, adică orice submulțime din Σ^*, este un limbaj (Σ^*, $\emptyset$, Σ sunt limbaje).

Specificarea limbajelor

- Limbaj → mulțime → specificare mulțime
  - Limbaj → finit → enumerarea elementelor
  - Limbaj → infinit → prin proprietăți, $L = \{w \in \Sigma^* \mid w \text{ are proprietatea } P\}$.

Operații cu limbaje

- operații specifice mulțimilor $\cup, \cap, \setminus$
- operații specifice:

Def - concatenare
L_1, L_2 - limbaje peste alf. Σ, concatenarea lor

$$L = L_1 \cdot L_2 \text{ sau } L = L_1 L_2, \text{ unde}$$

$$L = \{w \mid w = x \cdot y, x \in L_1, y \in L_2\}$$

Def - închiderea tranzitivă (Kleene Star)

L^* este mulțimea tuturor șirurilor obținute prin concatenarea a 0 sau mai multe șiruri din L. (concatenarea a 0 șiruri este ε, concatenarea unui șir este acel șir).

$$L^* = \{ w \in \Sigma^* \mid w = w_1 \circ w_2 \circ \dots \circ w_k,\ k \geq 0,\ w_1, \dots, w_k \in L \}.$$

ex:

$\Sigma = \{0, 1\}$

$L_1 = \{ w \in \Sigma^* \mid w$ are un număr par de $0 \}$

$L_2 = \{ w \in \Sigma^* \mid w$ începe cu 0 și restul simbolurilor sunt $1 \}$

$L = L_1 \circ L_2 = \{ w \in \Sigma^* \mid w$ are număr impar de $0 \}$

Obs:

Utilizarea notației Σ^* pt a reprezenta mulțimea tuturor șirurilor peste Σ este consistentă cu notație Kleene Star asupra Σ privit ca un limbaj finit.

Notație $L^+ = LL^*$

$$L^+ = \{ w \mid w = w_1 \circ w_2 \circ \dots \circ w_k,\ k \geq 1,\ w_1 \dots w_k \in L \}.$$

Reprezentarea finită a limbajelor

Limbaj → finit → enumerarea elem.
↘ infinit → ?

reprezentare finită – șir de simboluri peste un alf. Σ.

Σ^* → numărabilă $\Rightarrow f : \mathbb{N} \to \Sigma^*$

1) $\forall k \geq 0$, enumăr șirurile de lg. k înaintea celor de lg. $k+1$
2) n^k enumerate lexicografic

mulțimea limbajelor $\Rightarrow \mathcal{P}(\Sigma^*) \overset{Not}{=} 2^{\Sigma^*}$

Def – mulțime putere → mulțimea tuturor submulțimilor care se pot forma pe o mulțime dată.

ex: $A = \{a, b\}$

$2^A = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$.

Primul rezultat al T. Calculabilității

Oricât de puternice ar fi metodele de reprezentare, numai o mulțime numărabilă de limbaje poate fi reprezentată, atât timp cât reprezentarea este finită.

Expresii Regulate (E.R.)

Def.

E.R. peste un alfabet Σ sunt șiruri peste $\Sigma \cup \{(,), \emptyset, \cup, *\}$ ce

1° $\emptyset$, $a \in \Sigma \rightarrow$ E.R.

2° α, β E.R $\rightarrow (\alpha\beta)$ E.R.

3° α, β E.R. $\rightarrow (\alpha \cup \beta)$ E.R.

4° α E.R $\rightarrow \alpha^*$ E.R.

5° Nimic altceva nu este o E.R.

Obs.

O E.R. reprezintă un limbaj, dacă $\cup \rightarrow$ reuniunea, $*$ – Kleene Star

Formal, relația E.R. $\rightarrow$ limbaje, $L : \Sigma^* \rightarrow 2^{\Sigma^*}$

Def. L

1° $L(\emptyset) = \emptyset$, $\forall a \in \Sigma$, $L(a) = \{a\}$

2° α, β E.R, $L((\alpha\beta)) = L(\alpha)\,L(\beta)$

3° α, β E.R, $L((\alpha \cup \beta)) = L(\alpha) \cup L(\beta)$

4° $L(\alpha^*) = L(\alpha)^*$

ex:

? $L(((a \cup b)^* a))$

$L(((a \cup b)^* a)) = L((a \cup b)^*) L(a)$ (2)

$= L((a \cup b)^*) \{a\}$ (1)

$= L((a \cup b))^* \{a\}$ (4)

$= (L(a) \cup L(b))^* \{a\}$ (3)

$= (\{a\} \cup \{b\}^*)^* \{a\}$ (1)

$\overset{Not}{=} \{a, b\}^* \{a\}$ → $\{a, aa, abba, \dots\}$

$= \{w \in \{a, b\}^* \mid w \text{ se termină cu } a\}$

Obs:

Orice limbaj care poate fi reprezentat printr-o E.R, poate fi reprezentat printr-o infinitate de E.R.

ex:

α, $(\alpha \cup \emptyset)$ reprez. ac limbaj

$((\alpha \cup \beta) \cup \gamma)$ și $(\alpha \cup (\beta \cup \gamma))$

$((\alpha\beta)\gamma$ și $(\alpha(\beta\gamma))$

Def Clasa limbajelor regulate ($\mathcal{R}$)

Clasa limbajelor regulate peste un alfabet Σ este multimea minimă de limbaje ce conține multimile $\{a\}$, $a \in \Sigma$ și care este închisă în raport cu operațiile de reuniune, concatenare, Kleene Star.

Proprietăți $\mathcal{R}$:

1° $\emptyset \in \mathcal{R}$, $\forall a \in \Sigma$, $\{a\} \in \mathcal{R}$

2° $A, B \in \mathcal{R}$, $A \cup B$, $A \cdot B$, $A^* \in \mathcal{R}$

3° Dc. $\mathcal{S}$ este o multime de limbaje care conține $\emptyset$, $\forall a \in \Sigma$, $\{a\} \in \mathcal{S}$ și este închisă în raport cu operațiile de $\cup$, $\cdot$, Kleene Star, atn. $\mathcal{R} \subseteq \mathcal{S}$.

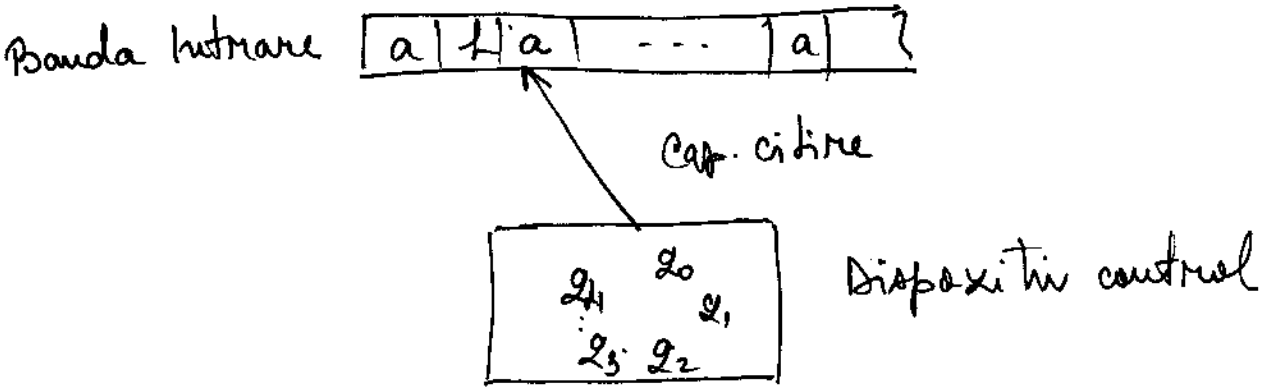
Cum $\mathcal{R}$ → definită prin proprietăți de închidere → minimă clasă

Un limbaj este regulat dacă și numai dacă este descris printr-o E.R.

Automate finite deterministe (AFD)

Un A.F. → model restrictiv al unui calc (UC, mem, IO)

Teoria A.F. → complexă și elegantă
→ aplicabilitate în proiectarea unor tipuri clasice de alg și programe:
au. lexicale, alg. de regăsire a unui subșir într-un șir



Def.

Un AFD este un tuplu $M = (K, \Sigma, \delta, A, F)$

K → mulțimea finită a stărilor,

Σ → alf. de intrare,

$A \in K$ → st. inițială,

$F \subseteq K$ → mulțimea st. finale,

$\delta : K \times \Sigma \to K$, funcția de tranziție

Dc. M, în starea $q \in K$, citește $\sigma \in \Sigma$, atl. $\delta(q, \sigma) \in K$ este starea unic determinată în care ajunge automatul.

Def.
Configurația unui AFD $(K, \Sigma, \delta, s, F)$ este un element din $K \times \Sigma^*$.

Def.
Trecerea între configurații

$\vdash_M : K \times \Sigma^* \to K \times \Sigma^*$

$(q, w) \vdash_M (q', w')$, (q, w), (q', w') config. posibile

$\Leftrightarrow w = \sigma w'$, $\sigma \in \Sigma$,

$\delta(q, \sigma) = q'$

$(q, \varepsilon) \to$ semnifică faptul că M a citit toate simbolurile din intrare.

Notație
$\vdash_M^*$ închiderea reflexivă și tranzitivă a $\vdash_M$

$(q, w) \vdash_M^* (q, w')$ trece în 0 sau m. multi pași

Def.
Un $w \in \Sigma^*$ este acceptat de $M \Leftrightarrow \exists q \in F$ a.î. $(s, w) \vdash_M^* (q, e)$.

Def.
Limbajul $L(M)$ acceptat de M este mulțimea șirurilor acceptate de M.

ex:

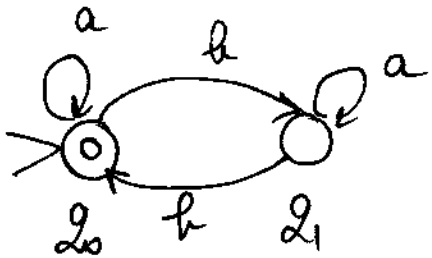
AFD, $M = (K, \Sigma, \delta, s, F)$

$K = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$s = q_0$

$F = \{q_0\}$



reprezentare prin diagrame de tranziție

δ:

q	σ	$\delta(q, \sigma)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_1

ex:

? M a.î. $L(M) = \{w \mid w \in \{a, b\}^*$, w nu conține 3 b-uri consecutive$\}$.

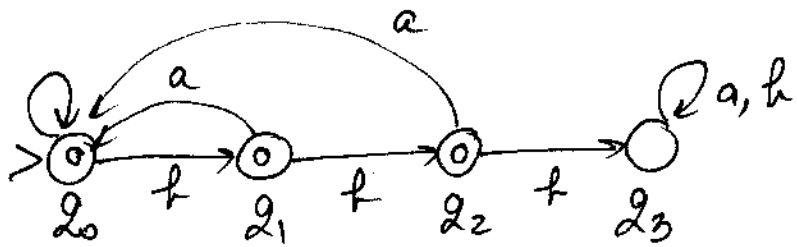
$M = \{K, \Sigma, \delta, s, F\}$

$K = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$s = q_0$

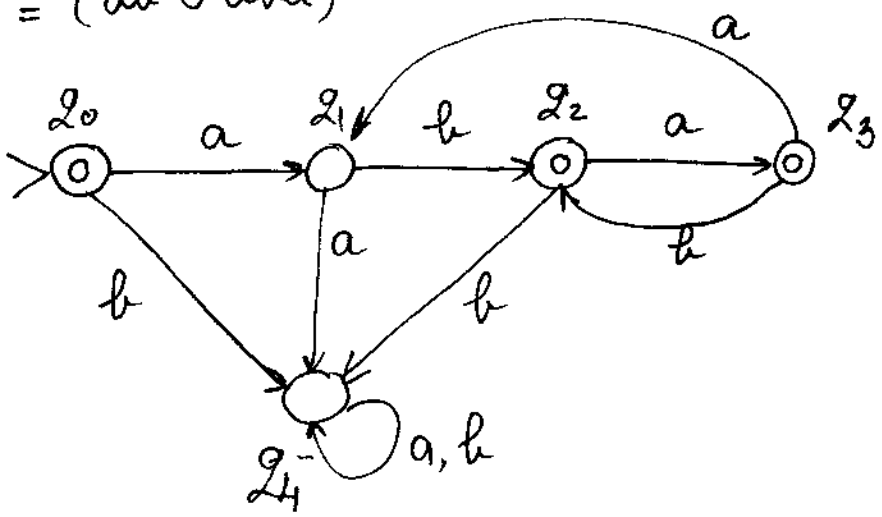
$F = \{q_0, q_1, q_2\}$



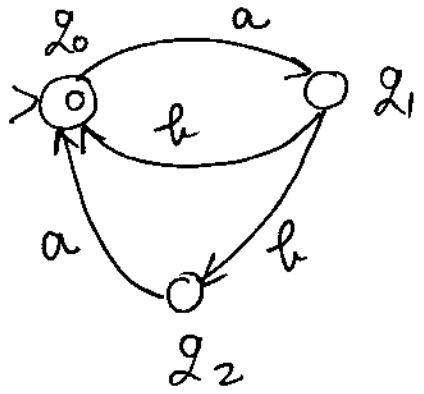
Automate Finite Nedeterministe

Nedeterminismul - proprietatea de a schimba starile intr-un mod care este partial determinat de st. curentă & simbolul de intrare.

ex: $L = (ab \cup aba)^*$



$\Rightarrow$ eliminăm restrictia: din fiecare stare să $\exists$ tranziții pe fiec. simbol din alfabet.

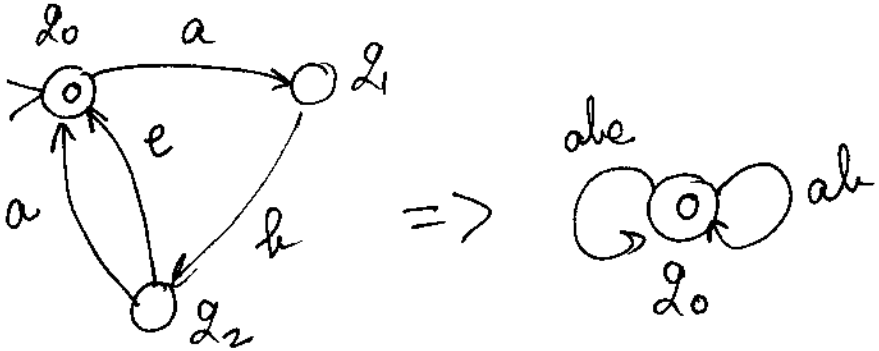


ex: ab $\xrightarrow{\text{acceptat}}$ $q_0 \to q_1 \to q_0$

abab $\xrightarrow{\text{acceptat}}$ $q_0 \to q_1 \to q_2 \to q_0$.

Poate face o alegere greșită : abab : $q_0 \to q_1 \to q_0 \to q_1$

$abb \notin L$ => $\nexists$ ciclu cale care să permită acceptarea șirului



Def.

Un AFN este un tuplu $M = (K, \Sigma, \Delta, s, F)$, unde

K - mulțimea finită a stărilor

Σ - alfabetul de intrare

$s \in K$ - starea inițială

$F \subseteq K$ - mulțimea stărilor finale

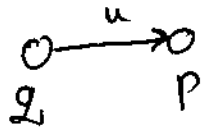
Δ - relația de tranziție

$$\Delta \subseteq K \times \Sigma^* \times K$$

↑ finită

$(q, u, p) \in \Delta \overset{\text{semnificație}}{\Longrightarrow}$ M în starea 'q' poate citi șirul de intrare 'u' și intra în starea 'p'

$\Updownarrow$



Def
O configurație a unui AFM, M, este un element din $K \times \Sigma^*$.

Def:
Relația dintre configurații, $\vdash_M$:
$(q, w) \vdash_M (q', w') \iff \exists\, u \in \Sigma^*$ a.î. $w = uw'$ și $(q, u, q') \in \Delta$

Obs:
$\vdash_M \Rightarrow$ ar trebuie să fie funcție

Notație
$\vdash_M^*$ închiderea reflexivă și tranzitivă a lui $\vdash_M$

Def.
Și rarul $w \in \Sigma^*$ este acceptat de AFM, M, dc. și numai dc. $\exists\, q \in F$ a.î. $(s, w) \vdash_M^* (q, e)$.

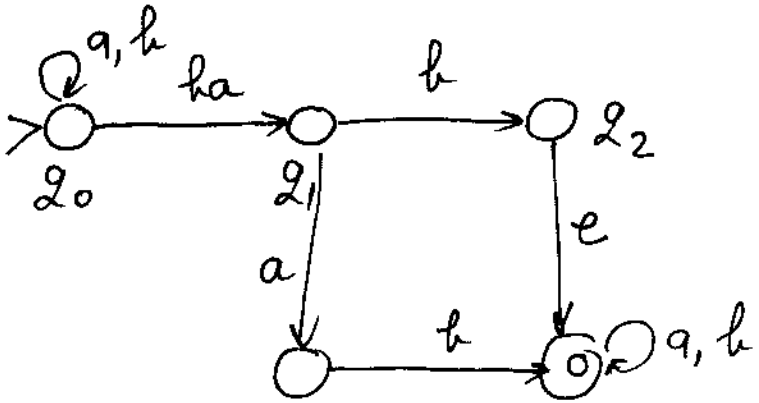
Def.
Limbajul acceptat de un AFM este mulțimea tuturor șirurilor acceptate de automat.

ex:
? M, $L = \{w \in \{a,b\}^* \mid w$ conține cel puțin o apariție a șirului tab sau $taab\}$.

$M = (K, \Sigma, \Delta, s, F)$

$K = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{a, b\}$



$(q_0, taabtabaab) \vdash_M (q_0, aabtabaab) q_3$

$\vdash_M (q_0, abtabaab)$

$\vdash_M (q_0, e)$

$(q_0, taahabaah) \vdash_M (q_1, ahabaah)$

$\vdash_M (q_3, habaah)$

$\vdash_M (q_4, abaah)$

$\vdash_M (q_4, baah)$

$\vdash_M (q_4, aah)$

$\vdash_M (q_4, ah)$

$\vdash_M (q_4, h)$

$\vdash_M (q_4, e)$

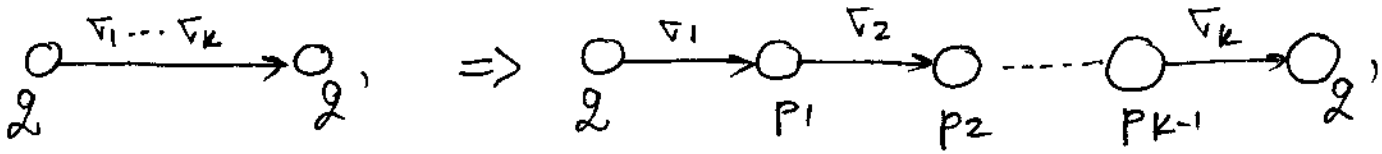
Echivalența AFD și AFN

Def.
Automatele finite M_1 și M_2 sunt echivalente, dc și numai dc. $L(M_1) = L(M_2)$.

Teorema
Pentru fiecare AFN există un AFD echivalent.

Dem
Fie $M = (K, \Sigma, \Delta, s, F)$
Pt. a transforma M într-un AFD echivalent, trebuie eliminate
→ tranzițiile de forma $(q, u, q') \in \Delta$ cu $u = \varepsilon$ sau $|u| > 1$
→ tranziții multiple aplicabile aceleași config.
→ tranziții care lipsesc.

Elimin. tranzițiilor (q, u, q'), $|u| > 1$. => introduc noi stări
noi tranziții



Formal, $(q, \sigma_1 \dots \sigma_k, q') \in \Delta$, $\sigma_1, \dots, \sigma_k \in \Sigma$, $k \geq 2$

adaug stările <u>nefinale</u> $p_1, \dots, p_{k-1}$ la K și noi tranziții

$(q, \sigma_1, p_1), (p_1, \sigma_2, p_2), \dots, (p_{k-1}, \sigma_k, q')$ la Δ.

Fie $M' = (K', \Sigma, \Delta', s', F')$ AFN rezultat prin transf.

Vrem să construim un AFD $M'' = (K'', \Sigma, \delta'', s'', F'')$ echivalent cu M'.

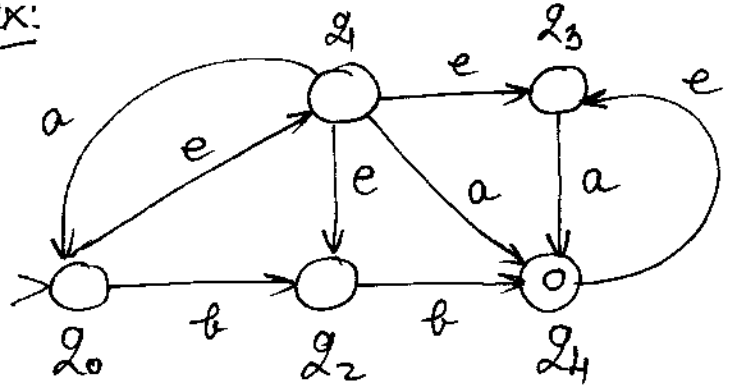
<u>Idea</u>: Vedem AFN ca având la fiecare moment <u>nu</u> o stare, ci o mulțime de stări, respectiv toate stările care pot fi atinse din st. curentă prin intermediul intrări parcurse.

$\Rightarrow K'' \subseteq 2^{K'}$

<u>Problema</u> tranzițiilor pe simul vid, AFN

$$E(q) = \{p \in K' \mid (q, e) \overset{*}{\vdash}_{M'} (p, e)\}$$

ex:



$E(q_0) = \{q_0, q_1, q_2, q_3\}$

$E(q_1) = \{q_1, q_2, q_3\}$

$E(q_2) = \{q_2\}$

$E(q_3) = \{q_3\}$

$E(q_4) = \{q_3, q_4\}$

Definim $M'' = (K'', \Sigma, \delta'', s'', F'')$

$K'' \subseteq 2^{K'}$

$s'' = E(s')$

$F'' = \{Q \subseteq K' \mid Q \cap F' \neq \emptyset\}$

$\forall Q \subseteq K', \forall \sigma \in \Sigma$, $\delta''(Q,\sigma) = \cup \{ E(p) \mid p \in K' \text{ și } (q,\sigma,p) \in \Delta', q \in Q \}$

Vreau să arăt că M'' este determinist și echivalent cu M'.

$M'' \to$ determinist din construcție, δ'' este o funcție,

$\delta''(Q,\sigma) = \emptyset$, adevărat, dar $\emptyset \subseteq 2^{K'}$, $\emptyset \in K''$

Echivalența revine la a demonstra, $\forall w \in \Sigma^*$ și $\forall p, q \in K'$,

$$(q,w) \vdash_{M'}^{*} (p,e) \iff (E(q),w) \vdash_{M''}^{+} (P,e), \ p \in P.$$

$\forall w \in \Sigma^*$, $w \in L(M') \iff (s',w) \vdash_{M'}^{*} (f',e)$, $f' \in F'$

$$(E(s'),w) \vdash_{M''}^{*} (Q,e), \ f' \in Q$$

$\iff$

$$(s'',w) \vdash_{M''}^{*} (Q,e), \ f' \in Q, \ Q \in F''$$

$\iff$

Dem $\Rightarrow$ prin inducție după $|w|$.

Pasul de bază

$|w| = 0$, $w = e$

Vreau să arăt $(q, e) \vdash_{M'}^{*} (p, e) \iff (E(q), e) \vdash_{M''}^{*} (\mathcal{I}, e)$, $p \in \mathcal{P}$

$(q, e) \vdash_{M'}^{*} (p, e)$: $p \in E(q)$

$(E(q), e) \vdash_{M''}^{*} (\mathcal{I}, e)$: $E(q) = \mathcal{I}$, $p \in \mathcal{P}$

$E(q) = \mathcal{I}$, $p \in \mathcal{P}$: $p \in E(q)$

Ipoteza inductivă

Pp. adev. pt șiruri w, $|w| \leq k$, $k \geq 0$

Pas de inducție

$\forall w$, $|w| = k+1$.

$\forall w \in \Sigma^*,\ w \in L(M) \iff w \in L(M'')$

$\forall w \in \Sigma^*,\ (q, w) \overset{*}{\vdash}_{M'} (p, e) \iff (E(q), w) \overset{*}{\vdash}_{M''} (P, e),\ p \in P$

$\forall q, p \in K'$,

$$(s', w) \overset{*}{\vdash}_{M'} (f', e) \iff (E(s'), w) \overset{*}{\vdash}_{M''} (Q, e),\ f' \in Q$$

$f' \in F'$ — $w \in L(M')$

$w \in L(M'')$

Dem → prin inducție după $|w|$

Pas de bază

$|w| = 0,\ w = e$

Ip. inductivă

pp. adev. pt w, $|w| \leq k$

Pas de inductie

$\forall w,\ |w| = k+1.$

$w = va,\ a \in \Sigma,\ v \in \Sigma^*$

$\Rightarrow$ pp $(q, w) \overset{*}{\underset{M'}{\vdash}} (p, e) \Rightarrow \exists\, r_1, r_2 \in K'$ a.î.

$$(q, va) \overset{*}{\underset{M'}{\vdash}} (r_1, a) \underset{M'}{\vdash} (r_2, e) \overset{*}{\underset{M'}{\vdash}} (p, e)$$

Cum $(q, va) \overset{*}{\underset{M'}{\vdash}} (r_1, a) \Rightarrow (q, v) \overset{*}{\underset{M'}{\vdash}} (r_1, e)$

Dar $|v| = k \overset{\text{ip. ind.}}{\Longrightarrow} (E(q), v) \overset{*}{\underset{M''}{\vdash}} (R_1, e),\ r_1 \in R_1.$

Cum $(r_1, a) \underset{M'}{\vdash} (r_2, e) \Rightarrow (r_1, a, r_2) \in \Delta' \Rightarrow$ din construcția M''

$E(r_2) \subseteq \delta''(R_1, a)$

$(r_2, e) \overset{*}{\underset{M'}{\vdash}} (p, e) \Rightarrow p \in E(r_2) \Rightarrow p \in \delta''(R_1, a)$

$(R_1, a) \underset{M''}{\vdash} (P, e),\ p \in P \Rightarrow (E(q), va) \overset{*}{\underset{M''}{\vdash}} (R_1, a) \underset{M''}{\vdash} (P, e)$

$\Leftarrow$

Pp. $(E(q), va) \vdash_{M''}^{*} (R_1, a) \vdash_{M''} (P, e)$ $\forall$ un anumit P ai $p \in P$;

un anumit R_1 ai $\delta''(R_1, a) = P$.

Din def. δ'', $\delta''(R_1, a)$ este reuniunea tuturor multimilor $E(r_2)$

unde $\forall$ o anumită stare $r_1 \in R_1$, (r_1, a, r_2) este o tranziție a lui M'.

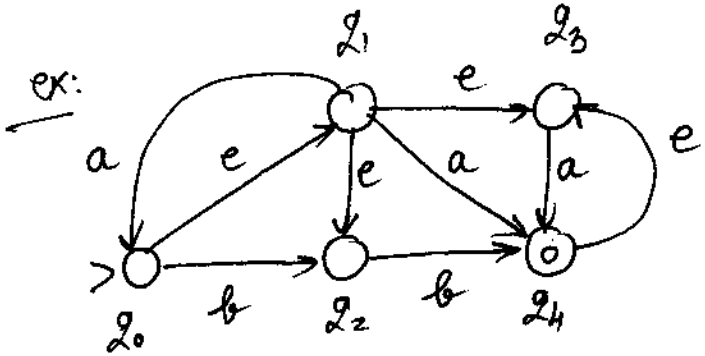
Cum $p \in P = \delta''(R_1, a)$, $\exists\, r_2$ ai $p \in E(r_2)$, $\forall$ un anumit $r_1 \in R_1$,

(r_1, a, r_2) este o tranziție a lui M'. $\Rightarrow (r_2, e) \vdash_{M'}^{*} (p, e)$ din $E(r_2)$

Din ip. ind. $\Rightarrow (q, v) \vdash_{M'}^{*} (r_1, e) \Rightarrow (q, va) \vdash_{M'}^{*} (r_1, a) \vdash_{M'} (r_2, e) \vdash_{M'}^{*} (p, e)$

<u>q.e.d</u>

ex:



$\delta''(Q,\tau) = \cup \{ E(p) \mid p \in K', (q,\tau,p) \in \Delta', q \in Q \}$

$E(q_0) = \{q_0, q_1, q_2, q_3\} = Q_0 = \Delta''$

$E(q_1) = \{q_1, q_2, q_3\}$

$E(q_2) = \{q_2\}$

$E(q_3) = \{q_3\}$

$E(q_4) = \{q_3, q_4\}$.

$\delta''(Q_0, a) = E(q_0) \cup E(q_4) = \{q_0, q_1, q_2, q_3, q_4\} = Q_1$.

$\delta''(Q_0, b) = E(q_2) \cup E(q_4) = \{q_2, q_3, q_4\} = Q_2$

$\delta''(Q_1, a) = Q_1$

$\delta''(Q_1, b) = Q_2$

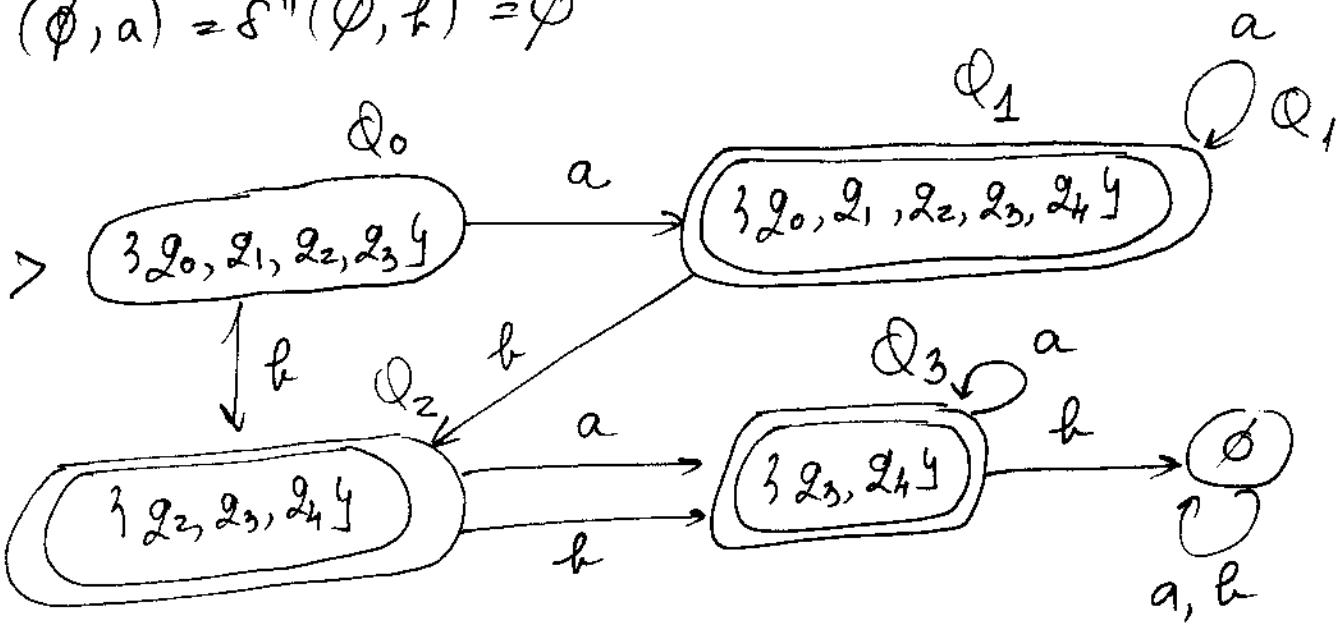
$\delta''(Q_2, a) = E(q_4) = \{q_3, q_4\} = Q_3$

$\delta''(Q_2, b) = E(q_4) = Q_3$

$\delta''(Q_3, a) = Q_3$

$\delta''(Q_3, b) = \emptyset$

$\delta''(\emptyset, a) = \delta''(\emptyset, b) = \emptyset$



Proprietăți ale limbajelor acceptate de A-Finite

Teorema

Clasa limbajelor acceptate de A-Finite este închisă în raport cu operațiile:

a) reuniune
b) concatenare
c) Kleene Star
d) complementare
e) intersecție

Dem:

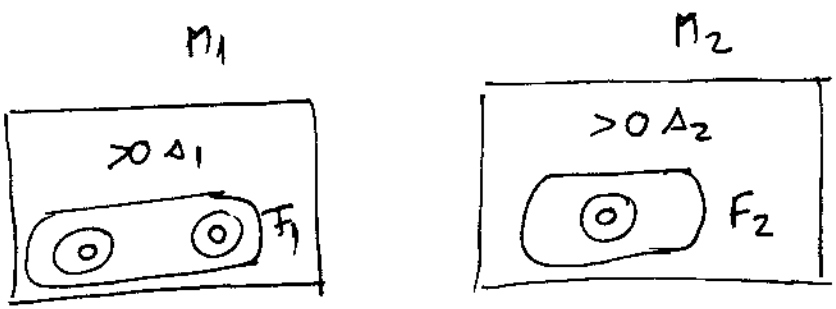
a) $L_1 = L(M_1)$, $L_2 = L(M_2)$

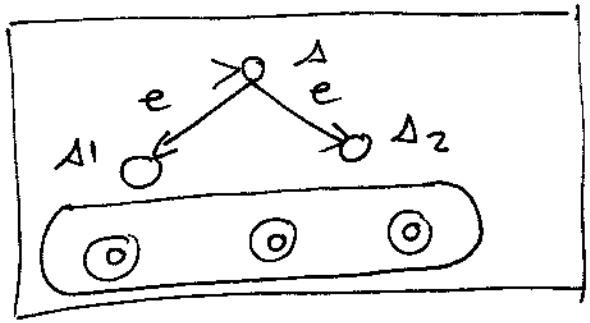
$M_1 = (K_1, \Sigma, \Delta_1, \Delta_1, F_1)$

$M_2 = (K_2, \Sigma, \Delta_2, \Delta_2, F_2)$

$K_1 \cap K_2 = \emptyset$

? M a.î. $L(M) = L(M_1) \cup L(M_2)$





$M = (K, \Sigma, \Delta, s, F)$, s este o noua stare $s \notin K_1 \cup K_2$

$K = K_1 \cup K_2 \cup \{s\}$

$F = F_1 \cup F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}$.

$w \in \Sigma^*$, $(s, w) \vdash_M^* (q, e)$, $q \in F \Leftrightarrow$

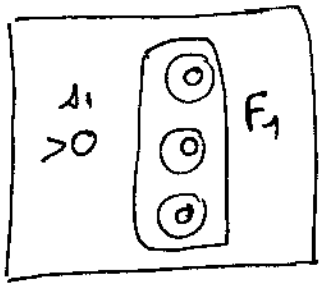
$(s_1, w) \vdash_{M_1}^* (q, e)$, $q \in F_1$ sau $(s_2, w) \vdash_{M_2}^* (q, e)$, $q \in F_2$

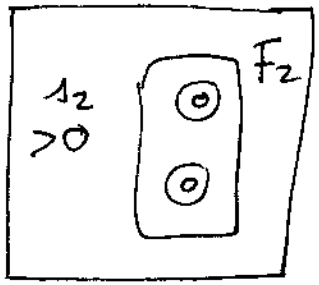
$L(M) = L(M_1) \cup L(M_2)$

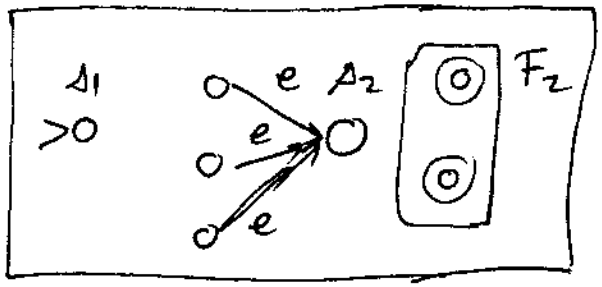
f) Concatenare

$M_1, M_2 \rightarrow$ AFN

? M a.î. $L(M) = L(M_1) \cdot L(M_2)$







$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

$M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$, $K_1 \cap K_2 = \emptyset$

$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup K_2$

$s = s_1$

$F = F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup (F_1 \times \{e\} \times \{s_2\})$

$(s, w) \vdash_M^* (q, e)$, $q \in F \iff \exists\, w_1, w_2 \in \Sigma^*$, $\exists\, p \in F_1$ a.î.

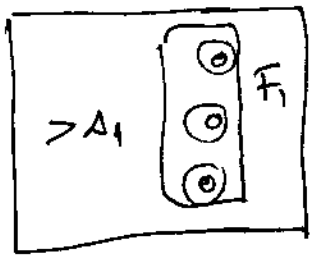
$w = w_1 w_2$ ş.i. $(s_1, w_1) \vdash_{M_1}^* (p, e)$ ş.i. $(s_2, w_2) \vdash_{M_2}^* (q, e)$

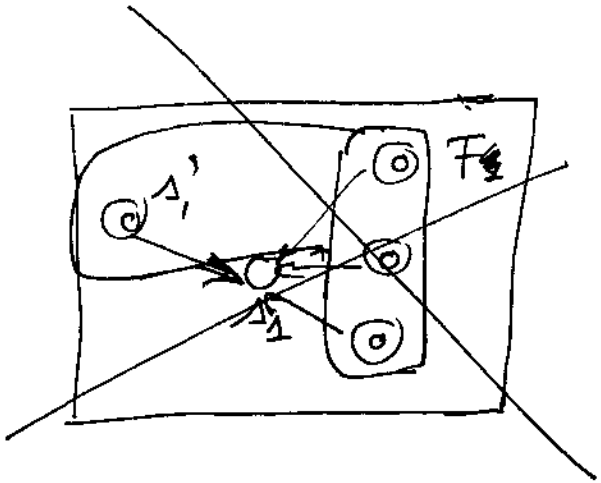
$$L(M) = L(M_1) \cdot L(M_2)$$

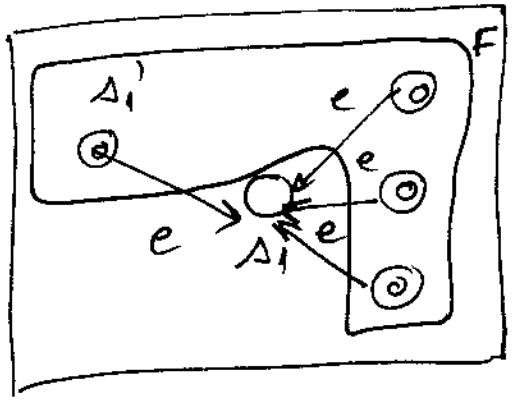
c) Kleene Star

$M_1 \rightarrow$ AFN

? M a.î. $L(M) = L(M_1)^*$







$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup \{s_1'\}$

$s = s_1'$

$F = F_1 \cup \{A_1'\}$

$\Delta = \Delta_1 \cup (F \times \{e\} \times \{A_1\}) \cup \{(A_1', e, A_1)\}$.

$w \in L(M)$, fie $w = e$, fie $w = w_1 \circ \dots \circ w_k$, $k \geq 1$.

$i = 1, \dots, k$, $\exists f_i \in F$ a.î. $(A_1, w_i) \overset{*}{\underset{M_1}{\vdash}} (f_i, e)$ $\Rightarrow w \in L(M_1^*)$.

$w \in L(M_1)^*$, adică fie $w = e$ sau $w = w_1 \dots w_k$, $w_1, \dots, w_k \in L(M_1)$

$w = e \Rightarrow w \in L(M)$ pt că A_1' este st. finală

$w \in L(M)$ pt că $f_1, \dots, f_k \in F_1$

$(A_1', w_1 \dots w_k) \overset{*}{\underset{M}{\vdash}} (\dots \overset{*}{\underset{M}{\vdash}} (f_k, e) \Rightarrow L(M) = L(M_1)^*$.

d) Complementare

$M = (K, \Sigma, \delta, A, F)$ AFD.

Σ^* – $L(M)$ acceptat de AFD $\bar{M} = (K, \Sigma, \delta, A, K - F)$

e) Intersecție

L_1, L_2 ai $L_1 = L(M_1)$, $L_2 = L(M_2)$

$$L_1 \cap L_2 = \bar{\Sigma}^* - ((\underbrace{\bar{\Sigma}^* - L_1}_{\text{AFD}}) \cup (\underbrace{\bar{\Sigma}^* - L_2}_{\text{AFD}}))$$

AFD AFD

AFN

↓ AFD

AFD.

Construcție directă

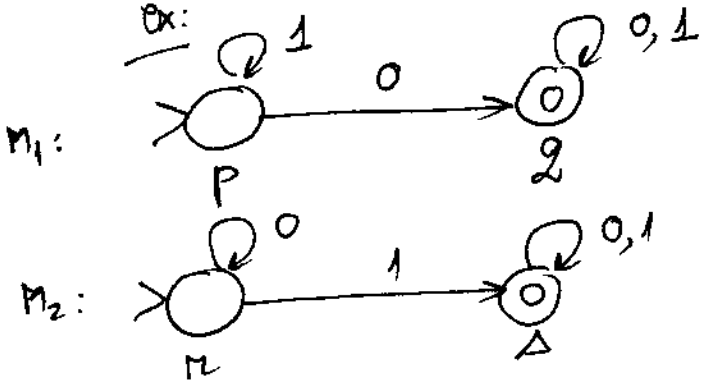
$M_1 = (K_1, \bar{\Sigma}, \delta_1, s_1, F_1)$

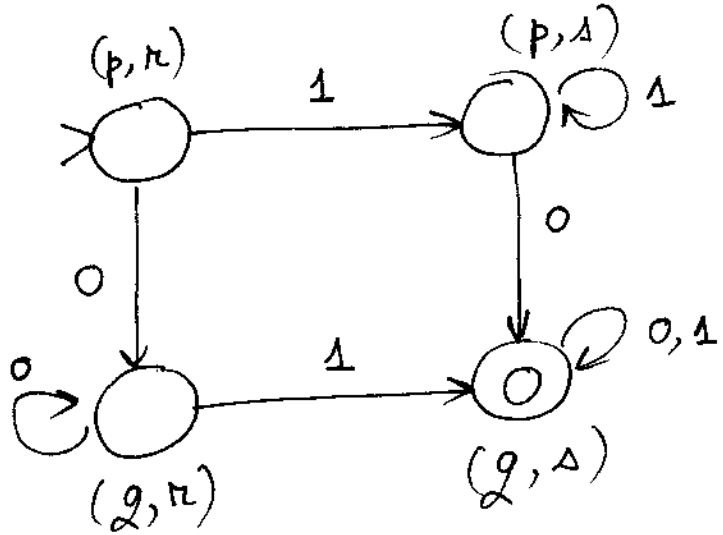
$M_2 = (K_2, \bar{\Sigma}, \delta_2, s_2, F_2)$

$M = (K_1 \times K_2, \bar{\Sigma}, \delta, (s_1, s_2), F_1 \times F_2)$

$\delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a))$

Ex:





Proprietăți algoritmice

Teorema

Există algoritmi care pot răspunde la urm. întrebări referitoare la $\mathcal{L}$(A.F.):

a) Dat. fiind un A.F. M și un șir w, este $w \in L(M)$?

b) A.F. M, $L(M) = \emptyset$?

c) A.F. M, $L(M) = \Sigma^*$?

d) A.F. M_1, M_2, $L(M_1) \subseteq L(M_2)$?

e) A.F. M_1, M_2, $L(M_1) = L(M_2)$?

Dem. pp. $M \to$ AFD.

a) trasez oglinda lui M pt șirul w, în $|w|$ pași.

b) verific dacă $\exists$ o secvență de săgeți care să unească într-o st finală

c) construiesc M' a.î. $L(M') = \Sigma^* - L(M)$ și verific $L(M') = \emptyset$

d) $(\Sigma^* - L(M_2)) \cap L(M_1) = \emptyset$

Automate finite și expresii regulate

$a^* b^*$
$\{a, b\}^*$

Teorema

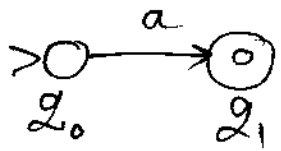
Un limbaj este regulat $\Leftrightarrow$ acceptat de un A.Finit.

Dem.

$\Rightarrow$ (l. regulat $\Rightarrow$ L(A.F.))

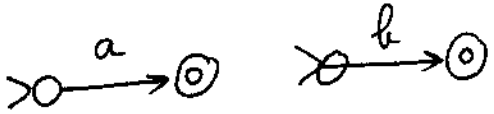
clasa l. regulate este clasa minimă de limbaje care conține $\emptyset$ și mulțimile $\{a\}, \forall a \in \Sigma$, închisă în rap. cu operațiile de reuniune, concatenare, Kleene Star.

Evident $\emptyset$, $\{a\}$ sunt acceptate AFN.

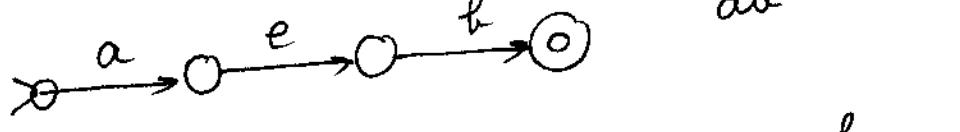


Din Th. de închidere, mulțimea limbajelor acceptate de A.F. este închisă în rap. reuniunea, concatenarea, Kleene star. $\Rightarrow$ orice limbaj regulat acceptat de A.F.

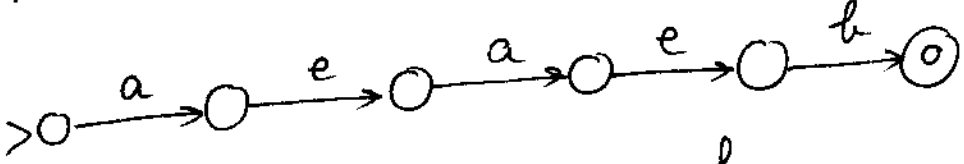
ex: $(ab \cup aab)^*$

1) 

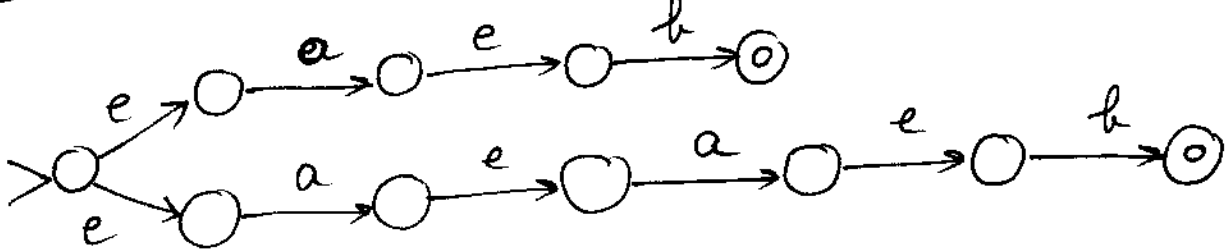
ab ; aab

2) 

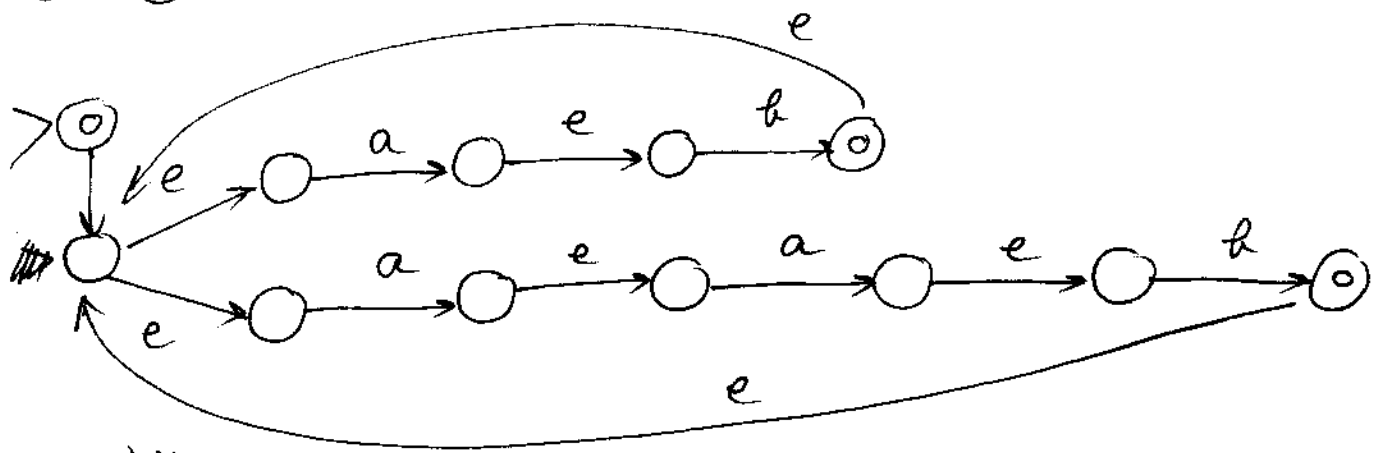
ab



aab

3) 

ab ∪ aab

4) 

$(ab \cup aab)^*$

$\Leftarrow$

Fie $M = (K, \Sigma, \delta, s, F)$

Trebuie să arăt că $\exists$ un limbaj R regulat a.î. $L(M) = R$.

Idee

Reprezint $L(M)$ ca reuniunea unui nr. finit de limbaje simple.

Fie $K = \{q_1, .., q_n\}$

$s = q_1$

Pt $i, j = 1, .., n$ și $k = 1, .., n+1$, $R(i,j,k)$ este mulțimea tuturor șirurilor din Σ^* care duc M din q_i în q_j fără să treacă prin nicio stare intermediară mai mare sau egală cu k (i, j pot fi $> k$).

$$R(i,j,k) = \{x \in \Sigma^* \mid (q_i, x) \overset{x}{\underset{M}{\vdash}} (q_j, e) \text{ și dc.}$$

$$(q_i, x) \overset{*}{\underset{M}{\vdash}} (q_e, y),\ y \in \Sigma^*,$$

atunci $l < k$ sau $y = e$ și $l = j$

sau $y = x$ și $l = i$

Pt $K = n+1$

$$R(i,j,n+1) = \{ x \in \bar{\Sigma}^* \mid (q_i, x) \overset{*}{\underset{M}{\vdash}} (q_j, e) \}$$

$$\Rightarrow L(M) = \cup \{ R(1,j,n+1) \mid q_j \in F \}$$

Important → fiecare $R(i,j,K)$ este regulat. și deci și $L(M)$ (def. prin $\cup$).

Dem. prin inducție după K

Pt $K=1$.

$$R(i,j,1) = \begin{cases} \{ \tau \in \bar{\Sigma},\ \delta(q_i,\tau) = q_j \}, & i \neq j \\ \{ e \} \cup \{ \tau \in \bar{\Sigma} \mid \delta(q_i,\tau) = q_j \}, & i = j \end{cases}$$

Fiec. astfel de mulțime este finită $\Rightarrow$ regulata.

Pt $K = 1, \dots, n$, având în vedere că mulțimile $R(i,j,K)$ au fost def. fiecare $R(i,j,K+1)$ poate fi def. în termenii celor def. anterior:

$R(i,j,k+1) = R(i,j,k) \cup R(i,k,k)R(k,k,k)^*R(k,j,k)$

Pt a trece din q_i în q_j fără a trece prin stări intermediare $> k$, m trebuie fie:

1° să treacă din $q_i \to q_j$ fără a trece prin st. intermed $> k-1$ sau

2° să treacă din(a) $q_i \to q_k$,

(t) $q_k \to q_k$ repetat

(c) $q_k \to q_j$

fără a trece prin stări intermed. $> k-1$.

Fiec $R(i,j,k)$ este regulat $\Rightarrow$ $R(i,j,k+1)$ regulat.

Demonstrații ale regularității limbajelor

ex: $\Sigma = \{0, \dots, 9\}$

$L \subseteq \Sigma^*$ mulțimea reprezentărilor întregilor pozitivi divizibili cu 2 sau 3.

$0, 3, 6, 12 \in L$

$1, 03, 00 \notin L$

? L regulat.

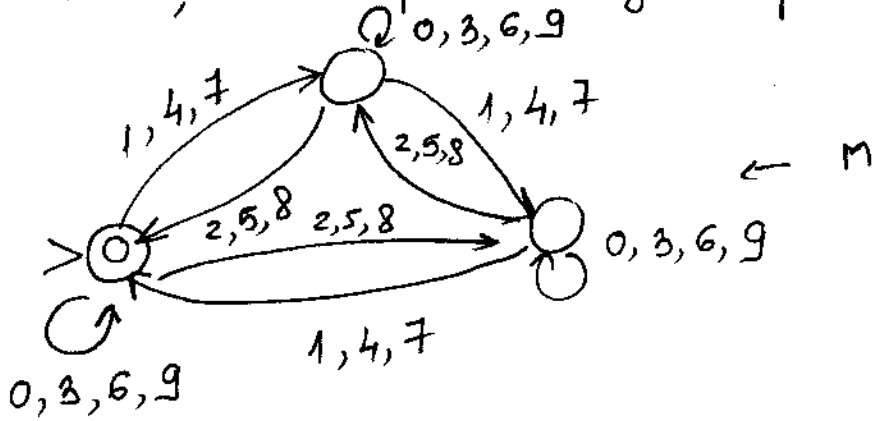
$L_1 \rightarrow$ mulțimea întregilor pozitivi

$L_1 = 0 \cup \{1, \dots, 9\}\Sigma^* \Rightarrow$ regulat fiind descris printr-o E.R.

$L_2 \rightarrow$ mulțimea repez. întregilor pozitivi divizibili cu 2.

$L_2 = L_1 \cap \Sigma^* \{0, 2, 4, 6, 8\}$

L_3 - multimea repres. în baza lor pozitivi divizibili cu 3.



$L_3 = L_1 \cap L(m)$

$L = L_2 \cup L_3$

ex:

$\Sigma = \{a, b\}$

$L \subseteq \Sigma^*$, $L = \{w \in \Sigma^* \mid |w|$ impară, w conține un nr par de 'a'$\}$.

? L regulat

$L_1 \rightarrow$ mulțimea șirurilor de lg. impară $\bar{z}(\bar{z}\bar{z})^*$

$L_2 \rightarrow$ mulțimea șirurilor cu un nr. par de 'a' : $b^*(ab^*ab^*)^*$

$L = L_1 \cap L_2$

Proprietăți ale limbajelor regulate

1° Pe măsură ce un șir parcurs stg → dr, cantitatea de memo. necesară pt a determina la sf. dc șirul este în limbaj → finită, fixată apriori, depend. de limbaj și nu de șirul particular.

$\exists a^n b^n \mid n \geq 0 \forall \Rightarrow \nexists$ A.F.

2° L. regulate cu nr. infinit de șiruri au submulțimi infinite cu o anumită structură repetitivă care se dat. în * din E.R., respectiv ciclu din diagrama de stări a A.F.

$\exists a^p \mid p$ nr prim $\forall \Rightarrow$ nu este regulat

Teorema (Lema de pompare)

Fie L un limbaj regulat infinit. Atunci există ~~șirurile x, y, z~~ ~~$y \neq e$~~, $n \geq 1$ a.î. $\forall w \in L$, $|w| \geq n$ poate fi scris $w = xyz$ a.î. $y \neq e$, $|xy| < n$ și $xy^i z \in L$, $\forall i \geq 0$.

Dem:

L regulat $\Rightarrow \exists$ AFD M care-l acceptă, $L = L(M)$

Pp. M are n stări, și fie w, $|w| \geq n$ ($L \rightarrow$ infinit, $\exists w$, $|w| \geq n$)

Fie $l = |w|$

$w = \sigma_1 \dots \sigma_l$, $\sigma_i \in \Sigma$, $l \geq n$

Fie opțiile lui M pt w:

$$(q_0, \sigma_1 \dots \sigma_l) \vdash_M (q_1, \sigma_2 \dots \sigma_l) \vdash_M \dots \vdash_M (q_{l-1}, \sigma_l) \vdash_M (q_l, e)$$

$q_0 \rightarrow$ st. inițială

$q_l \rightarrow$ st. finală

$l \geq n$
M are n stări $\xrightarrow{\text{principiul cutiei}}$ $\exists\, i,j$ a.î. $0 \leq i < j \leq l$, $q_i = q_j$

Șirul $\tau_i \dots \tau_j$ duce M din q_i înapoi în q_i $\Rightarrow$ nu este vid pt. că $i+1 \leq j$.

Dar atunci acest șir poate fi scos din w sau orice nr. de repetări ale șirului pot fi introd. în w după al j-lea simbol. $\Rightarrow$ M acceptă șirul.

M acceptă $\underbrace{\tau_1 \dots \tau_i}_{x} \underbrace{(\tau_{i+1} \dots \tau_j)^p}_{y} \underbrace{\tau_{j+1} \dots \tau_e}_{z}$, $p \geq 0$.

$$(q_0, xy^p z) \overset{x}{\underset{M}{\vdash}} (q_i, y^p z) \overset{*}{\underset{M}{\vdash}} (q_i, y^{p-1} z) \overset{*}{\underset{M}{\vdash}} \cdots \overset{*}{\underset{M}{\vdash}} (q_i, z) \overset{z}{\underset{M}{\vdash}} (q_e, e)$$

De notat $|xy| \leq n$

Ex:

$L = \{ a^n f^n \mid n \geq 0 \}$ nu este regulat

pp. $L \rightarrow$ regulat, infinit $\xrightarrow{L.P.}$

$w = a^n f^n \in L$

$w = xyz$, $|xy| \leq n$, $y \neq e$

$y = a^i$, $i > 0$

$xz = a^{n-i} f^n \notin L$

$\Rightarrow$ contradictie

Ex:

$L = \{ a^p \mid p \text{ prim} \}$

pp. $L \rightarrow$ regulat, infinit $\xrightarrow{L.P.}$

$x = a^q$, $y = a^r$, ~~$z = a^s$~~, $q, s \geq 0$, $r > 0$

$a^{q+i \cdot r+s} \in L \Rightarrow q + ir + s \Rightarrow$ nr prim $\forall i \geq 0 \Rightarrow$ imposibil

Exemplu de limbaj care respectă L.P. => nu este regulat.

$L_{balanțe} = \{a^n b^n \mid n \geq 0\}$.

$L_1 = \underbrace{ac^+ac^+ \dots ac^+bc^+bc^+ \dots bc^+}_{nr_a = nr_b}$

$len(L.P.) = 3$ => orice subșir de 3 caract → include c^+

=> dc elimin c (cf. L.P.) => 2 simb a alăturate
b alăturate
→ $\notin L_1$.

$L_{balanțe} = \bigcup_{K=0}^{\infty} (a)^K (b)^K$

$L_1 = \bigcup_{K=0}^{\infty} (ac^+)^K (bc^+)^K$

? L_2 => regulat $L_1 \cup L_2$ s. satisf. L.P. dar să nu fie regulat

↓

$0^n 1^n \cup \{0,1\}^* \rightarrow \{0,1\}^*$

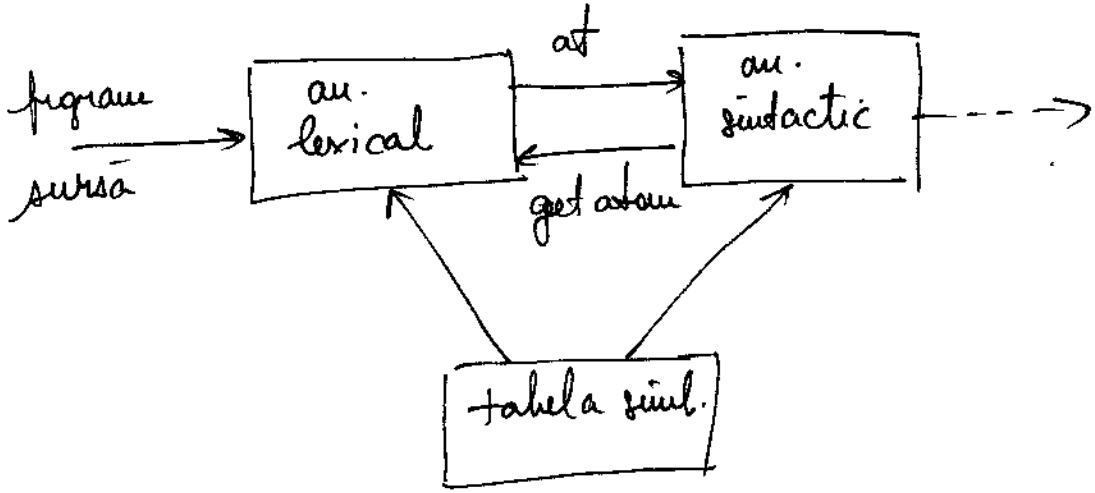
$L_2 \rightarrow$ disjunct L_1

$L_2 = (a^* b^* c)^* (a \cup b)^2 (a \cup b \cup c)^*$. $\rightarrow$ regulat

pt că $L_1, L_2 \rightarrow$ disjuncte $\Rightarrow$ $L_1 = \underbrace{L \cup L_2 - L_2}_{\text{prop incidere}}$.

Analiza lexicală

An. lexicală → prima etapă a proc. de compilare.



At. lexicali → cuv. cheie
operatori
identificatori
constante
siruri de literali
simboluri de punctuație

v = m * 2;

< id, ↑T.S. pt v >

< op- atr >

< id, ↑TS · pt m >

< op- mult >

< nnn, val. 2 >

< ; >

Definiția regulate

Σ → alfabet simboluri de față

$d_1 \to r_1$

$d_2 \to r_2$

- - - - -

$d_n \to r_n$

d_i → nume distinct

r_i → expresie regulată peste $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$.

ex:
identificatori în Pascal → șir de litere și cifre care începe cu o literă

litera → A|B|C...|Z|a|b|...|z
cifra → 0|1|...|9
id → litera (litera|cifra)*

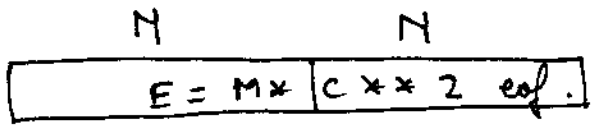
ex:
const. numerică fără semn în Pascal: 5350, 39.34, 6.33E4

cifra → 0|1|...|9
cifre → cifra cifra*
p-fract → .cifre | e
exponent → (E(+|-|e) cifre) | e
numar → cifre p-fract exponent

Convenții de notație

1° opt. postfix + →
2° $r? \equiv r | e$

Input buffer



N = 1024 sau 4096.

inițial → ei 2 ptr indică în capătul lexemei
Deplasez dr fwd-ptr => potrivire => p. fwd-ptr pe 1 nul caracter
dim dr. lexemei orte și, adec beg-ptr în p. fwd-ptr.

=> buff circular

Recunoașterea atomilor lexicali

instr → if expr then instr
| if expr then instr else instr
| e

expr → termen oprel termen
| termen

termen → id | nmm

if → if

then → then

else → else

oprel → < | <= | = | <> | > | >=

id → litera (litera | cifra) *

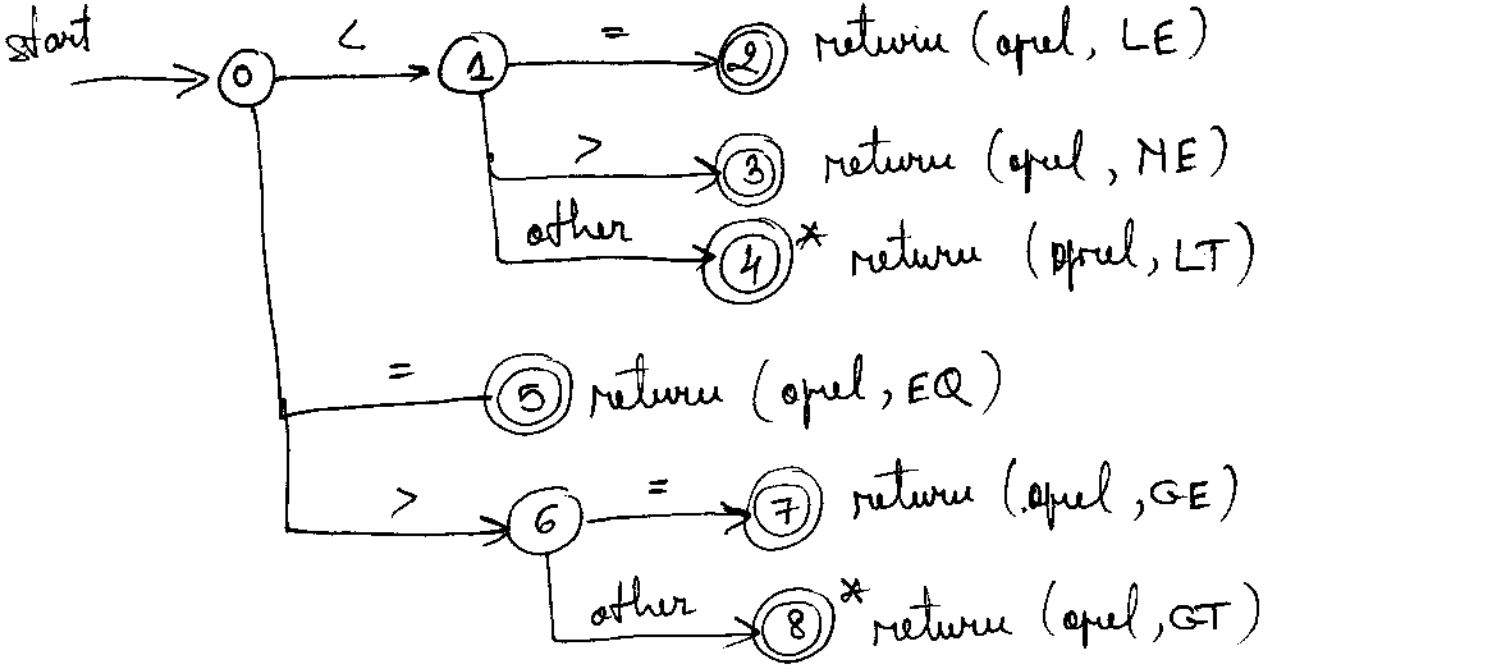
num → cifra$^+$ (. cifra$^+$)? (E (+|-)? cifra$^+$)?

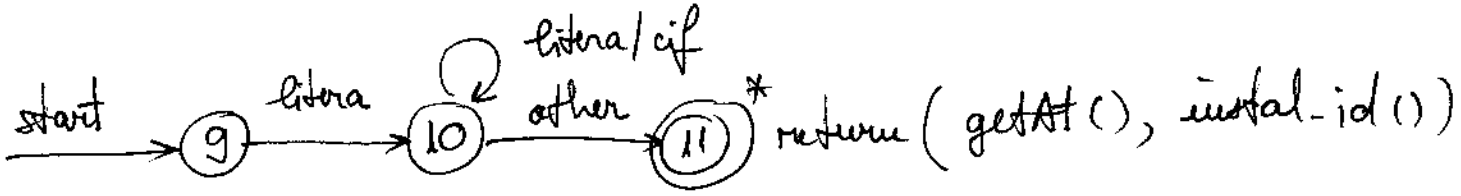
delim → blanks | tab | newline

ws → delim$^+$

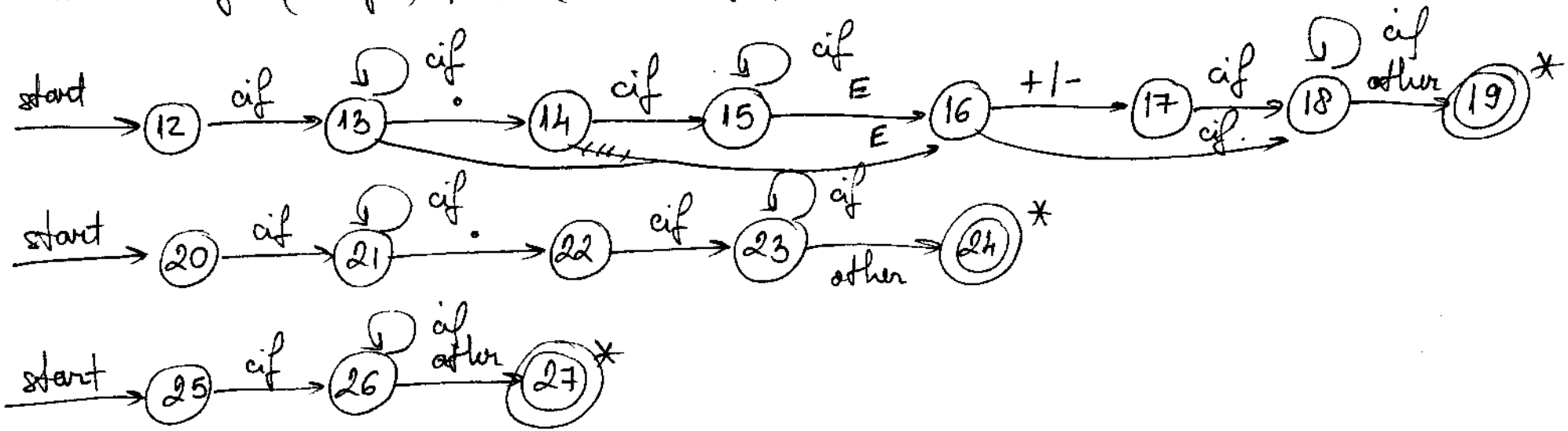
Expresii regulate	At. lexical	Atribut
ws	—	—
if	if	—
then	then	—
else	else	—
id	ID	↑ T.S.
num	NUM	↑ T.S.
<	oprel	LT
<=	oprel	LE

Diagrama de tranziție

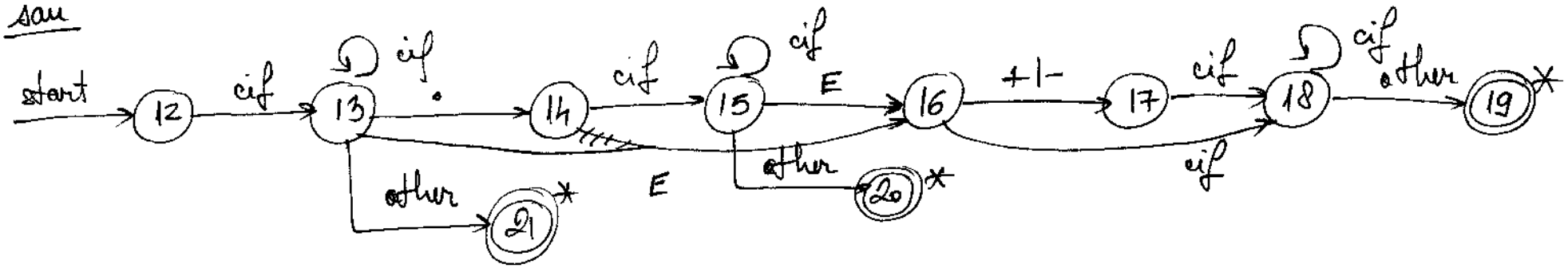




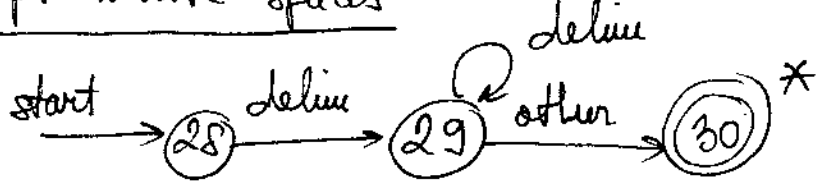
num → dif+ (. dif+)? (E(+|-)? dif+)?



sau



D.T. pt white-spaces



Implementarea unei D.T. (diagr. tranziții)

Fiec. stare → asoc. un segment de cod.

Dc ∃ arce care pleacă din starea resp. → citit urm. caracter și selectată calea.

nextchar() → 1) citește un caracter din buff.
2) avansează fwd. pointerul
3) întoarce caract. citit

Pt. a întoarce un at. lexical → var. globală → lex_value

Clasa at. lexical este întoarsă de fișia principală a an. lexical → urm Atom()

```
int state =0, start =0;
int lex_val;

int fail() {
    fwd_ptr = beg_ptr;
    switch (start) {
     case 0: start = 9; break;
     case 9: start =12; break;
     case 12: start =20; break;
     case 20: start =25; break;
     case 25: recover(); break;
     default: /* err. comp. */
    }
    return start;
}
```

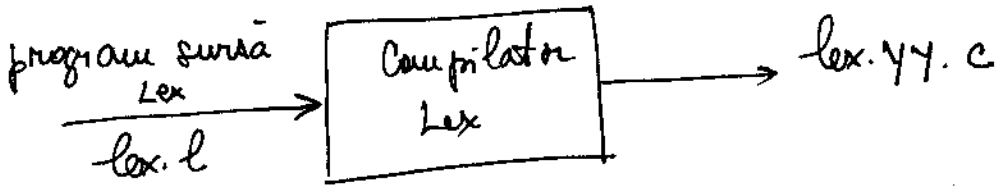
```
Atom wwwAtom() {
    while(1) {
        switch (state) {
            case 0: c = nextchar();
                    if ( c == blank || c == tab || c == newline ) {
                        state = 0;
                        lex_ptr ++;
                    }
                    else if (c == '<') state = 1;
                        else if ( c == '=') state = 5;
                            else if ( c == '>') state = 6;
                                else state = fail();
                    break;
```

```
/* cazwile 1-8 */
case 9 : c = nextchar();
         if (isletter(c)) stare = 10;
         else stare = fail();
         break;
case 10: c = nextchar();
         if (isletter(c) || isdigit(c)) stare = 10;
         else stare = 11;
         break;
case 11: retract(1); instal_id();
         return getAt();
 /* cazwile 12-24 */
case 25 : c = nextchar();
          if (isdigit(c)) stare = 26;
          else stare = fail();
          break;
```

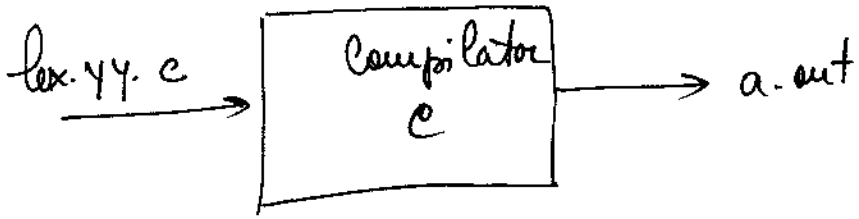
```
case 26: c = nextchar();
         if (isdigit(c)) state = 26;
         else state = 27;
         break;
case 27: retract(1); install_num();
         return NUM;

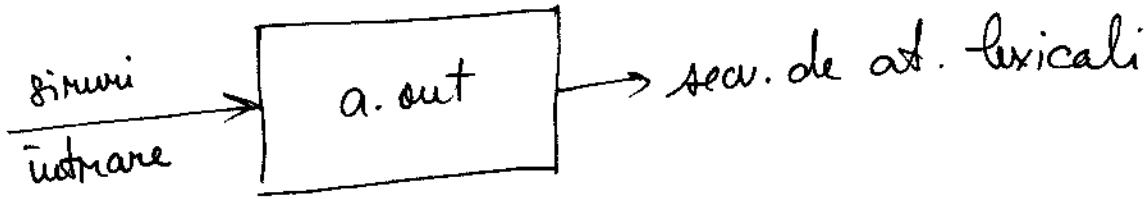
    }
  }
}
```

Limbaj pentru specificarea an. lexicale LEX



o reprezentare tabelară a diagr. de tranziții construite din expresiile reg. din lex.l + fișier standard





Specificare lex.

Un program lex → 3 părți:

declarații

%%

reguli de translatare

%%

funcții auxiliare

Declarații → def. variabile, constante, def. regulate

Reguli de translatare:

p1 { actiune 1 }

p2 { actiune 2 }

⋮

pn { actiune n }

unde p_i → exp. regulată

acțiune$_i$ → fragment de cod care descrie ce acțiune trebuie să execute an. lexical când este găsită o potrivire cu p_i

Fcții auxiliare → fcții apelate în acțiuni

Obs:
An. lexical întoarce doar at. lexical. Pt a întoarce o valoare atribut împreună cu info despre lexemă => var. globală yylval.

Spec lex.

```
%{ /* def. constante
      LT, LE, EQ, NE, GT, GE, IF, THEN, ELSE, ID, NUM, OPREL */
%}
/* def. regulate */
```

```
delim   [ \t\n]
ws      {delim}+
lit     [A-Za-z]
dig     [0-9]
id      {lit} ({lit} | {dig})*
num     {dig}+ (\.{dig}+)? (E [+\-]? {dig}+)?

%%
{ws}    { /* nicio actiune */ }
if      { return IF; }
then    { return THEN; }
else    { return ELSE; }
{id}    { yylval = instal_id(); return ID; }
{num}   { yylval = instal_num(); return NUM; }
"<"     { yylval = LT; return OPREL; }
```

"<=" { yylval = LE ; return OPREL; }

;

%%

instal_id() { /* fctie care pune lexema (sir de caract.) al carei prim caracter este indicat de yytext si a carei lungime este yyleng, in tab. de simboluri si intoarce intr. din tabela */ }

instal_num() { /* similar pt. a adauga o const. numerica in T.S. */ }.

Obs:

→ Tot ceea ce este intre %{ si %} este copiat direct in lex.yy.c si nu este tratat ca parte a def. regulate din regulile de translat.

→ ac. lucru ⇒ fctii / proc. def. in a 3a sectiune

→ \. ⇒ pct, altfel reprez. clasa tuturor caracterelor except newline

→ \- => -, altfel simbol de interval [A-Z]

yylval → def. în lex.yy.c
yytext → pointer către primul caracter din lexemă (leg-ptr)
yyleng → lungimea lexemei (am folosit fwd-ptr).

Limbaje Independente de Context

Gramatici Indep. de Context

L. regulate < acceptori → A.F.
generatori → E.R.

$a(a^* \cup b^*)b$

Ex. S → non simbol interpretat ca șir în limbaj
M → simbol corresp. $(a^* \cup b^*)$

$S \to aMb \overset{!}{\Rightarrow}$ regulă

$M \to A$
$M \to B$
$A \to aA \mid e$
$B \to bB \mid e$

Pornesc cu S, găsesc un simbol ~~care~~ în șirul generat de S, care apare la stg. într-una dintre reguli. Înloc simb cu șirul care apare în dr. regulii, etc

$S \to aAb \to aAb \to aaAb \to aaaAb \to aaab$

Def. autoioară → gramatică (G)

? G. indep. de context

(aa) Ab)
✓ contextul lui A

$A \to aA \Rightarrow A$ se poate înlocui prin șirul aA indiferent de contextul lui A.

Def:

O gramatică indep. de context, G, este un tuplu (V, Σ, R, S), unde:

$V \to$ alfabet

$\Sigma \subseteq V \to$ mulțimea simb. terminale

$R \subseteq (V-\Sigma) \times V^* \to$ mulțimea regulilor

↑ submulțime finită

$S \in V-\Sigma \to$ simbolul de start

Obs:

$V-\Sigma \to$ mult. simb. neterminale (neterminali)

Gramatici Independente de Context

Def.

$G = (V, \Sigma, R, S)$

$A \in V - \Sigma$, $u \in V^*$, $A \underset{G}{\rightarrow} u$, $(A, u) \in R$

$\forall u, v \in V^*$, $u \underset{G}{\Rightarrow} v \iff \exists x, y, v' \in V^*$, $A \in V - \Sigma$ a.î.

$$u = xAy$$
$$v = xv'y$$
$$A \underset{G}{\rightarrow} v'$$

Def.

Relația $\overset{*}{\underset{G}{\Rightarrow}}$ este închiderea reflexivă și tranzitivă $\underset{G}{\Rightarrow}$

Def.

$L(G) = \{ w \in \Sigma^* \mid S \overset{*}{\underset{G}{\Rightarrow}} w \}$ limbajul generat de gram. G.

Obs:

$w_1 \underset{G}{\Rightarrow} w_2 \underset{G}{\Rightarrow} \dots \underset{G}{\Rightarrow} w_n \implies$ o derivare în G a lui w_n din w_1

ex: E

$G=(V, \Sigma, R, S)$

$V = \{x, 1, 2, +, *, (,), T, F, E\}$

$\Sigma = \{x, 1, 2, +, *, (,)\}$

$R = \{ E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid x1 \mid x2 \}$

? $(x_1 * x_2 + x1) * (x_1 + x_2)$

$E \Rightarrow T$

$\Rightarrow T*F$

$\Rightarrow T*(E)$

$\Rightarrow T*(E+T)$

$\Rightarrow T*(T+T)$

$\Rightarrow T*(F+T)$

$\Rightarrow T*(x_1+T)$

.... $\Rightarrow T*(x_1+x_2)$

$\Rightarrow (E)*(x_1+x_2)$

$\Rightarrow (E+T)*(x_1+x_2)$

$\Rightarrow (E+F)*(x_1+x_2)$

$\Rightarrow (E+x_1)*(x_1+x_2)$

$\Rightarrow (T+x_1)*(x_1+x_2)$

- - — -

$\Rightarrow (x_1 * x_2 + x_1) * (x_1 + x_2)$?

Ex:

$G = (V, \Sigma, R, S)$

$V = \{S, (,)\}$

$\Sigma = \{(,)\}$

$R = \{ S \to \varepsilon$

$S \to SS$

$S \to (S)$

$\}$

$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S((S)) \Rightarrow S(()) \Rightarrow (S)(()) \Rightarrow ()(())$

? $L(G)$ regulat

pp $L(G)$ regulat

$L(G) \cap (^*)^*$ $\overset{\text{Th. inchidere}}{\Rightarrow}$ regulat

$L(G) \cap (^*)^* = \{ (^n)^n \mid n \geq 0 \} \to$ regulat

Limbaje regulate și Limbaje Indep. de Context

Def.
O GiC., $G = (V, \bar{Z}, R, S)$ este o gram. regulată $\Leftrightarrow R \subseteq (V - \bar{Z}) * \bar{Z}^* ((V - \bar{Z}) \cup \{\varepsilon\})$.

ex:
$G = (V, \bar{Z}, R, S)$

$V = \{S, A, B, a, b\}$

$\bar{Z} = \{a, b\}$

$R = \{ S \to bA \mid aB \mid \varepsilon$

$A \to abS$

$B \to baS$

$\}$

$S \Rightarrow bA \Rightarrow baS \Rightarrow baaB \Rightarrow baabaS \Rightarrow baabab$

$L(G) = (abab \cup baba)^*$

? gram are reguli care în p. dr. conțin (eventual) 1 singur neterminal (primul sau ultimul) din șir => NU este garantat a fi regulată.

ex: $S \to A$

$A \to 01 \mid 0B$ $\quad L = \{0^n 1^n \mid n \geq 0\}$

$B \to A1$

Teorema

Un limbaj este regulat <=> generat de o gram. regulată.

dem.

=>

pp. L este limbaj regulat => $\exists M$, AFD, a.î. $L = L(M)$

$M = (K, \Sigma, \delta, s, F)$ și construiesc o GIC, $G = (V, \Sigma, R, S)$

$V = \Sigma \cup K$

$S = s$

$R = \{q \to ap \mid \delta(q, a) = p\} \cup \{q \to e \mid q \in F\}$

pp. $\Sigma \cap K = \emptyset$

Idee: regulile lui G sunt def. ca deriv. din G să simuleze tranzițiile lui M

$\forall \tau_1, \dots, \tau_m \in \Sigma, p_0, \dots, p_m \in K$

$(p_0, \tau_1 \dots \tau_m) \vdash_M (p_1, \tau_2 \dots \tau_m) \vdash_M \dots \vdash_M (p_m, e)$

$\Leftrightarrow$

$p_0 \Rightarrow_G \tau_1 p_1 \Rightarrow_G \tau_1 \tau_2 p_2 \Rightarrow_G \dots \Rightarrow_G \tau_1 \dots \tau_m p_m$

pt că $\delta(q, a) = p \Leftrightarrow q \to ap$

Lem. $L(M) = L(G)$

pp. $w \in L(M) \Rightarrow (A, w) \vdash_M^* (p, e)$, $p \in F$

$\Rightarrow A \Rightarrow^* wp \vdash A \Rightarrow^* w$, $w \in L(G)$

$p \to e \in R$

$L(G) \subseteq L(M)$

pp $w \in L(G)$

$S \Rightarrow_G^* w \Rightarrow A \Rightarrow_G^* w$

Regula utilizată în ultimul pas al deriv. $p \to e$, $p \in F$ $\vdash A \Rightarrow^* wp \Rightarrow w$

Fie $G=(V, \Sigma, R, S)$ gram. regulată

? M a.î. $L(M)=L(G)$

Idea → derivările din G mimate de pașii A.F.

Fie $M=(K, \Sigma, \Delta, s, F)$

$K=(V-\Sigma) \cup \{f\}$

$s=S$

$F=\{f\}$

$\Delta=\{(A, w, B) \mid A \to wB \in R, A, B \in V-\Sigma, w \in \Sigma^*\}$

$\cup \{(A, w, f) \mid A \to w \in R, A \in V-\Sigma, w \in \Sigma^*\}$

$\forall A_1, .., A_n \in V-\Sigma, w_1, .., w_n \in \Sigma^*$

$$A_1 \underset{G}{\Rightarrow} w_1 A_2 \underset{G}{\Rightarrow} w_1 w_2 A_3 \underset{G}{\Rightarrow} \cdots \underset{G}{\Rightarrow} w_1 \cdots w_{n-1} A_n \underset{G}{\Rightarrow} w_1 \cdots w_n$$

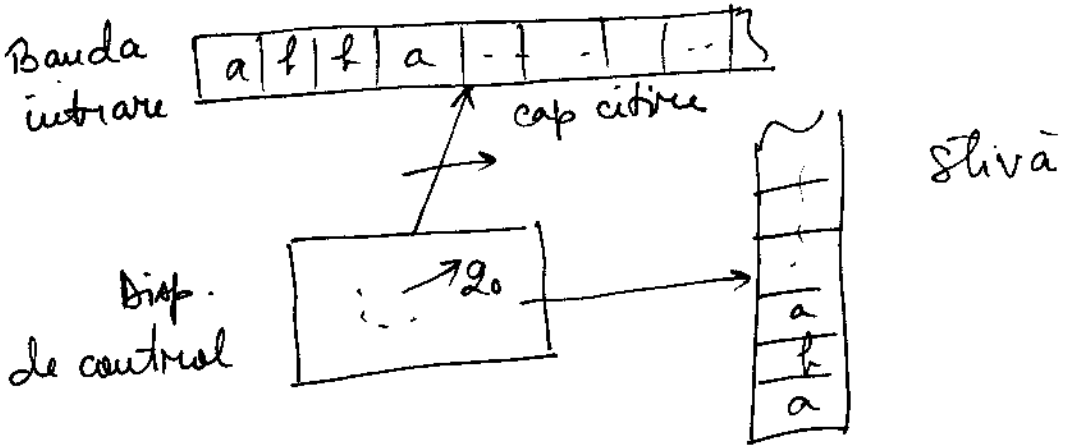
$$\Leftrightarrow (A_1, w_1 \cdots w_n) \vdash_M (A_2, w_2 \cdots w_n) \vdash_M \cdots \vdash_M (A_n, w_n) \vdash_M (f, e)$$

$w \in L(G), w \in \Sigma^*, S \overset{*}{\Rightarrow} w \Rightarrow (S, w) \vdash_M^* (f, e)$ a.î. $w \in L(M)$

$w \in L(M)$ a.î. $(S, w) \vdash_M^* (f, e) \Rightarrow S \overset{*}{\underset{G}{\Rightarrow}} w$ adică $w \in L(G)$.

Automate cu stivă (pushdown)

Fie limbajul $\{ww^R \mid w \in \Sigma^*\}$.



Def.
Un automat pushdown este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$

$K \rightarrow$ mulțimea finită a stărilor
$\Sigma \rightarrow$ alfabet de intrare
$\Gamma \rightarrow$ alfabetul stivei
$s \in K \rightarrow$ st. inițială
$F \subseteq K \rightarrow$ mulțimea st. finale

$\Delta \rightarrow$ relația de tranziție

$\Delta \subseteq (K \times \bar{\Sigma}^* \times \Gamma^*) \times (K \times \Gamma^*)$

$((p,u,\beta),(q,\gamma)) \in \Delta$, M în st. p cu β în vf. stivei poate citi u de pe banda de intrare, în locul β cu γ pe stivă și intră în st. q.

Caz. particulare:

$((p,u,e),(q,a))$ adg. 'a' pe stivă

$((p,u,a),(q,e))$ scoate 'a' de pe stivă

Def.
O configurație este un element din $K \times \bar{\Sigma}^* \times \Gamma^*$

Def.
Un A.P.D. M acceptă $w \in \bar{\Sigma}^* \iff (s,w,e) \overset{*}{\underset{M}{\vdash}} (p,e,e)$, $p \in F$.

Def.
Limbajul acceptat de A.P.D. M este mulțimea șirurilor acceptate de M.

ex:
A.F. poate fi considerat un A.P.D. care nu lucrează cu stiva.

$M = (K, \Sigma, \Delta, \Gamma, \Delta, F)$ A.P.D. , $\Gamma = \emptyset$

$M' = (K, \Sigma, \Delta', \Delta, F)$ A.F.

$\Delta' = \{((p, u, e), (q, e)) \mid (p, u, q) \in \Delta\}$

ex: ? M care acceptă $\{wcw^R \mid w \in \{a, b\}^*\}$.

$M = (K, \Sigma, \Gamma, \Delta, \Delta, F)$

$K = \{\Delta, f\}$,

$\Sigma = \{a, b, c\}$

$\Gamma = \{a, b\}$

$F = \{f\}$

Δ:

1. $((s, a, e), (s, a))$
2. $((s, b, e), (s, b))$
3. $((s, e, e), (f, e))$
4. $((f, a, a), (f, e))$
5. $((f, b, b), (f, e))$

Stare	Intrare	Stiva	Trans. utilizată
s	abbcbba	e	—
s	bbcbba	a	1
s	bcbba	ba	2
s	cbba	bba	2
s	bba	bba	3
f	ba	ba	5
f	a	a	5
f	e	e	4
f			

? M a.î. $L(M) = \{ww^R \mid w \in \{a, b\}^*\}$

L(A.P.D.) și L(Gi.C)

Fie $G = (V, \Sigma, R, S)$

$w \in L(G) \iff w \in \Sigma^*$ și există o derivare

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_{n-1} \Rightarrow w$$

$w_1, \dots, w_{n-1} \in V^* \ (n>0)$

Derivare stg $\rightarrow$ la fiec. pas în locuința cel mai din stg. neterm. din șir

ex: $V = \{S, (,)\}$

$\Sigma = \{(,)\}$

$R = \{S \rightarrow e \mid SS \mid (S)\}$

$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()()$

Formal: $\alpha \underset{G}{\overset{L}{\Rightarrow}} \beta$, $\alpha, \beta \in V^* \iff \alpha = \alpha_1 A \alpha_2$, $\beta = \alpha_1 \gamma \alpha_2$,

$A \underset{G}{\rightarrow} \gamma$, $\alpha_1 \in \Sigma^*$

$\overset{*L}{\underset{G}{\Rightarrow}}$ închiderea reflexivă și tranzitivă a $\overset{L}{\underset{G}{\Rightarrow}}$

Lema

Pt orice GIC $G=(V, \Sigma, R, S)$ și orice șir $w \in \Sigma^*$, $S \overset{*}{\underset{G}{\Rightarrow}} w \Leftrightarrow$

$S \overset{*L}{\underset{G}{\Rightarrow}} w$

Lema

Pt orice gramatică indep. de context $G = (V, \Sigma, R, S)$ și orice șir $w \in \Sigma^*$,

$$S \overset{*}{\underset{G}{\Rightarrow}} w \iff S \overset{st}{\underset{G}{\Rightarrow}} w$$

Dem.

$\Rightarrow$ $\forall$ deriv. stg. este o derivare $\Rightarrow$ evident

$\Leftarrow$

Fie $w_0 \underset{G}{\Rightarrow} w_1 \underset{G}{\Rightarrow} \dots \underset{G}{\Rightarrow} w_n$, o derivare, nu neapărat stg.

$w_0 = S$

$w_n = w \in \Sigma^*$

Idee: vom arăta cum se poate transforma într-o deriv. stg.

Fie k nr. minim de pași ai $w_k \Rightarrow w_{k+1}$ nu este o deriv. stg.

$\Rightarrow$ de $\nexists k \Rightarrow$ g.e.d.

Def. o altă deriv. în n pași

$$w_0' \underset{G}{\Rightarrow} w_1' \underset{G}{\Rightarrow} \dots \Rightarrow w_n'$$

ai $w_0' = w_0$ și $w_n' = w_n$

Ac. nouă deriv. este fie stg. sau are proprietatea că dc. $w'_{k'} \underset{G}{\Rightarrow} w'_{k'+1}$, este cel mai recent pas al derivării care nu este stg, atunci $k' > k$

Cum ambele deriv. au ac lg $\Rightarrow$ dem. prin inducție

$w_k \underset{G}{\Rightarrow} w_{k+1}$ nu este stg. $\Rightarrow$

$w_k = \alpha A \beta B \gamma$ ($\alpha \in \bar{Z}^*$, $\beta, \gamma \in V^*$, $A, B \in V - \bar{Z}$)

$w_{k+1} = \alpha A \beta \delta \gamma$, $B \underset{G}{\rightarrow} \delta$

Cum $w_n \in \bar{Z}^*$, $k+1 < n \Rightarrow$ există un pas ulterior la care A este înlocuit

Fie $l > k$ cel m. mic nr. aș

$w_l = \alpha A \varepsilon$, $\varepsilon \in V^*$

$w_{l+1} = \alpha \xi \varepsilon$, $A \underset{G}{\rightarrow} \xi$

$\beta B \gamma \overset{*}{\underset{G}{\Rightarrow}} \varepsilon$ în $l-k$ pași

Ad putem interschimba utiliz. regulii $B \underset{G}{\rightarrow} \delta$ cu $A \underset{G}{\rightarrow} \xi$

$w_0 \overset{*}{\underset{G}{\Rightarrow}}{}^{L} w_k = \alpha A \beta B \gamma$ — k pași

$\overset{L}{\Rightarrow} \alpha \xi \beta B \gamma$ — 1 pas

$\overset{L}{\Rightarrow}{}^{*} \alpha \xi \varepsilon$ — l - k pași

$\overset{*}{\underset{G}{\Rightarrow}} w_n$

Ac. nouă deriv. are lung n și cel puțin k+1 deriv. stg. la început.

Teorema

Clasa limbajelor acceptate de A.P.D. este clasa limbajelor generate de GIC (LIC)

Dem:

Lema 1

Orice LIC este acceptat de un A.P.D.

Dem:

Fie $G = (V, \Sigma, R, S) \rightarrow$ GIC

Trebuie să construim un APD M a.î. $L(M) = L(G)$.

$M = (\{p, q\}, \Sigma, V, \Delta, p, \{q\})$

Δ:

1. $((p, e, e), (q, S))$
2. $((q, e, A), (q, x))$, $A \to x \in R$
3. $((q, a, a), (q, e))$, $a \in \Sigma$.

Ex:

$G = (V, \Sigma, R, S)$

$V = \{S, a, b, c\}$

$\Sigma = \{a, b, c\}$

$R = \{ S \to aSa \mid bSb \mid c \}$

$L(G) = \{ wcw^R \mid w \in \{a, b\}^* \}$

$M = (\{p, q\}, \Sigma, V, \Delta, p, \{q\})$

$\Delta = \{((p, e, e), (q, S))$ (T1)

$((q, e, S), (q, aSa))$ (T2)

$((q, e, S), (q, tSt))$ (T3)

$((q, e, S), (q, c))$ (T4)

$((q, a, a), (q, e))$ (T5)

$((q, t, t), (q, e))$ (T6)

$((q, c, c), (q, e))$ (T7)

Stare	Sir intrare	Stiva	Transz. utilizz.
p	atcta	e	
q	atcta	S	T1
q	atcta	aSa	T2
q	tcta	Sa	T5
q	tcta	$tSta$	T3
q	cta	Sta	T6
q	cta	cta	T4
q	ta	ta	T7
q	a	a	T6
q	e	e	T5

Prop. 1

Dc. $S \overset{*L}{\underset{G}{\Rightarrow}} \alpha_1\alpha_2$, $\alpha_1 \in \bar{\Sigma}^*$, $\alpha_2 \in (V-\bar{\Sigma})V^* \cup \{\varepsilon\}$ atunci $(q, \alpha_1, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \alpha_2)$.

Prop 2.

Dc. $(q, \alpha_1, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \alpha_2)$, $\alpha_1 \in \bar{\Sigma}^*$, $\alpha_2 \in V^*$ și $S \overset{*L}{\underset{G}{\Rightarrow}} \alpha_1\alpha_2$

Propp. sunt suf. pt dem. lemei pt că

$$S \overset{*L}{\underset{G}{\Rightarrow}} \alpha \ (\alpha_2 = \varepsilon) \ , \alpha \in \bar{\Sigma}^*$$

$$\Leftrightarrow \quad (q, \alpha, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \varepsilon)$$

$$\alpha \in L(G) \Leftrightarrow \alpha \in L(M)$$

Dem Prop 1

pp. $S \overset{*L}{\underset{G}{\Rightarrow}} \alpha$, $\alpha = \alpha_1\alpha_2$, $\alpha_1 \in \bar{\Sigma}^*$, $\alpha_2 \in (V-\bar{\Sigma})V^* \cup \{\varepsilon\}$

Dem. prin inducție după lg. derivării α din S.

Pas de bază

Dc. deriv. are $lg=0 \Rightarrow S=\alpha \Rightarrow \alpha_1 = e,\ \alpha_2 = S$

$$(q, \alpha_1, S) \overset{*}{\underset{m}{\vdash}} (q, e, \alpha_2)$$
$$\quad\ \| \qquad\qquad\quad \|$$
$$\quad\ e \qquad\qquad\quad S$$

Ip. inductivă

pp. $S \overset{*L}{\underset{G}{\Rightarrow}} \alpha_1\alpha_2$ în cel mult n pași $\Rightarrow (q, \alpha_1, S) \overset{*}{\underset{m}{\vdash}} (q, e, \alpha_2)$

Pas de inducție

$$S = u_0 \overset{L}{\underset{G}{\Rightarrow}} u_1 \overset{L}{\underset{G}{\Rightarrow}} u_2 \overset{L}{\underset{G}{\Rightarrow}} \dots \overset{L}{\underset{G}{\Rightarrow}} u_{n+1} = \alpha$$

și fie $\alpha = \alpha_1\alpha_2$ cf. specific.

$\Rightarrow$ u_n are cel puțin un neterminal

$u_n = \beta_1 A \beta_2$

$u_{n+1} = \beta_1 \gamma \beta_2$, $\beta_1 \in \Sigma^*$, $A \in V - \Sigma$, $A \underset{G}{\rightarrow} \gamma$

Din ip. inductivă:

$(q, \beta_1, \zeta) \overset{0}{\underset{M}{\vdash}} (q, e, A\beta_2)$.

$A \underset{G}{\to} \gamma \Rightarrow ((q, e, A), (q, \gamma)) \in \Delta$ (tranziție a lui M)

$(q, e, A\beta_2) \underset{M}{\vdash} (q, e, \gamma\beta_2)$

$\alpha = \beta_1 \gamma \beta_2$, $\beta_1 \in \bar{\Sigma}^*$

$\alpha = \alpha_1 \alpha_2$, $\alpha_1 \in \bar{\Sigma}^*$, $\alpha_2 = e$ sau începe cu un neterminal.

$|\alpha_1| \geq |\beta_1|$

$|\alpha_2| \leq |\gamma\beta_2|$, pt că $\alpha_1 \in \bar{\Sigma}^*$ și γ poate începe cu terminale.

$\Rightarrow \alpha_1 = \beta_1 \delta$, $\delta \in \bar{\Sigma}^*$

și $\delta\alpha_2 = \gamma\beta_2$

$(q, \delta, \gamma\beta_2) \overset{*}{\underset{M}{\vdash}} (q, e, \alpha_2)$ (prin tranziții de reducere).

$$(q, \alpha_1, S) \overset{}{\vdash_M} (q, p_1 \delta, S)$$
$$\vdash_M^* (q, \delta, A\beta_2)$$
$$\vdash_M (q, \delta, \gamma\beta_2)$$
$$\vdash_M^* (q, e, \alpha_2) \quad q \text{ e.d.}$$

Prop 2 (Tema)

Lema 2

Dc un limbaj este acceptat de un APD, atunci este un limbaj indep de context.

Dem.

Am. fi util să restricționăm APD.

Numim $M = (K, \Sigma, \Gamma, \Delta, s, F)$ simplu, dc:

(1) $((q, u, \beta), (p, \gamma)) \in \Delta$, $|\beta| \leq 1$

(2) $((q, u, e), (p, \gamma)) \in \Delta$, $((q, u, A), (p, \gamma A)) \in \Delta$, $A \in \Gamma$

$(pop(A), push(A))$

Fie $M = (K, \Sigma, \Gamma, \Delta, s, F)$ APD $\Rightarrow$ construim un APD simplu care acceptă $L(M)$.

Elim. tranzițiile $((q, u, \beta), (p, \delta))$, $|\beta| > 1 \Rightarrow M$ va scoate de pe stivă secvențial simb. din β.

Dc $\beta = B_1 \dots B_n$, $n > 1$, $B_1, \dots, B_n \in \Gamma$

Adg. la K stările $t_1, \dots, t_{n-1}$ și în Δ în locuim

$((q, u, \beta), (p, \delta))$ cu

$((q, e, B_1), (t_1, e))$

$((t_1, e, B_2), (t_2, e))$

- - - - - -

$((t_{n-2}, e, B_{n-1}), (t_{n-1}, e))$

$((t_{n-1}, u, B_n), (p, \delta))$

Se repetă pt toate tranzițiile care nu respectă (1)

Pt cond. (2) $\Rightarrow$ adg. $((q, u, A), (p, \delta A))$, $A \in \Gamma$

oricâte ori $((q, u, e), (p, \delta)) \in \Delta$.

Vrem să arăt că dc $M = (K, \Sigma, \Gamma, \Delta, s, F)$ este un APDȘ atunci $L(M)$ este generat de o GIC.

Idee: gramatica trebuie să genereze șiruri acceptate de M.

Fie $G = (V, \Sigma, R, S)$

V va conține pe lângă S și $\sigma \in \Sigma$, simb. $\langle q, A, p \rangle$ pt toate stările $p, q \in K$, $A \in \Gamma \cup \{\varepsilon\}$.

$A \in \Gamma$, $\langle q, A, p \rangle$ => gen. o porțiune din șirul de intrare care ar putea fi citit între momm. de timp în care M este în st. q cu A în vf. stivei și cel în care M scoate A de pe stivă și intră în st p.

$A = e$, $\langle q, e, p \rangle$

Lema 2 rezultă din prop. pt că

$$\underbrace{\langle A, e, f\rangle \overset{*}{\underset{G}{\Rightarrow}} x}_{x \in L(G)}, f \in F \iff \underbrace{(A, x, e) \overset{*}{\underset{M}{\vdash}} (f, e, e)}_{x \in L(M)}$$

Dem.

pp. $\langle q, A, p\rangle \overset{*}{\underset{G}{\Rightarrow}} x$ dem prin ind. după lg. deriv. că

$$(q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$$

Pas de bază

Deriv. 1 pas $\Rightarrow \langle q, A, p\rangle \underset{G}{\Rightarrow} x$ nu poate fi decât dipul (4)

$p = q$, $A = x = e$

$(q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$ prin reflexivitate

$\| \quad \| \quad \|$

$p \quad e \quad e$

Regulile din R:

1° $\forall f \in F$, $S \to \langle \Delta, e, f \rangle$

2° $\forall ((q, u, A), (n, B_1 \dots B_m)) \in \Delta$, $q, r \in K$, $u \in \Sigma^*$, $m > 0$.

$B_1, \dots, B_m \in \Gamma$, $A \in \Gamma \cup \{e\}$,

$\langle q, A, p \rangle \to u \langle n, B_1, q_1 \rangle \langle q_1, B_2, q_2 \rangle \dots \langle q_{m-1}, B_m, p \rangle$

$\forall p, q_1, \dots, q_{m-1} \in K$.

3° $\forall ((q, u, A), (n, e)) \in \Delta$, $q, r \in K$, $u \in \Sigma^*$, $A \in \Gamma \cup \{e\}$, $p \in K$

$\langle q, A, p \rangle \to u \langle n, e, p \rangle$

4° $\forall q \in K$, $\langle q, e, q \rangle \to e$

Prop.

$\forall q, p \in K$, $A \in \Gamma \cup \{e\}$, $x \in \Sigma^*$

$\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x \iff (q, x, A) \vdash_M^{x} (p, e, e)$

Ip. ind.

$\langle q, A, p\rangle \overset{*}{\underset{G}{\Rightarrow}} x$ $\Rightarrow$ printr-o derivare în cel mult K pași ($K>1$).

Pasul de inducție

Pp x este derivat din $\langle q, A, p\rangle$ în K+1 pași

Primul pas este de tipul (2) sau (3).

Tipul (2)

$\langle q, A, p\rangle \Rightarrow u\langle r, B_1, q_1\rangle\langle q_1, B_2, q_2\rangle \cdots \langle q_{n-1}, B_n, p\rangle \overset{x}{\Rightarrow} x$

$n\geq 1$, $B_1, \ldots B_n \in \Gamma$, $r, q_1, \ldots, q_{n-1} \in K$, $(q, u, A), (r, B_1 \ldots, B_n)) \in \Delta$.

Notez: $q_0 = r$

$q_n = p$

$\exists$ și numerele $x_1, \ldots, x_n \in V_T^*$ a.î.

$\langle q_{i-1}, B_i, q_i\rangle \overset{*}{\underset{G}{\Rightarrow}} x_i$, $i = 1, \ldots, n$ printr-o deriv. în cel mult K pași

$x = u x_1 \ldots x_n$

Din ip. ind. $(q_{i-1}, z_i, B_i) \vdash_M^+ (q_i, e, e)$, $i = 1, .., n$

combinând $(q, u, A) \vdash_M (n, e, B_1 \dots B_n)$

$\Rightarrow (q, u z_1 \dots z_n, A) \vdash_M (n, z_1 \dots z_n, B_1 \dots B_n)$

$\vdash_M^* (q_1, z_2 \dots z_n, B_2 \dots B_n)$

$\vdash_M^* (p, e, e)$

Primul pas de <u>tipul 3</u>

$\langle q, A, p \rangle \underset{G}{\Rightarrow} u \langle n, e, p \rangle \underset{G}{\overset{*}{\Rightarrow}} x$, $((q, u, A), (n, e)) \in \Delta$.

Analog se dem. $z \in \bar{z}^*$, $x = uz$, $(q, uz, A) \vdash_M (n, z, e) \vdash_M^* (p, e, e)$

Proprietăți ale LIC

Propr. de închidere

Teorema

LIC sunt închise în rap. cu operațiile de reunire, concatenare, Kleene Star.

Dem.

Fie $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$

pp. că $V_1 - \Sigma_1$ și $V_2 - \Sigma_2$ sunt disjuncte.

Reunire

Fie S un nou simbol, $G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}, S)$

Ad. $L(G_1) \cup L(G_2) = L(G)$.

Cum reg. corresp. lui S sunt $S \to S_1$ și $S \to S_2$, $S \overset{*}{\Rightarrow} w$, $w \in (\Sigma_1 \cup \Sigma_2)^*$

dc și numai dc fie $S_1 \overset{*}{\Rightarrow} w$ fie $S_2 \overset{*}{\Rightarrow} w$. Cum G_1 și G_2 au mulțimi de neterminale disjuncte, $S \overset{*}{\underset{G}{\Rightarrow}} w \iff S_1 \overset{*}{\underset{G_1}{\Rightarrow}} w$; analog pt. G_2.

Concatenare

Construcție similară; $L(G_1)L(G_2)$ este generat de

$$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S)$$

Kleene star

$L(G_1)^*$ este generat de

$$(V_1, \Sigma_1, R_1 \cup \{S_1 \to e, S_1 \to S_1 S_1\}, S_1)$$

Teorema

Intersecția unui LIC cu un LR este un LIC.

Dem.

Fie $L \to$ LIC

$R \to$ LR

$L = L(M_1)$, $M_1 = (K_1, \bar{Z}_1, \Gamma_1, \Delta_1, \Delta_1, F_1)$

$R = L(M_2)$, $M_2 = (K_2, \bar{Z}_2, \delta_2, \Delta_2, F_2)$

Idee: de a combina M_1 și M_2 într-un APD M care realizează opțiunile lui M_1 și M_2 în paralel și acceptă dc amândouă ar accepta.

$M = (K, \bar{Z}, \Gamma, \Delta, \Delta, F)$

$K = K_1 \times K_2$

$\bar{Z} = \bar{Z}_1 \cup \bar{Z}_2$

$\Gamma = \Gamma_1$

$\Delta = (\Delta_1, \Delta_2)$

$F = F_1 \times F_2$

$\Delta \ni (((q_1, q_2), u, \beta), ((p_1, p_2), \gamma))$

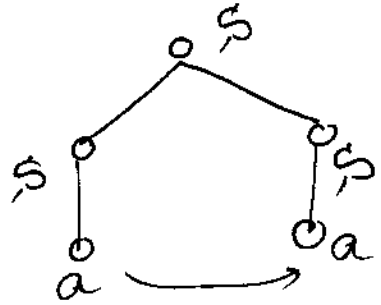
$\Leftrightarrow ((q_1, u, \beta), (p_1, \gamma)) \in \Delta_1$

$(q_2, u) \overset{*}{\vdash}_{M_1} (p_2, e)$

Propr. de periodicitate ale LiC

Fie $G = (\{S, a\}, \{a\}, \{S \to SS \mid a\}, S)$

$$S \underset{G}{\Rightarrow} SS \underset{G}{\overset{*}{\Rightarrow}} aa$$



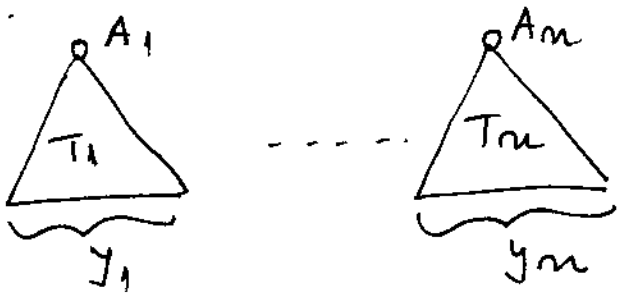
Def.
Fie $G = (V, \Sigma, R, S)$ o GiC. Def. arborele de derivare astfel:

1° o A
arb. de deriv. pt $\forall A \in V$

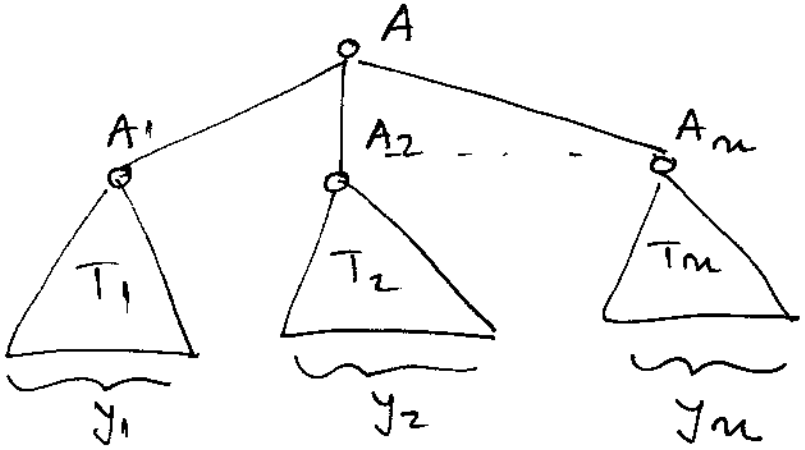
2° $A \to e \in R$



3° Dc.



sunt arb. de derivare ($n \geq 1$) cu rădăcinile $A_1 \dots A_n$ și și-nuri generate $y_1 \dots y_n$, $A \to A_1 \dots A_n \in R$,



4° Mimic altceva nu este un arb. de derivare

ex: $G = (V, \Sigma, R, S)$

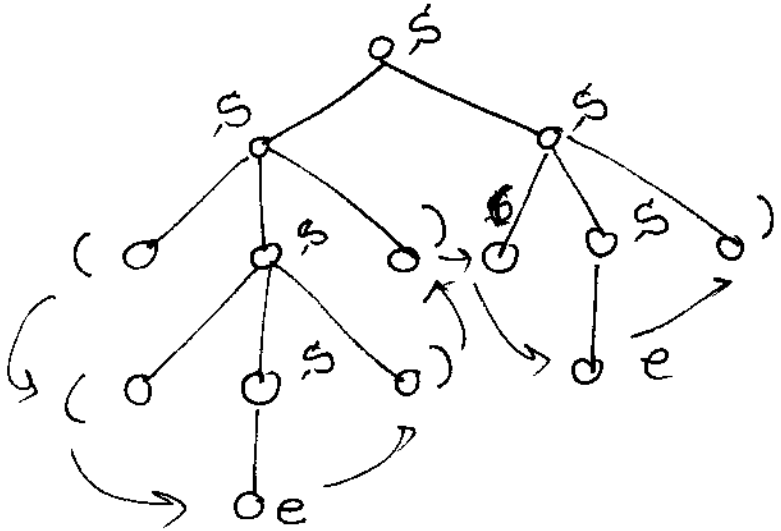
$V = \{S, (,)\}$

$\Sigma = \{(,)\}$

$R = \{S \to SS \mid (S) \mid e\}$

$S \overset{*}{\Rightarrow} (())()$

$S \underset{G}{\Rightarrow} SS \underset{G}{\Rightarrow} (S)S \underset{G}{\Rightarrow} ((S))S \underset{G}{\Rightarrow} (())S \underset{G}{\Rightarrow} (())(S) \underset{G}{\Rightarrow} (())()$



Teorema (Lema de pompare)

Fie G o GIC. Atunci există un număr K depinzând de G a.î. $\forall w \in L(G)$, $|w| > K$ poate fi scris ca $w = uvxyz$ a.î. $v \neq e$ sau $y \neq e$ și $uv^n xy^n z \in L(G)$, $\forall n \geq 0$.

Dem:

Fie $G = (V, \Sigma, R, S)$

este suficient să dem. că $\exists K$ a.î. $\forall w \in L(G)$, $|w| > K$, are o deriv. de forma:

$$S \underset{G}{\overset{*}{\Rightarrow}} uAz \underset{G}{\overset{*}{\Rightarrow}} uvAyz \underset{G}{\overset{*}{\Rightarrow}} uvxyz$$

unde $u, v, x, y, z \in \Sigma^*$, $A \in V - \Sigma$, $v \neq e$ sau $y \neq e$.

Atunci derivarea $A \overset{*}{\Rightarrow} vAy$ poate fi repetată de oricâte ori pt. a obține $uv^n xy^n z$.

Fie $p = \max\{|\alpha| \mid A \to \alpha \in R\}$

$\forall m \geq 1$, un arb. de derivare de înălțime m poate avea cel mult p^m frunze, deci poate genera un șir de lg. cel mult p^m.

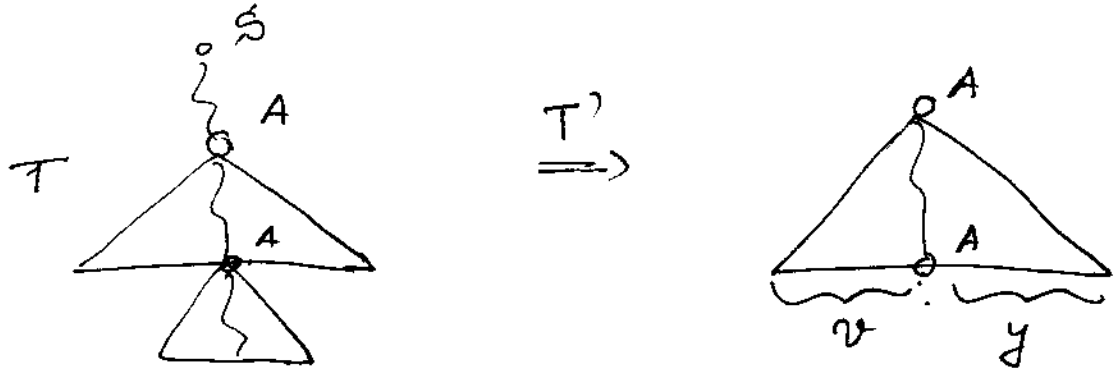
Fie $m = |V - \Sigma|$

$K = p^m$

$pp\ |w| > K$

Fie T un arb. de deriv. cu rād. S și șir generat w.

Alt. T are cel puțin o cale cu m. mult de $|V - \Sigma| + 1$ noduri, deci cel puțin o cale care include 2 noduri etichetate cu ac. simbol neterminal $A \in V - \Sigma$.



$v, y \in \Sigma^*$

Este posibil ca $v=y=c$, dar nu poate fi cazul pt. orice alegere a căii și orice 2 noduri pe ea cu ac. etichetă.

Dc $v=y=c$, at. arb. T' poate fi scos din T, fără a schimba șirul generat de arbore, agățând arb. de derivare cu nodul cel mai jos ca rădăcină, de nodul A de mai sus. Dc. toate căile de lg. $>m$ ar putea fi scurtate astfel, fără a schimba șirul generat, am putea obține un arb. care generează w cu înălțime $< m \Rightarrow$ imposibil.

Teorema

$L = \{a^n f^n c^n \mid n \geq 0\}$ nu este LIC.

Dem:

prin reducere la absurd:

pp. $L = L(G)$, $G \to G$ IC

Fie $K(G)$ și $n > K/3$

$w = a^n f^n c^n \in L(G)$ are o repriz. $w = uvxyz$ a.î. $v \neq e$ sau $y \neq e$ și $uv^i x y^i z \in L(G)$, $\forall i \geq 0$.

Impasibil $\Rightarrow$ D.v sau y conține 2 simb din $\{a, f, c\} \Rightarrow uv^2xy^2z$ conține un 'f' înainte de 'a' sau un 'c'.

Teorema

Limbajele indep. de context nu sunt închise în rap. cu intersecția. sau complementarea.

Dem.

$L_1 = \{a^n f^n c^m : n, m \geq 0\}$

$L_2 = \{a^m f^n c^n : m, n \geq 0\}$. | LiC

$L_1 \cap L_2 = \{a^n f^n c^n \mid n \geq 0\} \Rightarrow$ ~~LiC~~

Dc. LiC ar fi fost închise în rap. cu complementarea $\Rightarrow$ ar fi fost închise în rap. cu intersecția.

$$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$$

Proprietăți algoritmice

Teorema

Există algoritmi care pot răspunde la următoarele întrebări ref. la GIC.

a) Fie o GIC G și un șir w, $w \in L(G)$?

b) $L(G) = \emptyset$?

Obs:

$L(G_1) = L(G_2)$ nu poate fi rezv. alg.

Dem:

a) Idee: inspectăm toate derivările posibile până la o anumită lg, respectiv toți arb. de deriv. până la o anumită dimens.

Dc. fac regulă $A \to u$, $u \in \Sigma$ sau $|u| \geq 2 \Rightarrow$ nicio derivare de lg $2|w|-1$ nu folosește inspectată.

Pot. rezolva dc convertesc G într-o gram. echivalentă care nu are reguli $A \to u$, unde $u = e$ sau $u \in V - \Sigma$.

Eliminăm $A \to e$ cu excepția $S \to e$ dc $e \in L(G)$.

Fie $G = (V, \Sigma, R, S)$

$A \to e$

$B \to uAv$, $u, v \in V^*$

adg. $B \to uv$ la R

alg. se repetă până când nu mai pot fi adg. reguli.

la sf elimin $A \to e$.

Fie G' gram. obț. prin acest fel de transf. a lui A are urm. propr:

1° $L(G') = L(G)$

2° $e \in L(G)$, $S \to e \in R(G')$

3° $\forall w \in L(G') \setminus \{e\}$ are o deriv (arb. de deriv) în care nu este ut. $A \to e$.

Elim. reg. $A \to B$.

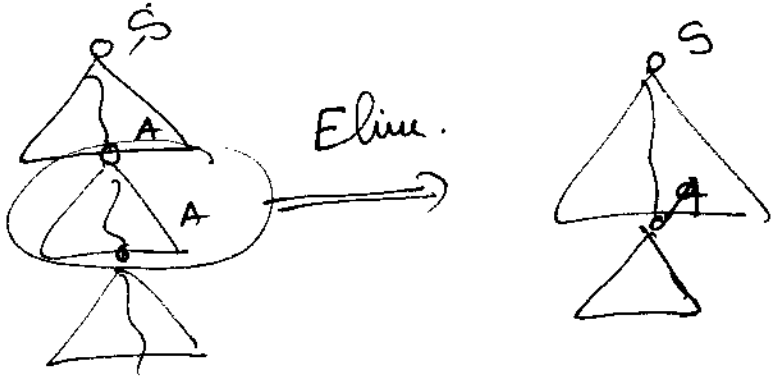
$\forall A, B$, dc $A \overset{*}{\underset{G}{\Rightarrow}} B$

$B \to u$, $A \overset{*}{\underset{G}{\Rightarrow}} B$ $\longmapsto$ se adg. $A \underset{G}{\to} u$

La sf. elimin reg. $A \to B$

(1) Fie $G = (V, \Sigma, R, S)$ $\overset{?}{\Rightarrow}$ generează $L(G) \neq \emptyset$

$\Rightarrow$ $\exists$ arb. de deriv. cu $\hat{s}$ir generat (terminale) de lungime cel mult $|V - \Sigma|$.



Proc. poate fi repetat $\Rightarrow$ arb de înălț $|V-\Sigma|$. $\Rightarrow$ $\forall$ arb. de înălț $|V-\Sigma|$ poate fi inspectat.

Pr 1.

Fie GiC :

$S \to aSbS \mid bSaS \mid e$

S.s. dem. $L(G)$ este mulțimea șirurilor cu nr. egal de 'a' și 'b'.

$L = L(G)$

$L' = \{w \in \{a,b\}^* \mid \#a(w) = \#b(w)\}$.

$L = L(G)$ trebuie să dem. $L(G) \subseteq L'$ și $L' \subseteq L(G)$

1) $L(G) \subset L'$

Vreau să arăt că $\forall w \in L(G)$, w are nr. egal de 'a' și 'b'. Dem. prin inducție după nr. de pași ai derivării $S \overset{*}{\underset{G}{\Rightarrow}} w$

Caz de bază:

$w = e$, $S \underset{G}{\Rightarrow} e$, w are un nr egal (0) de 'a' și 'b'

Ip. inductivă:

Dc. $S \overset{\leq}{\underset{G}{\Rightarrow}} w$ în m. puțin de n pași $\Rightarrow$ w are nr. egal 'a' și 'b'

Pas de inducție

Fie w a.î. $S \overset{*}{\underset{G}{\Rightarrow}} w$ în p pași

Dc. w începe cu 'a', $S \underset{G}{\Rightarrow} aS$ + $S \overset{*}{\Rightarrow} aw_1$ + w_2

$S \overset{*}{\underset{G}{\Rightarrow}} w_1$ și $S \overset{*}{\underset{G}{\Rightarrow}} w_2$ în cel mult $n-1$ pași. Din ip. ind. w_1 și w_2

au nr. egal de 'a' și 'b'. w are nr egal $((|w_1|+|w_2|)/2+1)$ de 'a' și 'b'.

Dc. w începe cu 'b', $S \underset{G}{\Rightarrow} bSaS \overset{*}{\Rightarrow} bw_1aw_2 \Rightarrow$ ac doua.

2) $L' \subseteq L(G)$

$\forall w \in \Sigma^*$, w are nr. egal de 'a' și 'b', $S \overset{*}{\underset{G}{\Rightarrow}} w$. Dem. prin ind. după $|w|$.

Caz de bază $w = \varepsilon$, $S \overset{0}{\underset{G}{\Rightarrow}} \varepsilon$

Ip. ind.

$|w| < n$, w are nr. egal de 'a' și 'b' $\Rightarrow S \overset{*}{\underset{G}{\Rightarrow}} w$

Pas de inductie

Fie w cu nr. egal de 'a' și 'b'. Pp. că w începe cu 'a'.

Idee: vreau să găsesc primul caracter după 'a' a.î. $\#_a - \#_b = 0$.

→ Pt. că am început cu 'a', $\#_a = 1$ ⇒ găsesc un caract a.î. $\#_a - \#_b = 0$ ⇒ b.

⇒ $w = a w_1 b w_2$

$\downarrow$

$\#_a = \#_b$

$a w_1 b \Rightarrow \#_a = \#_b$.

$\cdot w \Rightarrow \#_a = \#_b \Rightarrow w_2 \Rightarrow \#_a = \#_b$.

$|w_1| < n$, $|w_2| < n$ | p.ind. → $S \overset{*}{\underset{G}{\Rightarrow}} w_1$, $S \overset{*}{\underset{G}{\Rightarrow}} w_2$

$S \underset{G}{\Rightarrow} aSbS \overset{*}{\underset{G}{\Rightarrow}} aw_1bS \overset{*}{\underset{G}{\Rightarrow}} aw_1bw_2 = w$.

→ Dc w începe 'b', $S \underset{G}{\Rightarrow} bSaS \overset{*}{\underset{G}{\Rightarrow}} bw_1aS \overset{*}{\underset{G}{\Rightarrow}} bw_1aw_2 = w$

Exemplu grilă

1° Fie $L \to$ limbaj și a un simbol. Def: $a \backslash L = \{w \mid aw \in L\}$

ex: $L = \{0, 001, 100\}$, $a \backslash L = \{\varepsilon, 01\}$

Știind că $L \to$ reg, $a \backslash L$ este:

a) L.R.

b) Lic

c) ?

dem.

$L = L(K, \Sigma, \delta, q_0, F)$

$a \backslash L = L(K, \Sigma, \delta, q_0', F)$

$q_0' = \delta(q_0, a)$

2° Fie $w = a_1 \dots a_n$, $x = t_1 \dots t_n$, $|w| = |x|$

Def. alt $(w, x) \to$ șirul în care simb. din w și x alternează, începând cu u

$(a_1 t_1 a_2 t_2 \dots a_n t_n)$

$L, M \rightarrow$ limbaje

$alt(L, M) = \{alt(w,x) \mid w \in L, x \in M, |w| = |x|\}$.

Dc $L, M \rightarrow$ regulate, $alt(L, M)$ este:

(a) L.R.

~~b)~~ L.I.C.

c) ?

dem.

$L = L(M_1)$
$M = L(M_2)$, $M_1, M_2 \rightarrow AFD$

$M_3 = (K_1 \times K_2 \times \{0,1\}, \Sigma_1 \cup \Sigma_2, \delta', ((q_1, q_2), 0), F_1 \times F_2 \times \{0\})$

$\delta'((q_a, q_b), 0), c) = (\delta_1(q_a, c), q_b, 1)$

$\delta'((q_a, q_b), 1), c) = (q_a, \delta_2(q_b, c), 0)$

(3) Fie alfabetul $\Sigma = \{0\}$ și fie

M_1 = mulțimea LiC peste Σ

M_2 = mulțimea LR peste Σ

a) $M_1 \subset M_2$

b) $M_1 = M_2$

c) $M_1 \supset M_2$

(4) Fie GiC, G, def. prin reg. $S \to aS \mid Sb \mid a \mid b$.

Şirurile din $L(G)$:

a) conțin "ba"

b) nu conțin "ba"

c) doar unele conțin "ba"

(5) Fie $L \to$ ~~LiC~~ și S o mulțime finită de șiruri:

Atunci $L' = L \setminus S$ este

a) LR.

b) LiC.

c) ~~LiC~~

dem.

pp. $L' \to$ LiC , $L' = L(G')$

construim. $G = G' \cup \{S' \to w \mid \forall w \in S\}$

$L \to \cancel{LiC} \Rightarrow L' \to \cancel{LiC}$

⑥ Fie $L_1 \to LiC$, $L_2 \to \cancel{LiC}$. Af. $L_1 \cap L_2 \to \cancel{LiC}$

a) A

b) F

Dem.

$L_2 = \overline{L_1}$, $L_2 \to \cancel{LiC}$

$L_1 \cap L_2 = \emptyset$ LiC

⑦ $L \to LiC$, $R \to LR$. $(\overline{L \cup R}) \to \cancel{LiC}$

a) A

b) F

Dem.

$R = \Sigma^*$

$L \cup R = \Sigma^*$

$\overline{\Sigma^*} = \emptyset$

(8) Pt. orice limbaj L, rezultatul L^*:

a) finit $L = \emptyset$

b) infinit

c) ?

(9) Se dau urm. gram:

G1: $A \to a \mid aA \mid bAA \mid AbA \mid Aab$

G2: $B \to b \mid bB \mid aBB \mid BaB \mid Bba$

G3: $C \to bCa \mid aCb \mid ba \mid ab \mid e$

Limbajul cu $\#a \geq \#b$ este generat de următoarea gram:

a) G1 și G2

b) G1 și G3

c) G2 și G3

(10) Fie $L = \{ w \in \{a, b, c\}^+ \mid \#a(w) = 2 * \#b(w)$ și

$\#c(w) = \#b(w)$ și

$|w| \to$ impar $\}$

a) LR

b) LiC

c) LDC

R: $L = \emptyset \in LR$.

(11) Alegeți enunțul fals:

a) APD poate cicla la infinit

(b) o cond. suf. ca APD să accepte șirul de intrare este s.s. ajle într-o stare finală cu stiva vidă

c) $\nexists$ LiC se poate construi un APD care să-l accepte.

Tehnici de construcție a GIC

→ a^* : $A \to aA \mid e$

→ $a^n f^n$: $S \to aSf \mid e$

→ $a^n f^{kn}$: $S \to aS\underbrace{ff\ldots f}_{k} \mid e$

→ $a^n f^m$, $m > n$:

$\|\|$

$a^n f^n f^k$, $k > 0$

$S \to XY$

$X \to aXf \mid e$

$Y \to fY \mid f$

→ $a^n f^m$, $m \neq n$

$L = L_1 \cup L_2$

$L_1 = \{a^n f^m \mid n < m\}$

$L_2 = \{a^n f^m \mid n > m\}$

→ $a^n f^m c^m d^n$, $\forall$ rel (m, n)

$S \to a S d \mid X$

$X \to f X c \mid e$

NU se poate GIC

→ să controlăm m. mult de 2 parametri: $a^n f^m c^n$

→ să contr. 2 parametri. care nu sunt sufixe sau prefixe

$a^n f^m c^n d^m$

Procedura

O procedură constă într-o mulțime finită de instrucțiuni care pot fi executate mecanic, într-un timp determinat, cu o cantitate fixă de efort. Proc. poate avea 0 sau mai multe intrări / ieșiri.

Algoritm

O procedură se oprește pt o anumită intrare, dc $\exists t$ (finit) a.î. după executarea a t instrucțiuni elementare, fie nu mai există o altă instr. de executat, fie instrucțiunea 'halt' a fost executată. Procedura care se oprește pt toate intrările → algoritm.

Funcții recursive

O procedură definește o relație între intrări → ieșiri numită funcție recursivă parțială.

Dc. procedura → algoritm → funcție recursivă totală.

Procedură → a defini un limbaj: pt un șir x, procedură decide dc $x \in L$ (limbaj)

O mulțime def. printr-o proc → recursiv enumerabilă

alg → recursivă.

Y un număr mare de notații formale ut. pt. a descrie o procedură:

1) M. Turing (Turing, 1936-1937)
2) gramatica Chomsky 0 (Chomsky, 1959-1963)
3) Alg. Markov (Markov, 1951)
4) Calcul Lambda (Church, 1941)
5) Sistem Post
6) Tag
7) Cele mai multe limbaje de programare

Ierarhia Chomsky

gramaticile pot fi clasificate în raport cu forma regulilor lor:

Fie $G = (V, \Sigma, R, S)$

1) Regulate (la dreapta)

$A \to xB \mid x \quad , x \in \Sigma^*, \; A, B \in V - \Sigma$

2) Independente de context

$A \to \alpha \quad , A \in (V - \Sigma) \, , \alpha \in V^*$

3) Dependente de context

$\alpha A \beta \to \alpha \delta \beta$

$|\alpha A \beta| \leq |\alpha \delta \beta|$

$\alpha, \beta \in V^* \quad , A \in V - \Sigma$

4) O gramatică care nu are restricțiile 1 ÷ 3 => G. nerestricționată

$V^* (V - \Sigma) V^* \times V^*$

Pt fiec. clasă de gram. din ierarhia Chomsky → clasă de acceptoare care acc. aceeași clasă de limbaje:

(1) L → regulat ⇒ A.F. (D/N)

(2) L → Lic ⇒ A.P.D. (nedeterminist)

(3) L → LDC ⇒ 2-way nondeterministic liniar bounded automaton

(4) L → recursiv enumerabil ⇒ M. Turing

Ex: GDC; ? G a.î. $L(G) = \{a^n b^n c^n \mid n \geq 0\}$.

$S \to aSBC \mid abC$

$cB \to Bc$

$bB \to bb$

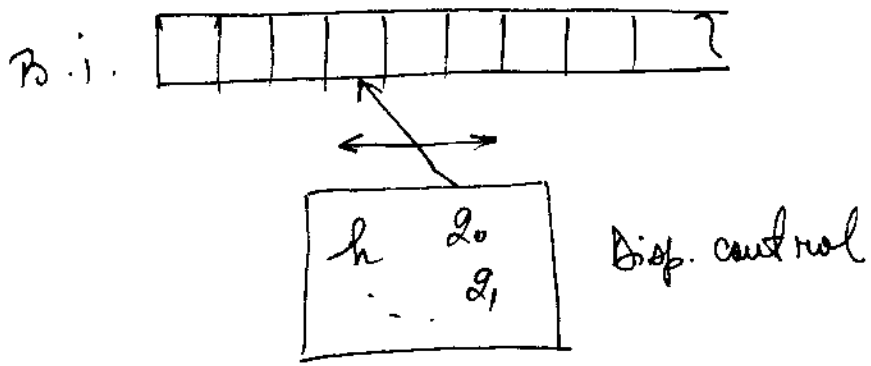
$cC \to cc$

$bC \to bc$

$S \Rightarrow aSBC \Rightarrow aabCBC \Rightarrow aabBCC \Rightarrow aabbCC \Rightarrow aabbcc$

Masina Turing

M.T. constă dintr-un disp. control și o bandă. Comunicarea dintre cele 2 componente este făcută printr-un cap care poate citi / în locui simb. pe bandă.



Mecanismul de control acționează în pași discreți; la fec. pas execută 2 acțiuni: într-un mod care depinde de <u>starea curentă</u> și de simbolul de pe bandă indicat de capul citire/scriere:

1) trece disp. de control într-o nouă stare

2) Fie
   a) scrie un simbol pe bandă în poz. curentă, în locul-ul pe al existent
   b) deplas. cap. o poz. la stg / dr

B.i. → are capăt stg, se întinde la dr. indefinit

Dc. M.T. încearcă să mute capul peste limita stg. a B.i. → înlănțuie să funcționeze.

Există o stare specială - de halt - h ⇒ încheierea efilor M.T.

Convenție → B.i. conține în fiecare pătrat → '#'

L, R → deplas. capului o poziție la stg / dr.

Def.

O M.Turing este un tuplu (K, Σ, δ, s), unde:

K → mulțimea finită a stărilor, $\ni h$

Σ → alfabetul de intrare, $\ni \#$, $\not\ni L, R$

$s \in K$ → st. inițială

$\delta : K \times \Sigma \to (K \cup \{h\}) \times (\Sigma \cup \{L, R\})$

Dc. $q \in K$, $a \in \Sigma$, $\delta(q, a) = (p, b)$

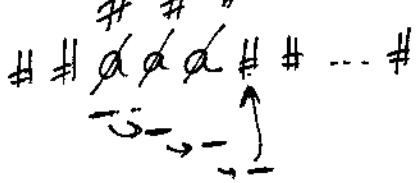
ex: ? M.T. $M = (K, \Sigma, \delta, s)$ care înlocuiește toate simb 'a' cu '#' mergând spre dr.

$K = \{q_0, q_1\}$

$\Sigma = \{a, \#\}$

$s = q_0$

q	∇	$\delta(q, \nabla)$
q_0	a	$(q_1, \#)$
q_0	#	$(h, \#)$
q_1	a	(q_0, a)
q_1	#	(q_0, R)



ex: ? M.T. $M = (K, \Sigma, \delta, s)$ care se deplas. la stg. până la # și se oprește

$K = \{q_0\}$

$\Sigma = \{a, \#\}$

$s = q_0$.

q	∇	$\delta(q,\nabla)$
q_0	a	(q_0, L)
q_0	$\#$	$(h, \#)$

Def:

Configurația unei M.T. $M = (K, \bar{\Sigma}, \delta, s)$ este un membru din:

$$(K \cup \{h\}) \times \bar{\Sigma}^* \times \bar{\Sigma} * (\bar{\Sigma}^* (\Sigma - \{\#\}) \cup \{e\})$$

Def:

Fie $M = (K, \bar{\Sigma}, \delta, s)$ și fie $(q_1, w_1, a_1, u_1), (q_2, w_2, a_2, u_2)$ config. ale lui M.

Atunci:

$$(q_1, w_1, a_1, u_1) \underset{M}{\vdash} (q_2, w_2, a_2, u_2)$$

$\Leftrightarrow$ $h \in \bar{\Sigma} \cup \{L, R\}$

$$\delta(q_1, a_1) = (q_2, h)$$

1) $t \in \bar{\Sigma}$, $w_1 = w_2$, $u_1 = u_2$, $a_2 = t$

2) $t = L$, $w_1 = w_2 a_2$

a) $u_2 = a_1 u_1$, dc $a_1 \neq \#$, $u_1 \neq e$

b) $u_2 = e$, $a_1 = \#$, $u_1 = e$

3) $t = R$, $w_2 = w_1 a_1$

a) $u_1 = a_2 u_2$

b) $u_1 = u_2 = e$, $a_2 = \#$

Obs:

$t = L$, $w_1 = e$ $\Rightarrow$ (q_1, w_1, a_1, u_1) nu duce la nicio config. pt. că

$\nexists\, w_2 \in \bar{\Sigma}^*$, $a_2 \in \bar{\Sigma}$ aș $w_1 = w_2 a_2$

$\Rightarrow$ M.T. se agață.

Def:

$\vdash_M^*$ $\Rightarrow$ închiderea reflexivă și tranzitivă a $\vdash_M$

Calcul cu M. Turing

Convenții:
→ șirul de intrare este scris aliniat la limita stg. a benzii cu capul poziționat pe '#' care delimit. dr. șirul
→ aceeași convenție pt poz. capului pt. rezultatul calculat.

Def:
Fie $\Sigma_0, \Sigma_1 \not\ni \#$. Fie $f: \Sigma_0^* \to \Sigma_1^*$. M. Turing $M = (K, \Sigma, \delta, s)$ calculează f, dc $\Sigma_0, \Sigma_1 \subseteq \Sigma$, $\forall w \in \Sigma_0^*$, $f(w) = u$ atunci $(s, \#w\underline{\#}) \vdash_M^* (h, \#u\underline{\#})$

Dc $\exists M \Rightarrow f$ → funcție Turing calculabilă.

ex: $\Sigma_0 = \Sigma_1 = \{a, b\}$
$f: \Sigma_0^* \to \Sigma_1^*$
$f(w) = \bar{w}$, $\bar{w}$ este rezultatul în locuirii $a \to b$ în w.

$M = (K, \Sigma, \delta, \Delta)$

$K = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b, \#\}$.

$\Delta = q_0$.

δ:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_1, L)
q_0	b	(q_1, L)
q_0	$\#$	(q_1, L)
q_1	a	(q_0, b)
q_1	b	(q_0, a)
q_1	$\#$	(q_2, R)
q_2	a	(q_2, R)
q_2	b	(q_2, R)
q_2	$\#$	$(h, \#)$

$(q_0, \# aa\underline{b}\#) \vdash_M (q_1, \# aa\underline{b})$

$\vdash_M (q_0, \# aa\underline{a})$

$\vdash_M (q_1, \# a\underline{a}a)$

$\vdash_M (q_0, \# a\underline{b}a)$

$\vdash_M (q_1, \# \underline{a}ba)$

$\vdash_M (q_0, \underline{\#}bba)$

$\vdash_M (q_1, \underline{\#}bba)$

$\vdash_M (q_2, \#\underline{b}ba)$

- - - - -

$\vdash_M (q_2, \#bba\underline{\#})$

$\vdash_M (h, \#bba\underline{\#})$

Noțiunea de funcție calculabilă pe șiruri poate fi extinsă:

$f : (\Sigma_0^*)^k \to \Sigma_1^*$, $k > 0$

$M = (K, \Sigma, \delta, s)$, $\Sigma_0, \Sigma_1 \subseteq \Sigma$.

$\forall w_1, \dots, w_k \in \Sigma_0^*$, $f(w_1, \dots, w_k) = u$, atunci $(s, \# w_1 \# w_2 \# \dots \# w_k \#) \overset{*}{\underset{M}{\vdash}} (h, \# u \#)$.

Calculab. cu M. Turing

Def.
Fie $\Sigma_0, \Sigma_1 \not\ni \#$ alfabete, $f: \Sigma_0^* \to \Sigma_1^*$. M. Turing $M = (K, \Sigma, \delta, s)$ calc. f dc. $\Sigma_0, \Sigma_1 \subseteq \Sigma$, $\forall w \in \Sigma_0^*$, $f(w) = u$, atunci $(s, \#w\#) \overset{*}{\underset{M}{\vdash}} (h, \#u\#)$.

Extinderea noțiunii de fcție calc. Turing.

A) $f: \mathbb{N} \to \mathbb{N}$

Fie $I \neq \#$, $\forall n \in \mathbb{N}$, $n \overset{Not}{=} I^n$

O fcție $f: \mathbb{N} \to \mathbb{N}$ este calc. de o M.T. M, dc. M calc. fcția

$f': \{I\}^* \to \{I\}^*$, unde $f'(I^n) = I^{f(n)}$, $n \in \mathbb{N}$.

Prin generalizare:

O M.T. calc $f: \mathbb{N}^k \to \mathbb{N}$, dc calc. $f': (\{I\}^*)^k \to \{I\}^*$, unde

$f'(I^{n_1}, .., I^{n_k}) = I^{f(n_1, .., n_k)}$, $\forall n_1, .., n_k \geq 0$.

ex:

Fie f fctia succesor: $f(n) = n+1$, $n \in \mathbb{N}$

$M = (K, \Sigma, \delta, s)$

$K = \{q_0, h\}$

$\Sigma = \{I, \#\}$

$s = q_0$

δ:

q	∇	$\delta(q, \nabla)$
q_0	I	(h, R)
q_0	$\#$	(q_0, I)

$(q_0, \#II\underline{\#}) \vdash_M (q_0, \#II\underline{I}) \vdash_M (h, \#III\underline{\#})$

în general:

$(q_0, \#I^n\underline{\#}) \vdash_M^* (h, \#I^{n+1}\underline{\#})$

$n=0$, $(q_0, \#\underline{\#}) \vdash_M (q_0, \#\underline{I}) \vdash_M (h, \#I\underline{\#})$

B) Limbaj decidabil Turing

Def.

Fie $\Sigma_0 \not\ni \#$ un alfabet. Atunci limbajul $L \subseteq \Sigma_0^*$ este decidabil Turing dc și numai dc. funcția $\chi_L : \Sigma_0^* \to \{\textcircled{T}, \textcircled{N}\}$ este calculabilă Turing, unde $\forall w \in \Sigma_0^*$,

$$\chi_L(w) = \begin{cases} \textcircled{T}, & w \in L \\ \textcircled{N}, & w \notin L \end{cases}$$

ex:

Fie $\Sigma_0 = \{a\}$, $L = \{w \in \Sigma_0^* \mid |w| \text{ pară}\}$. $M = (K, \Sigma, \delta, s)$ este o proc. de decizie pt L.

$K = \{q_0, q_1, \dots, q_6\}$

$\Sigma = \{a, \textcircled{T}, \textcircled{N}, \#\}$

$s = q_0$

δ:

q	∇	$\delta(q, \nabla)$
q_0	#	(q_1, L)
q_1	a	$(q_2, \#)$
q_1	#	(q_4, R)
q_2	#	(q_3, L)
q_3	a	$(q_0, \#)$
q_3	#	$(q_6, \#)$
q_4	#	$(q_5, \textcircled{T})$
q_5	$\textcircled{T}$	(h, R)
q_5	$\textcircled{N}$	(h, R)
q_6	#	$(q_0, \textcircled{N})$

$(q_0, \# a^n \underline{\#}) \overset{*}{\underset{M}{\vdash}} (h, \# \textcircled{T} \underline{\#})$, n este par

$(q_0, \# a^n \underline{\#}) \overset{*}{\underset{M}{\vdash}} (h, \# \textcircled{N} \underline{\#})$, n este impar

c) Limbaj Turing acceptat

Def.

Fie $\Sigma_0 \not\ni \#$, alfabet. Spunem că o M. Turing acceptă șirul $w \in \Sigma_0^*$, dacă M.T. se oprește pt w.

M.T. M acceptă limbajul $L \subseteq \Sigma_0^*$ $\Leftrightarrow$ $L = \{ w \in \Sigma_0^* \mid M \text{ acceptă } w \}$.

ex:

$\Sigma_0 = \{a, b\}$

$L = \{ w \in \Sigma_0^* \mid w \text{ conține cel puțin 'a'} \}$.

L este acceptat de $M = (K, \Sigma, \delta, s)$,

$K = \{q_0\}$

$\Sigma = \{a, b, \#\}$

$s = q_0$

δ:

q	σ	$\delta(q, \sigma)$
q_0	a	(h, a)
q_0	b	(q_0, L)
q_0	$\#$	(q_0, L)

Compunerea M. Turing

Lema

Fie M o M. Turing și fie $(q_i, w_i \underline{a_i} u_i)$, $i = 1, 2, 3$ config. ale M. T. M.

Dacă $(q_1, w_1 \underline{a_1} u_1) \overset{*}{\underset{M}{\vdash}} (q_2, w w_2 \underline{a_2} u_2)$, pt un anumit w

și $(q_2, w_2 \underline{a_2} u_2) \overset{*}{\underset{M}{\vdash}} (q_3, w_3 \underline{a_3} u_3)$

atunci $(q_1, w_1 \underline{a_1} u_1) \overset{*}{\underset{M}{\vdash}} (q_3, w w_3 \underline{a_3} u_3)$.

Ideea dem: dacă M nu se agață în timpul efectuării:

$$(q_2, w_2 \underline{a_2} u_2) \overset{*}{\underset{M}{\vdash}} (q_3, w_3 \underline{a_3} u_3)$$

și cum M nu are o modalitate directă de a detecta capătul stg. al benzii nu trebuie să facă nicio încercare în timpul ac. efect. de a-și muta capul la stg. primului simbol din $w_2 a_2 u_2$. Dar apoi dc. poziția din config. $(q_2, w w_2 \underline{a_2} u_2)$ va ajunge în $(q_3, w w_3 \underline{a_3} u_3)$ fără a pătrunde în primele $|w|$ pătrate ale benzii.

Mașini Turing de față:

1) scriere simboluri

$|\Sigma|$ mașini de scriere simboluri

$M_a = (K, \Sigma, \delta, s)$

$K = \{q_0, q_1\}$

$s = q_0$

$\delta(q, h) = (h, a)$, $h \in \Sigma$.

$M_a \overset{Not}{=} a$

2) deplas. L/R

$V_L = (\{q_0, q_1\}, \Sigma, \delta_L, q_0) \overset{Not}{=} L$

$\delta_L(q, a) = (h, L)$, $a \in \Sigma$

$V_R = (\{q_0, q_1\}, \Sigma, \delta_R, q_0) \overset{Not}{=} R$

$\delta_R(q, a) = (h, R)$, $a \in \Sigma$

Reguli de compunere a M. Turing

ex: $M_1, M_2 \rightarrow$ M. Turing

$> M_1 \rightarrow M_2$

ex: M_1, M_2, M_3

$> M_1 \xrightarrow{a} M_3$

$\downarrow b$

M_2

Def.

O schemă de M. Turing este un tuplu $(\mathcal{M}, \eta, M_0)$,

$\mathcal{M} \rightarrow$ o mulțime finită de M.T. cu alfabetul comun Σ și mulțimi de stări disjuncte

$M_0 \in \mathcal{M} \rightarrow$ masina inițială

$\eta \rightarrow$ fdie, $\eta : \mathcal{M} \times \Sigma \rightarrow \mathcal{M}$

Fie $\bar{M}$ → M.T. compusă

$\bar{M}$ începe efectuarea simulând efectuarea M.T. inițiale M_0. Când M_0 se oprește, $\bar{M}$ începe efectuarea altei M.T. din $\mathcal{M}$.

$\eta(M_0, a)$ → nedef. ⇒ $\bar{M}$ se oprește

$\eta(M_0, a) = M \in \mathcal{M}$ ⇒ $\bar{M}$ continuă efectuarea din st. inițial a lui M.

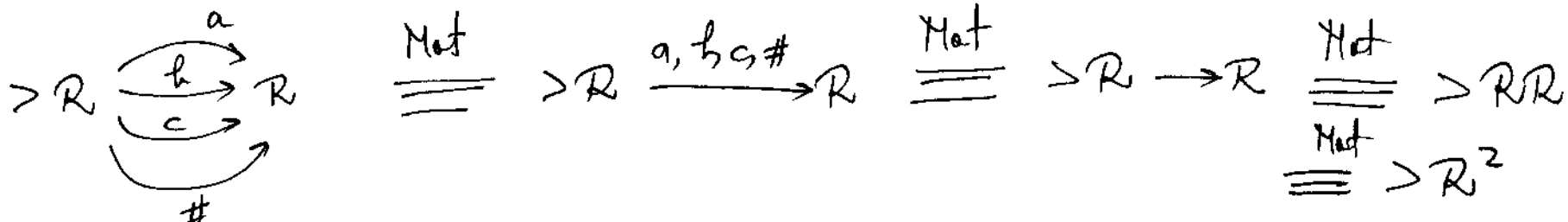
Reguli de reprezentare a unei scheme $(\mathcal{M}, \eta, M_0)$:

1° M.T. M_0 este marcată cu >

2° $a \in \Sigma$, $\eta(M, a) = M'$, săgeată etichetată cu a, $M \xrightarrow{a} M'$

ex: $> R \xrightarrow{a} R$

ex: $\Sigma = \{a, b, c, \#\}$



Notație:

→ M^K ⇒ $\underbrace{M \ldots M}_{K}$

→ $\bar{\nabla}$ ⇒ orice simbol cu excepția ∇

→ $R_{\#}$ ⇒ M. Turing care se deplasează dr. până la #

> 

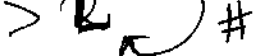
→ $L_{\#}$ ⇒ M. Turing care se deplasează stg. până la #

> 

→ $R_{\bar{\#}}$ ⇒ M. Turing care se deplas. la dr. până la $\bar{\#}$

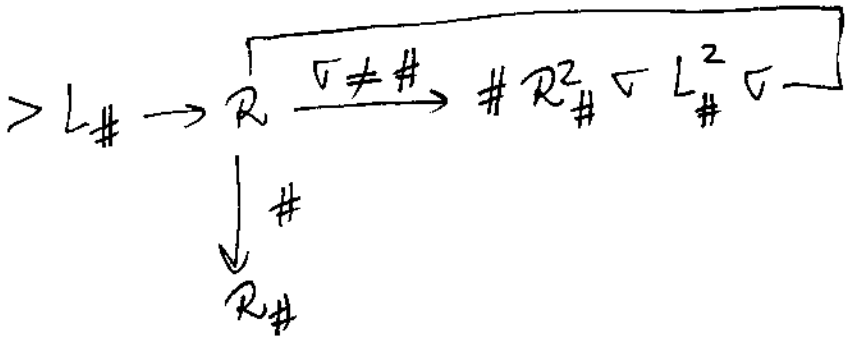
> 

→ $L_{\bar{\#}}$ ⇒ M. Turing care se deplas. la stg. până la $\bar{\#}$

> 

Ex: ? M. Turing de copiere

$(s, \#w\underline{\#}) \overset{*}{\underset{M}{\vdash}} (h, \#w\#w\underline{\#})$



$\#abc\underline{\#} \xrightarrow{L_\#} \underline{\#}abc\# \xrightarrow{R} \#\underline{a}bc\# \xrightarrow[\#]{a\neq\#} \#\#\underline{b}c\# \xrightarrow{R^2_\#} \#\#bc\#\underline{\#} \xrightarrow{a}$

$\xrightarrow{a} \#\#bc\#\underline{a} \xrightarrow{L^2_\#} \#\underline{a}bc\#a \xrightarrow{R} \#a\underline{b}c\#a \xrightarrow{\sigma=b\neq\#}$

$\xrightarrow{\#} \#a\underline{\#}c\#a \xrightarrow{R^2_\#} \#a\#c\#a\underline{\#} \xrightarrow{b} \#a\#c\#a\underline{b} \xrightarrow{L^2_\#}$

$\longrightarrow \#a\underline{\#}c\#ab \xrightarrow{b} \#a\underline{b}c\#ab \xrightarrow{R} \#ab\underline{c}\#ab \xrightarrow[\#]{\sigma=c\neq\#}$

$\longrightarrow \#ab\#\#ab \xrightarrow{R^2_\#} \#ab\#\#ab\underline{\#} \xrightarrow{c} \#ab\#\#abc \xrightarrow{L^2_\#} \#ab\underline{\#}\#abc$

$\xrightarrow{c} \#ab\underline{c}\#abc \xrightarrow{R} \#abc\underline{\#}abc \xrightarrow{R\#} \#abc\#abc\underline{\#}$

$\#u\#v\#w\#$ $\xrightarrow{C}$ $\#u\#v\#w\#w\#$

Extensii ale M. Turing

1° Permitem ca banda de intrare să fie nelimitată stg/dr.

2° Permitem m. multe benzi

3° Permitem m. multe capete pe bandă

4° Banda poate fi bidimensională

Banda nelimitată stg/dr.

$M = (K, \Sigma, \delta, s)$

O config. (q, w, a, u) , $q \in K$, $w, u \in \Sigma^*$, $a \in \Sigma$

unde w nu începe cu #

u nu se termină #

$(q, w, a, u) \overset{Not}{=} (q, wau)$

$\vdash_M$ este similară celei pt M.T. Standard

O dif. este că dc $\delta(q, a) = (p, L) \Rightarrow (q, \underline{a}u) \vdash_M (p, \underline{\#}au)$

$\delta(q, a) = (p, R)$, $(q, \underline{\#} au) \vdash_M (p, \underline{a} u)$

Lema

Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$ o M.T. cu bandă nelimitată stg/dr. Atunci există o M.T. Standard $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ cu $\forall w \in (\Sigma_1 - \{\#\})^*$ există rel:

a) Dacă M_1 se oprește pt w,

$(s_1, w\underline{\#}) \vdash_{M_1}^{*} (h, u\underline{a}v)$, $u, v \in \Sigma_1^*$, $a \in \Sigma_1$

atunci M_2 se oprește pt w,

$(s_2, \#w\underline{\#}) \vdash_{M_2}^{*} (h, \#u\underline{a}v)$

b) Dacă M_1 nu se oprește, nici M_2 nu se oprește pt w.

Def.

Spunem că o M.T. $M = (K, \Sigma, \delta, s)$ cu bandă nelim. stg/dr. calculează o funcție $f: \Sigma_0^* \to \Sigma_1^*$, $\Sigma_0, \Sigma_1 \not\ni \#$, dacă și numai dc. $\forall w \in \Sigma_0^*$

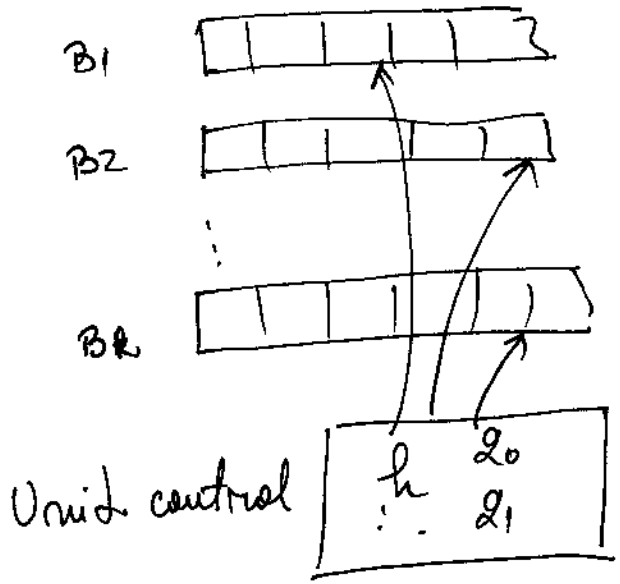
$f(w) = u \Rightarrow (s, \#w\underline{\#}) \vdash_M^{*} (h, u\underline{\#})$

Teorema

Orice funcție calculabilă, limbaj care este acceptat sau decis de o M.T. cu bandă nelimitată stg/dr este calc. respectiv limbajul este acceptat sau decis de o M.T. standard.

Mai multe benzi

Fie $\forall k > 0$ fixat, o M.T. cu k benzi constă dintr-o unit. de control și k benzi (limitate stg.). Fiecare bandă este conectată la unit. de control printr-un cap de citire/scriere.



Convenții:

1) Șirul de intrare este plasat pe 1ma bandă, aliniat la stg, cu capul pe #. pe # care delim. dreapta șirul.
2) Celelalte benzi sunt inițial vide
3) La sf. operațiilor → rezultatul 1ma bandă

? M.T. cu k benzi

#w# → #w#w# , w nu conține #.

La început: 1ma Bandă : #w#
2a Bandă : #

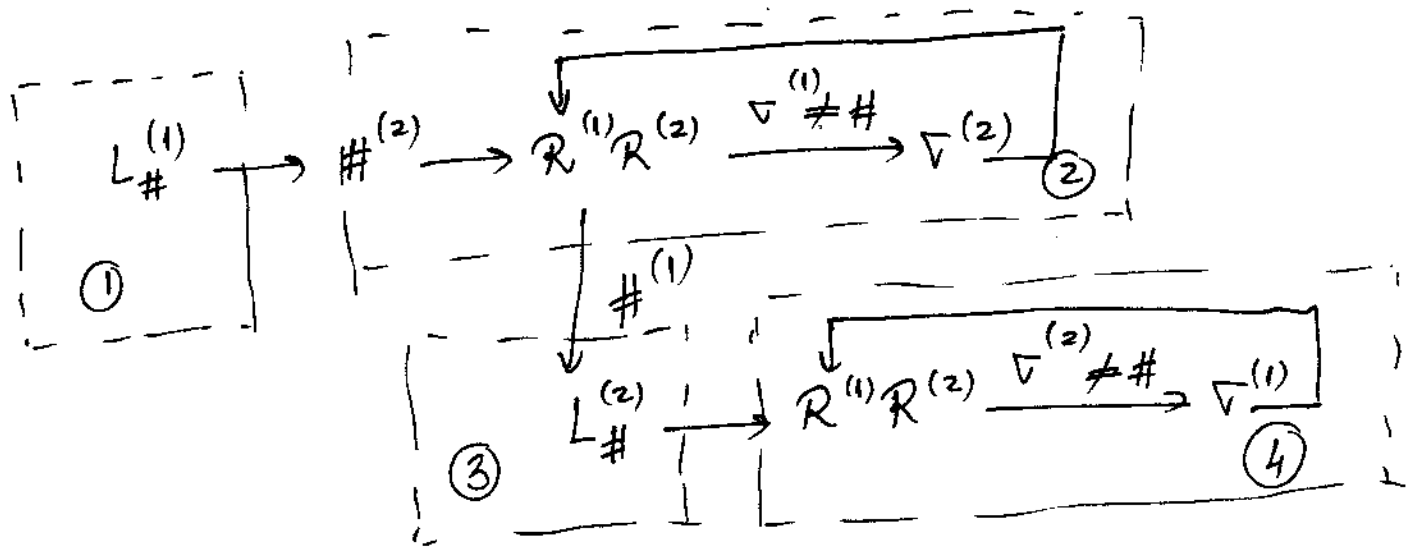
După (1): 1ma Bandă : #w#
2a Bandă : #

După (2): 1ma Bandă : #w#
2a Bandă : #w#

După (3): 1ma Bandă : #w#
2a Bandă : #w#

După (4): 1ma faudă : #w#w#

2a faudă : #w#



Def.
Configurația unei M.T. cu $k>0$ benzi este $(q, w_1 \underline{a_1} u_1, w_2 \underline{a_2} u_2, \ldots, w_k \underline{a_k} u_k)$.

Lema
Fie $k>0$ și M_1 o M.T. cu k-benzi, cu alf. Σ_1 și st. init. s_1. Atunci există o M.T.S. $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ cu $\Sigma_1 \subseteq \Sigma_2$ și $\forall w \in (\Sigma_1 - \{\#\})^*$,

a) Dacă M_1 se oprește pt w,

$(s_1, \#w\underline{\#}, \underline{\#}, \dots, \underline{\#}) \overset{*}{\underset{M_1}{\vdash}} (h, w_1 \underline{a_1} u_1, \dots, w_k \underline{a_k} u_k)$

pt. anumite $w_1, \dots, w_k, u_1, \dots, u_k, a_1, \dots a_k$, atunci M_2 se oprește pt w

$(s_2, \#w\underline{\#}) \overset{*}{\underset{M_2}{\vdash}} (h, w_1 \underline{a_1} u_1)$

b) Dc. M_1 se agață pt w, atunci M_2 se agață pt w.

c) Dc. M_1 nu se oprește pt w, nici M_2 nu se oprește.

Teorema

Orice funcție care se calculează, orice limbaj acceptat / decis de o M.T. cu k benzi, este calculată respectiv limbaj acceptat / decis de o M. T. standard.

Mai multe capete pe bandă

Într-un pas, pot citi / scrie, muta indep. pe bandă.

Convenție → dc 2 capete încearcă să scrie lucruri diferite pe ac. pătrat.

Teorema

M.T. cu nr. multe capete este echivalentă cu M. T. standard.

Bandă cu 2 dimensiuni

Se extind ofițe M.T. → suprascrie simb. din par. ortă
→ deplasare L, R, U, D

Teorema
M.T. cu bandă cu 2 dimensiuni poate fi simulată de o M.T. standard.

M.T. Nedeterministă

Def.
O M.T. Nedeterministă este un tuplu $M = (K, \Sigma, \Delta, s)$, $K, \Sigma, s \rightarrow$ ac. semnif. ca în cazul M.T. Standard, $\Delta \subseteq (K \times \Sigma) \times ((K \cup \{h, y\}) \times (\Sigma \cup \{L, R\}))$.

Def.
Config, $\vdash_M$, $\vdash_M^*$ → def. similar cu M.T.S.

Obs:
$\vdash_M$ => o relație

Cum o MTN poate avea 2 rezultate dif. pt ac. intrare => ? rezultat.

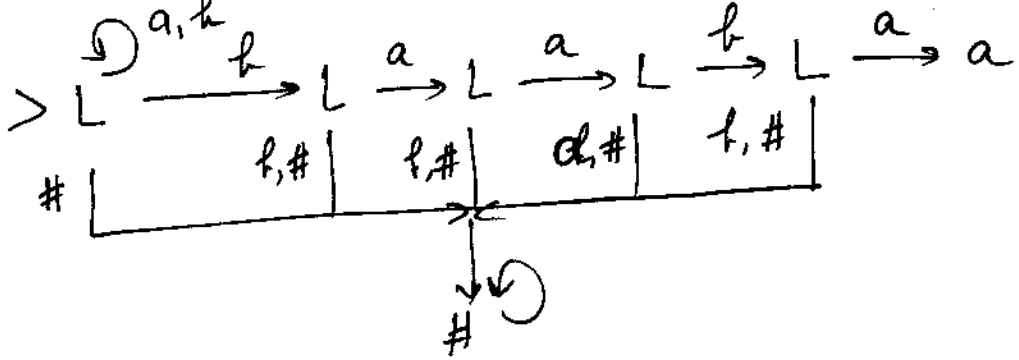
Soluție 1. => MTN. ca acceptor (doar dc se oprește, nu ce rămâne pe bandă)

Soluție 2 => la calculul unei funcții → același rezultat pe toate căile

ex:

$L = \{ w \in \{a, b\}^* \mid w$ conține cel puțin o apariție a subșirului $abaab \}$.

Intrare M.T. $\Rightarrow$ $\#w\#$, scanează dr-stg. și alege nedet. un pct. în care verifică $baaba$



ex:

Un număr compus $\rightarrow$ produsul a 2 nr. naturale, fiecare mai mari > 1.

$4, 6, 8, 9, 10 \rightarrow$ nr. comp.

$1, 3, 5, 7, 11 \rightarrow$ nu sunt.

$L = \{ I^n \mid n$ nr compus $\}$

1° Alege nedet. 2 nr. $p, q > 1$, transformă $\#I^n\#$ → $\#I^n\#I^p\#I^q\#$

2° Ut M.T. multiplicative → $\#I^n\#I^p\#I^q\#$ → $\#I^n\#I^{p\cdot q}\#$

3° Verifică $I^n = I^{p\cdot q}$

Notații

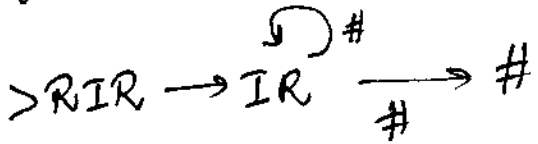
G → M.T. care generează un șir I

P → M.T. multiplicare

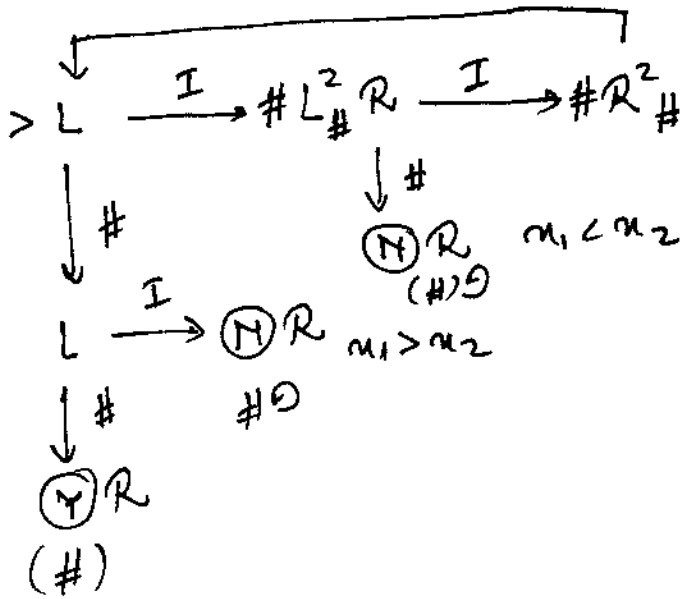
E → M.T. care testă egalit.

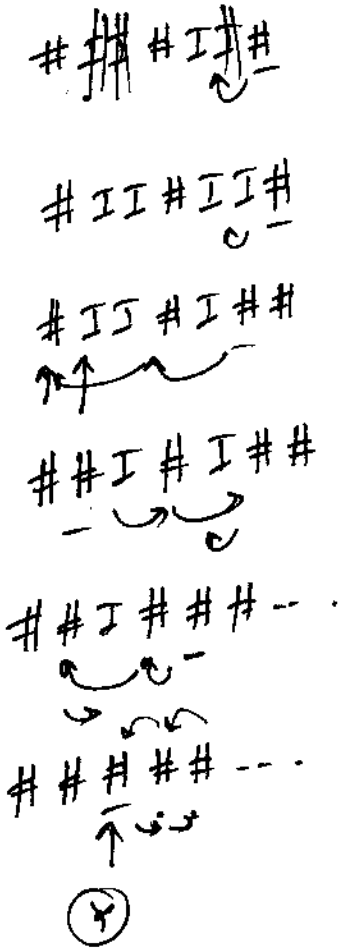
$M = GGPE$

G: generează I^p, $p \geq 2$

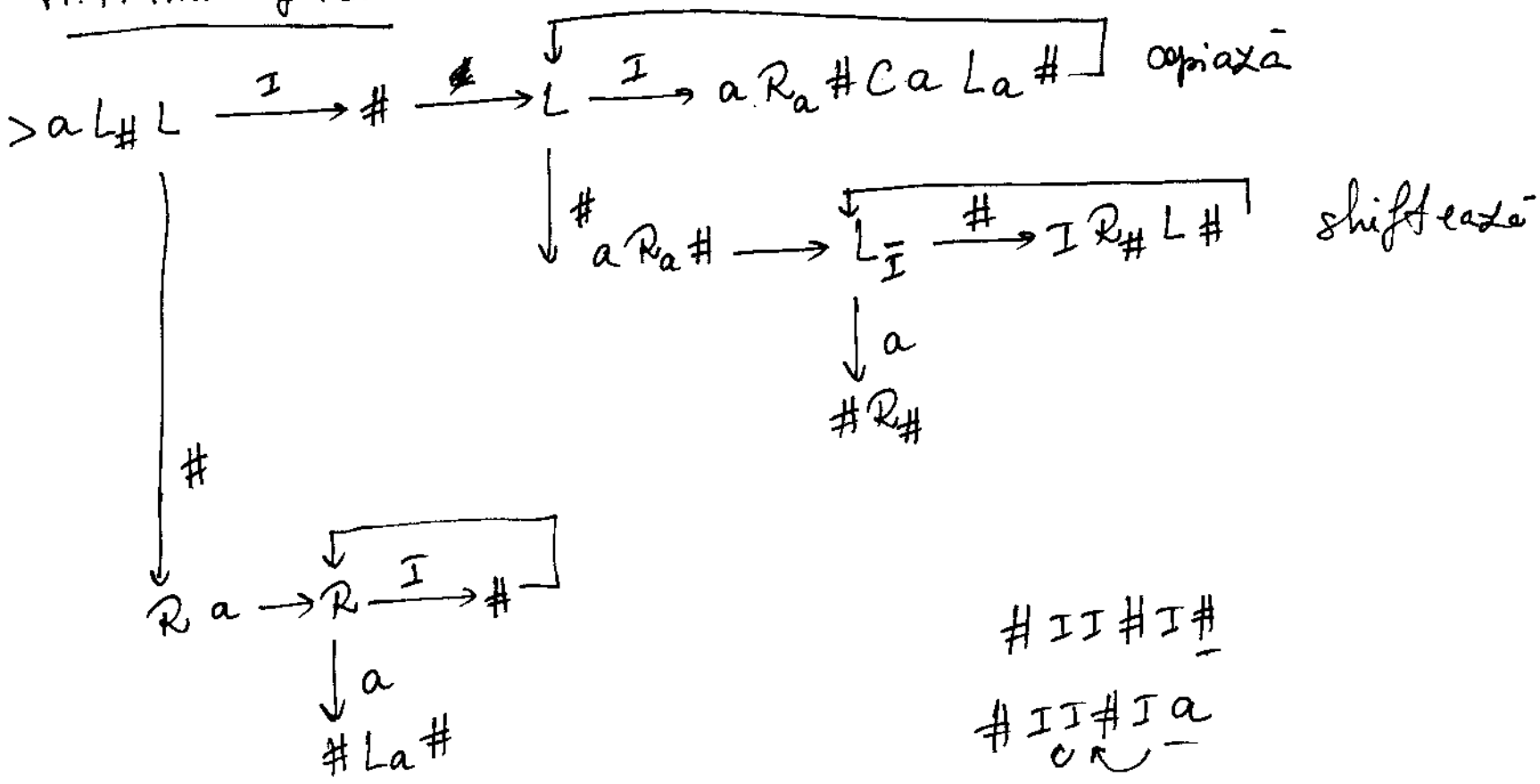


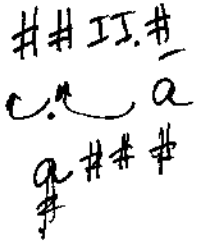
Marina E.

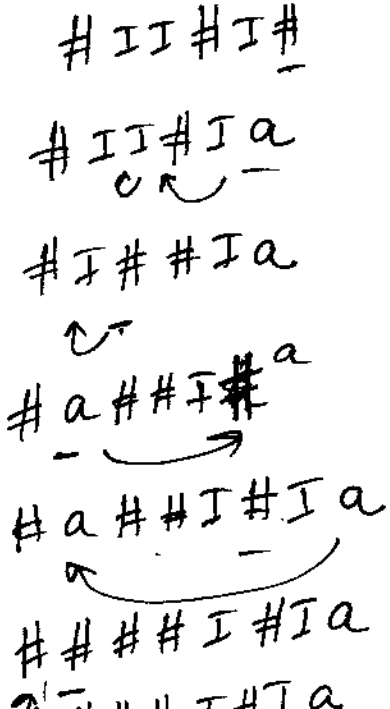




M.T. multiplicare







Teorema

Pt orice M.T.M. M_1 putem construi o M.T. standard echivalentă.

Masina Turing Universală

$M.T. \to (K, \Sigma, \delta, s)$

cum $K, \Sigma \to$ mulțimi finite $\Rightarrow$ pot să reprez. ca un șir, codificând stările și simb. de intrare peste un alfabet fixat.

Convenții

pp. că există mult. infinit numărabile

$$K_\infty = \{q_1, q_2, q_3, \dots\}$$

$$\Sigma_\infty = \{a_1, a_2, a_3, \dots\}.$$

ai pt fiec. M.T. mulțimea stărilor este o submulțime finită din K_∞, alf. de intrare $\to$ submulțime finită din Σ_∞.

Codific stările, simb de intrare ca șiruri peste alf. $\{I\}$

∇	$\lambda(\nabla)$
q_i	I^{i+1}
h	I
L	I

∇	$\lambda(\nabla)$
a_i	I^{i+2}

Fie $c \rightarrow$ alt simbol

Codificăm M.T. peste alf. $\{I, c\}$

$M = (K, \Sigma, \delta, s)$, $K \subseteq K_\infty$, $\Sigma \subseteq \Sigma_\infty$

$K = \{q_{i_1}, q_{i_2}, \dots, q_{i_k}\}$, $i_1 < i_2 < \dots < i_k$

$\Sigma = \{a_{j_1}, a_{j_2}, \dots, a_{j_e}\}$, $j_1 < j_2 < \dots < j_e$

$s = q_{i_m}$, $1 \leq m \leq k$

Def. kl şiruri $\dot{S}_{pr}$, $1 \leq p \leq k$, $1 \leq r \leq l$

$\dot{S}_{pr} \rightarrow$ codifică valoarea funcției de tranziție pt. perechea (stare, simbol), respectiv (q_{i_p}, a_{j_r})

$\delta(q_{i_p}, a_{j_r}) = (q', h)$, $q' \in K \cup \{h\}$

$h \in \Sigma \cup \{L, R\}$

$\dot{S}_{pr} = c\, w_1 c\, w_2 c\, w_3 c\, w_4 c$,

$w_1 = \lambda(q_{i_p})$

$w_2 = \lambda(q_{j_r})$

$w_3 = \lambda(q')$

$w_4 = \lambda(h)$

$f(M) = c\, S_0\, c\, S_{11} S_{12} \dots S_{1e} S_{21} S_{22} \dots S_{2e} \dots S_{k1} S_{k2} \dots S_{ke}\, c$

↑ codific. M.T.

$S_0 = \lambda(q_0)$ → codific. st. inițiale

din $f(M)$ se poate reconst M.

Pot. utiliza M.T. Universală → folosește codificarea $f(M)$ a unei alte M.T. ca program.

Intuiție → M.T. Universală (U) primește 2 arg. → o descriere a unei M.T. M și un șir de intrare w și execută operațiile pe care le-ar fi executat M.

Stări M.T. cât și w → codificate

$w = t_1 \dots t_n$, $t_i \in \Sigma_\infty$

$f(w) = c\, \lambda(t_1)\, c\, \lambda(t_2)\, c \dots c\, \lambda(t_n)\, c$

Proprietatea pe care $U = (K_U, \Sigma_U, \delta_U, s_U)$ trebuie să o aibă:

$\forall M = (K, \Sigma, \delta, s)$, $\forall w \in \Sigma^*$

1° Dc $(h, u\underline{a}v)$ este o config. de oprire a lui M ai

$(s, \#w\#) \vdash_M^* (h, u\underline{a}v)$

Atunci $(s_U, \#\mathcal{J}(M)\,\mathcal{J}(w)\#) \vdash_U^* (h, \#\mathcal{J}(uav)\#)$

2° Dc $(s_U, \#\mathcal{J}(M)\,\mathcal{J}(w)\#) \vdash_U^* (h, u'a'v')$

atunci $a' = \#$, $v' = e$, $u' = \#\mathcal{J}(uav)$

pt u, a, v ai $(h, u\underline{a}v)$ este o config. de oprire a lui M și

$(s, \#w\#) \vdash_M^\infty (h, u\underline{a}v)$

Funcționarea M.T. Universale → utilizează o M.T. cu 3 benzi U

U utilizează benzile astfel:

→ 1-ma bandă conține codific. benzii lui M

→ 2-a banda → conține codific. lui M.

→ 3a bandă → conține codific. st. lui M la pasul curent al simulării

U' → pornește cu $\#f(M)f(w)$ pe 1ma bandă și celelalte benzi goale.
U' pune $f(M)$ pe a 2a bandă și deplas. $f(w)$ la capătul stg. al primei benzi. precedat de $\#c\,f(\#)$ și încheiat $\lambda(\#)c$.

1ma bandă → $\#f(\#w\#)$

Din $f(M)$ → U' extrage codific. sf. init. a lui M → pune pe a 3a bandă.
U' simulează pașii celui lui M. Între pașii simulați, U' păstrează capetele corresp. B2, B3 la lim. stg., capul B1 în dreptul c-ului care marchează sf. codific. simb. scanat de M ext.
U' găsește pe a 2a bandă

$cc\,I^i c\,I^j c\,I^k c\,I^l cc$
(B1 ↳)

↓

șirul de I-uri care se termină la poz. ext. a capului pe B3

$Dc\,I^l \to \lambda(L)$ sau $\lambda(R)$ → mută capul cu câteva simb. stg/dr

$I^l \to \lambda(a)$, $a \in \Sigma\infty$ ⇒ în loc codific. pe B1 (cu shiftare!

În final U' pune I^k pe a 3a bandă.

Uncomputability

Teorema

Orice limbaj Turing decidabil este Turing acceptat.

dem.

Fie $L \rightarrow$ limbaj decis de M.T. M, L este acceptat de M.T. M'

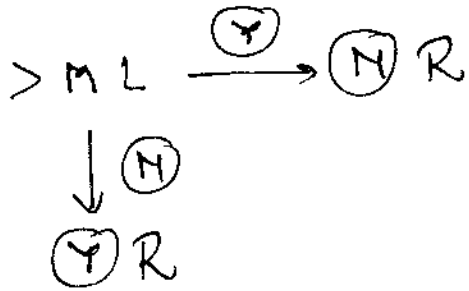


Teorema

Dacă L este Turing decid. atunci $\bar{L}$ (complement) este Turing-decid.

dem.

$L \rightarrow$ decis de M.T. M, $\bar{L} \rightarrow$ decis de M.T. $\bar{M}$



Întrebări:
Este orice limbaj Turing acceptat. Turing decidabil?
Este complementul oricărui limbaj Turing acceptabil, Turing acceptabil? → (NU)

→ Dc. M_1 ar fi o M.T. care acceptă L atunci putem proiecta o M.T. M_2 care decide L astfel: M_2 execută operațiile necesare pt. a preexiza eventual rezultat al operațiilor lui M_1 pe intrarea w;
M_2 s-ar opri cu (Y) sau (N) pe baza de în fișier de acceptă a lui w de către M_1.
Revine la întrebarea dc ∃ o super M.T. care prezice rezultatul operațiilor efectuate de M.T. arbitrare pe intrări arbitrare.
⇒ dc ∃ o "super" M.T. care prezice astfel de ceva:

$$K_0 = \{ \langle M \rangle \langle w \rangle \mid M \text{ se oprește pe șirul } w \}$$

este decis de o M.T. M_0 ⇒ ∀ L.T-acceptat este T-decidabil.

Pt orice M.T. M_1 care acceptă limb. L, putem construi o M.T. M_2 care decide L, astfel:

a) construim M.T. M_1^* care $\forall w$, transformă $\#w\#$ în $\#f(M_1)f(w)\#$

Fie $M_2 = M_1^* M_0$ => transf $\#w\#$ în $\#f(M_1)f(w)\#$ și cedează controlul lui M_0 care prin ipoteză decide dc M_1 acceptă w.

Se poate dem. $K_0 \to$ Turing acceptat => acceptat de o variantă a M.T.U.

$\forall$ limbaj Turing-acceptat este Turing decidabil (=) K_0 Turing acceptat este Turing decid.

<u>Anătăm</u> $K_0 \to$ <u>nu</u> este Turing decid.

Pp. $K_0 \to$ Turing decid. =>

$K_1 = \{f(M) \mid M$ acceptă $f(M)\}$ Turing decid.

Dc. M_0 ar decide K_0, atunci M_1 care decide K_1 poate fi construită => transformă $\#w\# \Rightarrow \#wf(w)\#$ și cedează contr. lui M_0.

M_1 obține același rez. pt $\#w\#$ ca M_0 pt $\#wf(w)\#$

Din def. K_0, M_0 obț ⑦ pt $\#w f(w) \# \Leftrightarrow$:

a) w este $f(n)$ pt o anu. M.T. M

b) M.T. M acceptă w, adică $f(M)$

Dar asta este def. lui K_1. Este suf. să dem. K_1 nu este decid. Turing

pp. K_1 → Turing decid. ⇒ $\bar{K}_1$ Turing-decid.

$\bar{K}_1 = \{ w \in \{I, c\}^* \mid$ w nu este codific nicăruei M.T. M
sau $w = f(n)$ pt o anu. M.T. M care nu
acceptă $f(M) \}$

Dar $\bar{K}_1$ → nu poate fi Turing acceptat.

Să pp. că M^* este o M.T. care acceptă $\bar{K}_1$. Este $f(M^*)$ în $\bar{K}_1$?

Din def. lui $\bar{K}_1$, $f(M^*) \in \bar{K}_1 \Leftrightarrow M^*$ nu acceptă $f(M^*)$.

Dar M^* s-a pp. că $\bar{K}_1$, $f(M^*) \in \bar{K}_1 \Leftrightarrow M^*$ acceptă $f(M^*)$

⇒, M^* acceptă $f(M^*) \Leftrightarrow M^*$ nu acceptă $f(M^*)$ → absurd.

Teorema

Nu orice limbaj Turing acceptat este Turing decidabil.

Teorema

Compl. unor limbaje Turing accept nu sunt Turing acceptate.
(pt că K_1, K_0 → Turing accept. dar $\bar{K}_1$ nu este Turing acceptat.)

Problemă care nu admite sol. alg. ⇒ problema opririi M.T.,
descrisă de K_0.

Teorema

a) Nu există nicium alg. care să dec. dată fiind o M.T. M și un șir w,
dc M acceptă w.

b) Pt o M.T. fixată M_0, nu există nicium alg. care să dec. dat fiind
w, dc M_0 acceptă w.

$K_0 = \{\langle M \rangle \langle w \rangle \mid M \text{ acceptă } w\}$

K_0 → poate fi spart în infinit de multe submulțimi

$K_M = \{\langle M \rangle \langle w \rangle \mid M \text{ acceptă } w\}$

câte una pt fiec M.T. M.

Nu rezultă imediat din faptul că K_0 → nu este Turing decid.

că oricare submulțime K_M nu este Turing decid.

ex. limbajul care constă dintr-un sg. șir ⇒ deis de M T. care verifică că intrarea este acel șir

Nu există o met. generală de a combina inf. de multe proc de decizie într-una sing.

Turing enumerabilitate, Turing acceptare, Turing decidabilitate

Teorema

Un limbaj este Turing decidabil $\Leftrightarrow$ atât el cât și complementul sunt Turing acceptate.

dem.

$\Rightarrow$ L Turing decidabil $\Rightarrow$ L Turing acceptat (Th.)

$\Rightarrow \bar{L}$ Turing decidabil (Th) $\Rightarrow$ Turing acceptat (Th)

$\Leftarrow$ Să pp. L acceptat de M.T. M_1 și $\bar{L}$ este acceptat de M.T. M_2. Putem construi o M.T. M care <u>decide</u> L : va fi o M.T. cu 2 benzi (Th. echiv. cu M.T.S)

M pune și mul de intrare w pe ambele benzi și pune capetele la stg. w.

M simulează M_1 și M_2 în paralel; la fiec. pas al unei apliu a lui M_i, un pas al lui M_1 este pus pe 1 ma bandă și un pas al lui M_2 este pus pe a 2a banda. Cum fie M_1 sau M_2 acceptă w, <u>nu</u> amândouă, M_1 sau M_2 va opri. M determină care dintre M_1 sau M_2 se oprește, M se oprește cu benzile

config. pt. a indica $w \in L$ sau $w \notin L$.

Def.
Fie Σ_0 un alfabet care nu conține #. Un limbaj $L \subseteq \Sigma_0^*$ este limbajul de ieșire al unei M.T. M $\iff$ L este mulțimea șirurilor $w \in \Sigma_0^*$ cu. pt un anumit șir $u \in \Sigma_0^*$, $(s, \#u\#) \vdash_M^* (h, \#w\#)$. ($s \to$ stare inițială a M.T. M).

Teorema
Un limbaj este Turing acceptat $\iff$ este limbajul de ieșire al unei M. Turing.

Dem:
$L \to$ limbaj de ieșire al unei M.T. M $\Rightarrow$ L acceptat de o au. M.T.
Din Th. de echiv. M.T. Δ cu M.T.N. este suficient să dem. că L este acceptat de o MTN.
Proiectăm M.T.N. M' care-l acceptă pe L: pt. un șir de intr w, M' scrie nedeterminist u pe bandă, salvând w pe a 2a pistă a benzii. Atunci când M se oprește, M' compară și cel calc. de M cu w. Dc. sunt același, atunci M' acceptă, altfel intră într-un ciclu infinit.

L → mulțimea șirurilor acceptate de o M.T. M, putem modifica M așa încât la acceptarea unui șir de intrare să se oprească în acel șir pe bandă. ⇒ Mulțimea șirurilor rămase pe bandă la sf. opțiunilor este aceeași cu mulțimea șirurilor acceptate de M ⇒ L.

Def.
O M.T. enumeră limbajul L ⇔ $\exists$ o anumită stare finală q,

$$L = \{ w \mid \exists \text{ un șir } u, (s, \#) \vdash_M^* (q, \# w \underline{\#} u) \}$$

unde s → st. inițială a lui M.

Def.
Un limbaj este Turing-enumerabil ⇔ este enumerat de o anu. M.T. M.

Teorema
Un limbaj este Turing-acceptat ⇔ Turing-enumerabil.

Dem:
⇒ L → Turing-acceptat de o M.T. M.
Putem construi o M.T. M' care pornește cu banda vidă, generează sistematic (ordine lexicografică) toate șirurile peste alfabetul lui L, și efectuează

operațiile pe care le-ar fi făcut M.
!!!! M' s-ar putea să nu termine ciclile pe fec. șir înainte de a ajunge la unu.
pt. că pt un anumit șir ciclile lui M nu se termină, deși există șiruri pe
care M le acceptă dar nu au fost încă generate.
M' ⟹ va funcționa astfel:
(1) M' execută un pas al ciclului lui M asupra 1 rului șir peste alf. lui M.
(2) M' execută 2 pași din ciclele lui M pe fec. dintre cele 2 șiruri
(3) M' execută 3 pași - - -
⋮
De fiecare dată când M' descoperă că M ar fi acceptat un șir, se oprește
în starea 2 pt a semnala acceptarea.

⟸ Dacă M.T. M enumeră L, putem modifica M să să accepte L: reconstruim M să
să salveze orice intrare primită înainte să înceapă enumerarea. De fec. dată
când M intră în st. 2, mașina modific. compară conținutul benzii cu
șirul de intrare salvat. Dc sunt la fel, șirul de intrare este acceptat,
altfel enumerarea continuă.

Teorema
Un limbaj este Turing acceptat <=> generat de o gr. gramatică.

Complexitatea algoritmilor

Hilbert - 1900

polinom: $5x^2yz^3 + 7xy^2 - x^5 - 10$.

a 10-a problemă enunțată de Hilbert → alg. care să verifice dc. un polinom are rad. întregi.

până în 1936 → def. intuitivă a noțiunii de alg.

1936 → Church și Turing au formalizat noțiunea de alg.

↓ ↓

calcul λ M. Turing

└ s-a dem. echiv. lor

1970 → Yuri Matijasevic → arată că nu există alg. pt pr 10 enunțată de Hilbert

$D = \{p \mid p$ este un polinom cu rad. întreagă$\}$.

? D este decidabil

$D \rightarrow$ este Turing acceptat.

$D_1 = \{p \mid p$ este un polinom peste x, cu rad. întreagă$\}$.

M.T. M_1 care acceptă D_1.

$M_1 \Rightarrow$ input: $f(p)$, $p \rightarrow$ polinom de var. x

evaluează p cu valorile $0, 1, -1, 2, -2, 3, -3, \ldots$

Dacă $p(x)$ devine $0 \Rightarrow M_1$ acceptă

pt $D \rightarrow$ M.T. M care acceptă $\Rightarrow$ funcț. analog. M.T. M_1.

$M_1 \rightarrow$ poate decide D_1 pt. că există a Th $\rightarrow$ rad. unui $p(x)$ se află în

$$\pm K \frac{C_{max}}{C_L}$$

Problemă: vrem să planificăm vizitele unui comis voiajor în 10 orașe; se dă harta și distanțele lor ⇒ itinerariul care minimizează întreaga dist.

n orașe → nr. itinerariilor posibile $(n-1)!$

10 orașe → $9! \leq 360000$

30 orașe → $29! >> 10^{30}$

Complexitate

Def.
Dacă M este o M. Turing, $t \in \mathbb{N}$, def. relația $\vdash_M^t$ între config. lui M, astfel:

$$C \vdash_M^0 C' \iff C = C'$$

$$C \vdash_M^{t+1} C' \iff$$

$\exists\, C''$ a.î. $C \vdash_M^t C''$ și $C'' \vdash_M C'$

Adică

$$C \vdash_M^t C'$$

$\iff C \vdash_M^* C'$ printr-un calcul (operație) în <u>exact</u> t pași.

Def.

Fie $T : \mathbb{N} \to \mathbb{N}$, $L \subseteq \Sigma_0^*$ un limbaj, $M = (K, \Sigma, \delta, s)$ o M.T. cu K benzi și $\Sigma_0 \subseteq \Sigma$. Spunem că M decide L în timpul T, dacă

$\forall w \in L$, $(s, \#w\underline{\#}, \underline{\#}, \dots, \underline{\#}) \overset{t}{\underset{M}{\vdash}} (h, \#\textcircled{Y}\underline{\#}, \underline{\#}, \dots, \underline{\#})$, $t \leq T(|w|)$

și $w \notin L$

$(s, \#w\underline{\#}, \underline{\#}, \dots \underline{\#}) \overset{t}{\underset{M}{\vdash}} (h, \#\textcircled{N}\underline{\#}, \underline{\#}, \dots, \underline{\#})$, $t \leq T(|w|)$

Spunem că L este decidabil în timpul T, dc. $\exists K > 0$ și o M.Turing cu K benzi, care decide L în timpul T.

Notație

$\mathrm{TIME}(T)$ – clasa limbajelor decidabile în timpul T.

Simulări limitate în timp

Teorema

Fie L un limbaj decis de o M.T. M cu bandă nelimitată stg/dr în timpul T_1. Atunci L este decis de o M.T. standard M_2 în timpul T_2, $\forall n \in \mathbb{N}$, $T_2(n) = 6T_1(n) + 3n + 8$.

Teorema

Fie L un limbaj decis în timpul T_1 de o M.T. cu $K > 0$ benzi, $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$. Atunci L este decis de o M.T.S. $M_2 = (K_2, \Sigma_2, \delta_2, 1_2)$ în timpul T_2, $\forall n \in \mathbb{N}$, $T_2(n) = 4T_1(n)^2 + (4n + 4K + 3)T_1(n) + 5n + 15$.

Corolar

Dacă L este un limbaj decis în timpul T de o M.T. cu $K > 0$ benzi, L este decis în $T' = O(T^2)$ de o M.T. cu 1 bandă.

Def.

Def. $\mathcal{P}$ (decidabil în timp polinomial),

$$\mathcal{P} = \bigcup \{ TIME(n^d) : d > 0 \}$$

unde $TIME(n^d)$ conține toate limbajele decise de o m. T. cu timp cuth-o limită de timp care este un polinom de grad 'd'.

Considerând reuniunea peste toate gradele → clasa limbajelor decise de M.T. în limite polinomiale.

Def.

Fie $T : \mathbb{N} \to \mathbb{N}$, $L \subseteq \Sigma_0^*$ un limbaj și $M = (K, \Sigma, \Delta, \Delta)$ o M.T.N.

M acceptă L redus în timpul T, dacă

$\forall w \in \Sigma_0^*$, $w \in L \iff (\Delta, \#w\#) \overset{\leq t}{\underset{M}{\vdash}} (h, v \underline{\nabla} u)$

$u, v \in \Sigma^*$, $\nabla \in \Sigma$, $t \leq T(|w|)$

$\Sigma_0 \subseteq \Sigma$.

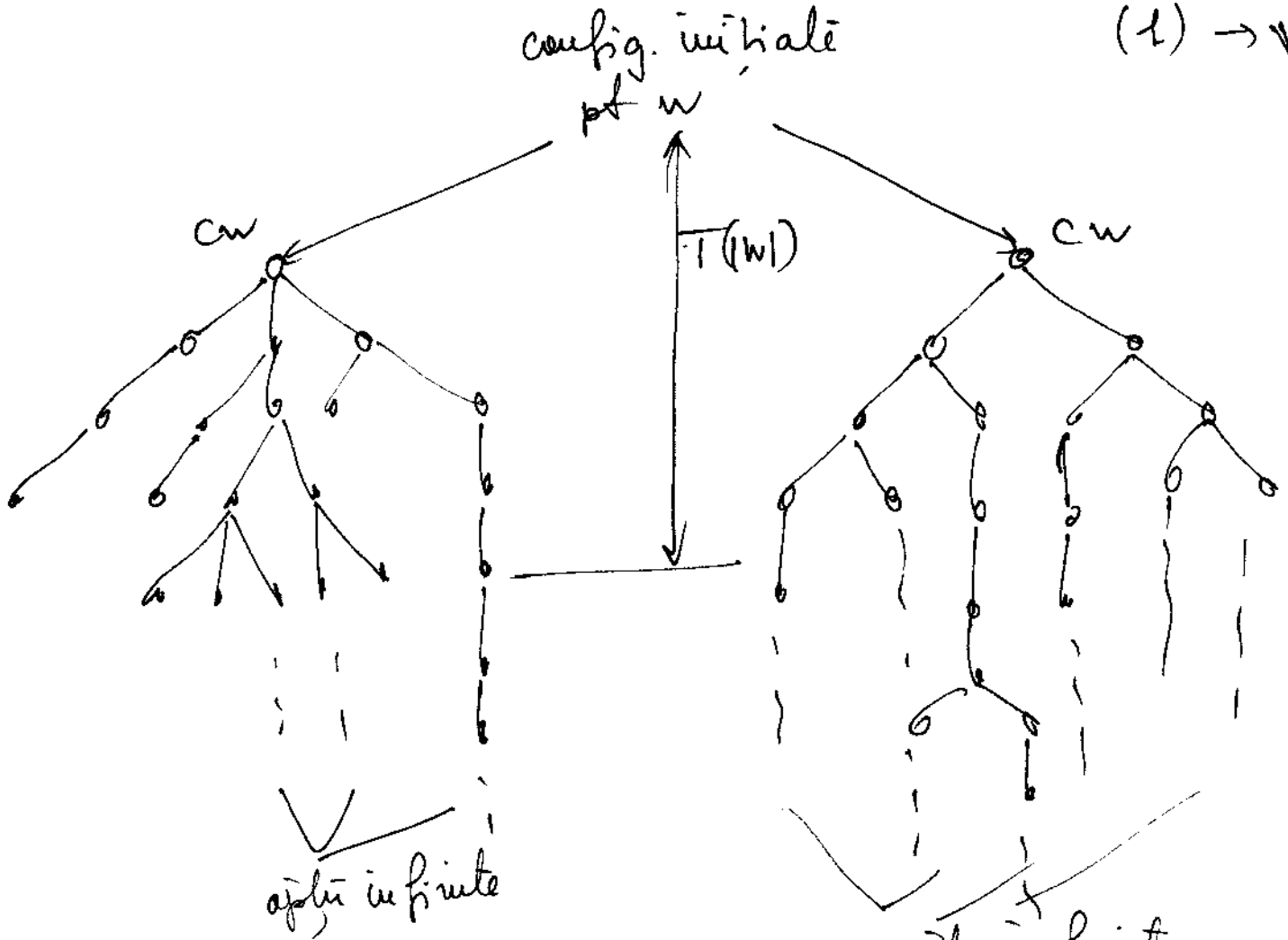
Notație

$NTIME(T)$ → clasa limbajelor acceptate nedet. în timpul T

$NP = \bigcup_{d} NTIME(n^d) : d > 0$

(a) → w este acceptat

(r) → w nu este accept.



Lema

Dacă L este acceptat în timpul T_1 de o M.T.M. cu k benzi M_1, atunci există o M.T.M. cu 1 bandă M_2 care acceptă L în $T_2 = O(T_1^2)$.

Complexitate

$TIME(T) \subseteq NTIME(T^2) \Rightarrow \mathcal{P} \subseteq \mathcal{NP}$

$? \; \mathcal{P} = \mathcal{NP}$

Def:
$f: \mathbb{N} \to \mathbb{N}$ este o funcție de numărare a pașilor dc $\exists k \geq 1$ și o M.T unig cu k benzi, M, a.î. $\forall w \in \Sigma^*$, M se oprește în $f(|w|)$ pași,

$$(\Delta, \#w\#, \#, \ldots, \#) \vdash_M^{f(|w|)} (h, u_1 a_1 v_1, u_2 a_2 v_2, \ldots, u_k a_k v_k)$$

$u_1, \ldots, u_k, v_1, \ldots, v_k \in \Sigma^*$, $a_1, \ldots, a_k \in \Sigma$.

Teorema
Fie L un limbaj acceptat în $T_1 - T_1$ funcție de numărare a pașilor - de o M.T.N. $M_1 = (K_1, \Sigma_1, \Delta_1, s_1)$. Atunci există $k_2 \geq 1$ și $k > 1$ și o MTD $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ cu k_2 benzi care decide L în timpul T_2, $T_2(n) = n^{T_1(n)}$.

Corolar

$\mathcal{PS} \subseteq \bigcup \{ TIME(n^d) : n, d > 0 \}$

Def.

Fie $L \subseteq \Sigma^* \$ \Sigma^*$ un limbaj, $\$ \notin \Sigma$. Spunem ca L este balansat polinomial, dc există un polinom p a.î. $x\$y \in L$, numai dc $|y| \leq p(|x|)$.

In particular, dc $L \subseteq \Sigma^* \$ \Sigma^*$, $\$ \notin \Sigma$, $L \setminus \$\Sigma^*$ este multimea tuturor sf. lor $x \in \Sigma^*$ a.î. $x\$y \in L$, $y \in \Sigma^*$.

Teorema

Fie $L \subseteq \Sigma^*$ un limbaj, $\$ \notin \Sigma$, $|\Sigma| \geq 2$. Atunci $L \in \mathcal{PS} \iff$ există un limbaj polinomial balansat $L' \subseteq \Sigma^* \$ \Sigma^*$ a.î. $L' \in \mathcal{P}$ și $L = L' \setminus \$\Sigma^*$.

ex: formalizarea problemei comis voiajorului

Datele:

a) $n \rightarrow$ nr. orase vizitate , $1, 2, .., n$

b) distanta intre fiecare 2 orase $\rightarrow$ matrice $n \times n$, D , $d_{ij} \rightarrow$ distanta intre orasele i si j

(nu impunem $d_{ij} = d_{ji}$)

Obs: Reprez. D prin codific. ei

Cerinta $\rightarrow$ s.s. gaseasca drumul cel mai scurt

$$t : \{1, 2, .., n\} \rightarrow \{1, 2, .., n\}$$

$\uparrow$ tur

$$\Delta(t) = \sum_{j=1}^{n-1} d_{t(j)\,t(j+1)} + d_{t(n)\,t(1)}$$

$\uparrow$

lungime tur

Obs:
Determinarea lungimii t pt. care $\Delta(t) \rightarrow$ minimă nu este o problemă de decizie, ci de evaluare de funcție. Pt a formula problema în termeni de decizie pp. că avem $k \rightarrow$ limită și $\forall$ t ai $\Delta(t) \leq k$.

$$PCV = \{ I^n \# \alpha(\Delta) \# I^k : \forall t \text{ ai } \Delta(t) \leq k \}$$

Nu știm dc $PCV \in \mathcal{P}$. Toți alg. amoscați $\rightarrow$ timp exponențial.

Putem arăta $PCV \in NP$.

Este suf. să obs. că:

$$PCV' = \{ I^n \# \alpha(\Delta) \# I^k \# \alpha(t) : \Delta(t) \leq k \}.$$

este polinomial falansat și este în $\mathcal{P}$.

$$\alpha(t) = I^{t(1)} c I^{t(2)} c \dots c I^{t(n)}$$

$PCV' \rightarrow$ polinomial falansat pt ca $|\alpha(t)| \leq n^2$

Dat fiind t și Δ se poate evalua $\Delta(t)$ pt. a decide dc $\Delta(t) \leq k$ în timp polinomial $\Rightarrow PCV \in NP$.

NP Completitudine

Def:
Fie Σ, Δ alfabete, $f: \Sigma^* \to \Delta^*$ este calculabilă în timpul T printr-o M.T.D. $M = (K, \Sigma', \delta, s)$ cu k benzi $\Leftrightarrow$

$\forall x \in \Sigma^*$

$(s, \#x\underline{\#}, \underline{\#}, \dots, \underline{\#}) \overset{t}{\underset{M}{\vdash}} (h, \#f(x)\underline{\#}, \underline{\#}, \dots, \underline{\#})$, $t \leq T(|x|)$

Spunem că f - polinomial calculabilă dacă $\exists$ un polinom T a.î. f este calculabilă în T.

Dacă f este calculabilă în T, $\forall x \in \Sigma^*$, $|f(x)| \leq T(|x|) + |x|$

ex:
Dacă $L \in \mathcal{P}$ atunci f_L:

$$f_L(x) = \begin{cases} \textcircled{1}, & x \in L \\ \textcircled{0}, & x \notin L \end{cases}$$

este din def. $\mathcal{P}$ calc - polinomial

Def.
Fie $L_1 \in \Sigma_1^*$, $L_2 \subseteq \Sigma_2^*$ limbaje. O funcție calculabilă polinomial
$\mathcal{C}: \Sigma_1^* \to \Sigma_2^*$ este numită transformare în timp polinomial din L_1 în L_2
$\iff \forall x \in \Sigma_1^*,\ x \in L_1 \iff \mathcal{C}(x) \in L_2$.

Lemă
Dc. $\mathcal{C}_1: \Sigma_1^* \to \Sigma_2^*$ și $\mathcal{C}_2: \Sigma_2^* \to \Sigma_3^*$ sunt transformări în timp polinomial
din L_1 în L_2 și, din L_2 în L_3 atunci $\mathcal{C}_1 \circ \mathcal{C}_2 : \Sigma_1^* \to \Sigma_3^*$ este o transformare
în timp polinomial din L_1 în L_3.

Dem.
$\mathcal{C}_1 \to$ calc de M.T. M_1 în timp polinomial T_1
$\mathcal{C}_2 \to$ calc de M.T. M_2 în timp polinomial T_2

$\mathcal{C}_1 \circ \mathcal{C}_2 \to$ calc. $M_1 M_2 : x \in \Sigma_1^*$, $M_1 M_2$ va calc $\mathcal{C}_1 \circ \mathcal{C}_2(x)$ în M-o
limită de timp $T_1(|x|) + T_2(T_1(|x|) + |x|) \Rightarrow$ l. polinomială

$x \in L_1 \Leftrightarrow \mathcal{C}_1 \circ \mathcal{C}_2(x) \in L_3$

$x \in L_1 \Leftrightarrow \mathcal{C}_1(x) \in L_2 \Leftrightarrow \mathcal{C}_2(\mathcal{C}_1(x)) = \mathcal{C}_1 \circ \mathcal{C}_2(x) \in L_3$.

Def. 1

Un limbaj $L \subseteq \Sigma^*$ este numit $\mathcal{NPS}$ complet, dacă și numai dacă

a) $L \in \mathcal{NPS}$

b) $\forall L' \in \mathcal{NPS}$, există o transformare în timp polinomial de la L' la L.

Teorema

Fie L un limbaj $\mathcal{NPS}$ complet. Atunci $\mathcal{P} = \mathcal{NPS} \Leftrightarrow L \in \mathcal{P}$.

Dem:

$\Rightarrow$

pp. $\mathcal{P} = \mathcal{NPS}$. Cum L este $\mathcal{NPS}$ complet din def. 1 $\Rightarrow L \in \mathcal{NPS} \overset{ip}{=} L \in \mathcal{P}$.

$\Leftarrow$

$L \in \mathcal{P} \Rightarrow$ pp. L este decis de o M.T.D. M_1 în timpul polinomial T_1.

Fie L' un limbaj în $\mathcal{NPS}$; idee $\rightarrow$ arăt că $L' \in \mathcal{P}$.

Cum L este $\mathcal{NPC}$ complet $\Rightarrow$ $L' \in \mathcal{NP}$ există o transf. în timp polinomial $\mathcal{C}$ de la L' la L. Pp. că $\mathcal{C}$ este calc. de o M.T. M_2 în timpul polinomial T_2.
Atunci M.T. $M_2 M_1$ decide L' în timp polinomial.
$M_2 M_1$ se oprește cu (T) pe x dacă pt $x \Leftrightarrow \mathcal{C}(x) \in L$.
Cum $\mathcal{C}$ este o transf. în timp polinomial, $\mathcal{C}(x) \in L \Leftrightarrow x \in L'$
$M_2 M_1$ se oprește pt x în $T_2(|x|) + T_1(T_2(|x|) + |x|)$ care este polinomial în $|x|$
Cum L' poate fi orice limbaj în $\mathcal{NP} \Rightarrow L' \in \mathcal{P} \Rightarrow \mathcal{P} = \mathcal{NP}$.

? cum găsim un limbaj $\mathcal{NPC}$ complet.
Soluție $\Rightarrow$ pornim de la construcția K_0

$K_0 = \{ \langle M \rangle \langle w \rangle 0 \, I^t : \text{MTN } M \text{ care acceptă } w \text{ în cel mult } t \text{ pași} \}$.

Teorema
K_0 este $\mathcal{NPC}$ complet.